マルチクラウド自動デプロイプラットフォーム

日本語解説書

PLAYER1-r7

2026年2月27日

Contents

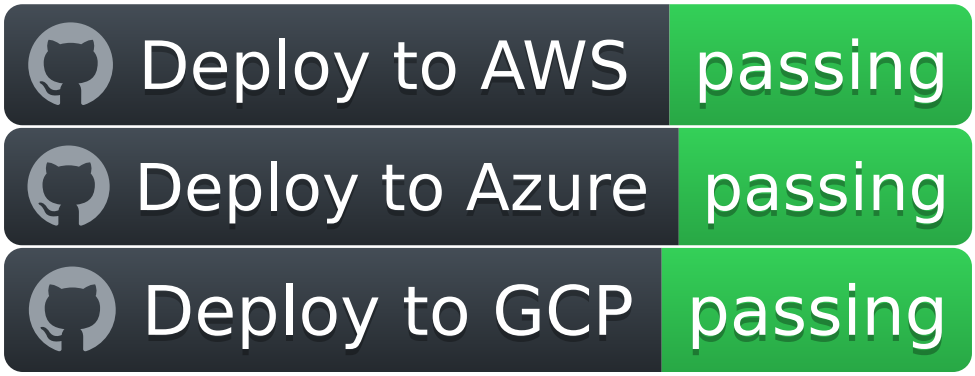
1　プロジェクト概要　2
2　重要ルール（日本語訳）　14
3　コンテキスト・概要（日本語訳）　20
4　アーキテクチャ（日本語訳）　26
5　API とデータモデル（日本語訳）　31
6　インフラ・Pulumi 解説（日本語訳）　35
7　CI/CDパイプライン（日本語訳）　40
8　ステータス・環境情報（日本語訳）　49
9　ソース実装解説書　67
10 インフラ・CI/CD解説書　79
11 カスタムドメイン設定　87
12 ステージング環境テスト　96
13 統合テストガイド　103
14 Lambda Layer最適化　111
15 アーキテクチャマッパー　117

Chapter 1

プロジェクト概要

Multi-Cloud

Figure 1.1: Multi-Cloud



マルチクラウド対応の自動デプロイシステム - AWS/Azure/GCP対応のフルスタックアプリケーション自動デ

Cloud Provider	API Endpoint	Frontend (CDN)	Direct Storage
AWS (ap-northeast-1)	API Gateway	CloudFront ☒	S3 Static
Azure (japaneast)	Functions	Front Door ☒	Blob Storage
GCP (asia-northeast1)	Cloud Functions	Cloud CDN ☒	Cloud Storage

Cloud Provider	Resource	セキュリティ実装	状態
AWS	CloudFront + Lambda	CORS 絞り込み + CloudTrail	☒ 本番反映
Azure	Key Vault + Functions	Purge Protection + Managed Identity + 診断ログ	☒ 本番反映
GCP	Cloud Functions	HTTPS redirect + Audit Logs + Cloud Armor	☒ 本番反映

☒ **IAM/RBAC 最小権限の法則適用済み**: デプロイユーザーと管理者を明確に分離 → 詳細: CRITICAL_RULES — Rule 16 | セキュリティガイド

☒ **全クラウドでCDN配信を実装！** CloudFront, Front Door, Cloud CDNによる高速・安全なコンテンツ配信

☒ **Infrastructure as Code**: Pulumiで全CDNリソースを管理（詳細: CDNセットアップガイド）

☒ 詳細情報: エンドポイント一覧

☒ **2026-02-27 更新**: S1 セキュリティ全本番反映完了 / S2 Managed Identity 有効化 / Task 20/21 Key Vault & Defender 実装済み

☒ **2026-02-22 Refactoring**: Infra & CI/CD Refactoring Report — Azure AFD SPA fix, workflow cleanup, Pulumi dead code removal, staging validation

- **マルチクラウド対応**: AWS、Azure、GCPに対応
- **フルスタック**: フロントエンド、バックエンド、データベースの完全なスタック
- **自動デプロイ**: GitHub Actionsによる完全自動化
- **IaC統合**: Pulumi 3.0+による Infrastructure as Code
- **完全Python版**: Pulumi + FastAPI + Reflexによる統一スタック
- **CI/CD**: プッシュやPRで自動的にビルド・デプロイ
- **簡単セットアップ**: スクリプト一つで環境構築

```
multicloud-auto-deploy/ ├──
 .github/workflows/     # GitHub ワークフローActions ├──
 infrastructure/        # インフラストラクチャコード │
    └── pulumi/           # コード（PulumiPython - AWS/Azure/）GCP ├──
 services/             # アプリケーションコード │
    ├── api/              # FastAPI バックエンド（Python ）3.12 │
    ├── frontend_react/   # React フロントエンド（Vite + ）TypeScript │
    └── frontend_reflex/  # Reflex フロントエンド（実験的） ├──
 scripts/              # デプロイ・テストスクリプト ├──
 docs/                 # ドキュメント │
    ├── CDN_SETUP.md                      # 設定ガイドCDN │
    ├── ENDPOINTS.md                      # エンドポイント一覧 │
    └── REFACTORING_REPORT_20260222.md    # 🆕 Infra & CI/CD refactoring report
    (2026-02-22) └──
 static-site/          # 静的サイト（環境選択画面）
```

- Python 3.12+
- Docker & Docker Compose
- Pulumi 3.0+
- AWS CLI 2.x / Azure CLI 2.x / gcloud CLI 556.0+
- GitHub アカウント

Frontend

- **Framework**: React 18+ (Vite)

- **Hosting**: Static Site (S3 / Azure Blob / Cloud Storage)
- **CDN**: CloudFront / Azure Front Door / Cloud CDN
- **Build**: Vite 7.3+, TypeScript

Backend

- **Framework**: FastAPI 1.0+ (Python 3.12)
- **AWS**: Lambda (x86_64) + API Gateway v2 (HTTP)
- **Azure**: Azure Functions (Python)
- **GCP**: Cloud Functions (Python 3.11)

Database

- **AWS**: DynamoDB (PAY_PER_REQUEST)
- **Azure**: Cosmos DB (Serverless)
- **GCP**: Firestore (Native Mode)

Infrastructure

- **IaC**: Pulumi 3.0+
  - Infrastructure as Code管理（`infrastructure/pulumi/`）
    - AWS: Lambda + API Gateway
    - Azure: Functions + Cosmos DB
    - GCP: Cloud Functions
- **CI/CD**: GitHub Actions

CI/CD

- GitHub Actions
- Automated builds and deployments
- Docker-based deployments
- S3-based Lambda deployment

1. **リポジトリをクローン**

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. **ローカル開発環境起動**

```
docker-compose up -d api

cd services/frontend_react
npm install
npm run dev
```

3. Pulumiでデプロイ

```
cd infrastructure/pulumi/aws/simple-sns
pip install -r requirements.txt
pulumi stack init staging
pulumi config set aws:region ap-northeast-1
pulumi up
```

☒ 詳細な移行ガイドは docs/PYTHON_MIGRATION.md を参照

1. リポジトリをクローン

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. GitHub Secretsを設定

```
./scripts/setup-github-secrets.sh
```

3. コードをプッシュして自動デプロイ

```
git push origin main
```

- ☒ セットアップガイド - 初期セットアップ手順
- ☒ CI/CD設定 - GitHub Actions自動デプロイ設定
- ☒ CI/CDテスト結果 - パイプライン検証レポート
- ☒ トラブルシューティング - よくある問題と解決策
- ☒ エンドポイント一覧 - 全環境のエンドポイント情報
- ☒ CDNセットアップガイド - CloudFront/Front Door/Cloud CDN設定
- ☒ 関数サイズ最適化 - デプロイパッケージサイズ削減（AWS 97%削減！）
- ☒ クイックリファレンス - よく使うコマンド集
- ☒ セキュリティ実装 - Key Vault / Managed Identity / WAF / CloudTrail / Audit Logs
- AWS デプロイ
- Azure デプロイ
- GCP デプロイ
- システムアーキテクチャ - 完全版システム設計
- クラウドアーキテクチャ可視化 - 公式アイコン付きインタラクティブHTML図生成
  - ☒ Staging環境 - AWS/Azure/GCP公式アイコン統合版
  - ☒ Production環境 - 本番環境構成図
  - ☒ 統合ビュー - Staging + Production比較図

プッシュやPRで自動的にビルド・デプロイが実行されます：

- `main`ブランチへのプッシュ → 全3クラウド本番デプロイ（AWS/Azure/GCP）
- `develop`ブランチへのプッシュ → ステージング環境へデプロイ
- PRの作成/更新 → ビルド検証
- 手動トリガー → 任意の環境へデプロイ

2026-02-27 デプロイ状況:

- ☒ AWS production: 9/0 SNS E2E test PASS
- ☒ Azure production: 17/0 SNS E2E test PASS
- ☒ GCP production: 13/0 SNS E2E test PASS
- ☒ 全環境セキュリティ実装完了（Pulumi反映）

ワークフロー	トリガー	デプロイ先	説明
deploy-aws.yml	`main`へのpush / 手動	AWS Lambda	Lambda + API Gateway更新
deploy-azure.yml	`main`へのpush / 手動	Azure Functions	Functions + Cosmos DB更新
deploy-gcp.yml	`main`へのpush / 手動	GCP Cloud Functions	Cloud Functions更新

以下のシークレットを設定してください（詳細は CI/CD設定ガイド 参照）：

Pulumi（必須）

- `PULUMI_ACCESS_TOKEN` - Pulumi Cloud認証トークン（すべてのデプロイで必須）

Azure Functions

- `AZURE_CREDENTIALS` - Service Principal認証情報（JSON）
- `AZURE_SUBSCRIPTION_ID` - Azure Subscription ID
- `AZURE_RESOURCE_GROUP` - リソースグループ名

GCP Cloud Functions

- `GCP_CREDENTIALS` - サービスアカウントキー（JSON）
- `GCP_PROJECT_ID` - プロジェクトID

AWS Lambda

- `AWS_ACCESS_KEY_ID` - AWS認証キーID
- `AWS_SECRET_ACCESS_KEY` - AWS認証シークレットキー

最新のデプロイ状況はGitHub Actionsで確認でき、詳細タスク進捗も公開中です。

最新ステータス（2026-02-27）： | タイプ | 対象 | 状態 | |——|——|——| | インフラ整備 | AWS/Azure/GCP Pulumi本番 | ☒ デプロイ完了 | | セキュリティ実装 | CORS / CloudTrail / Key Vault / Managed Identity / Audit Logs | ☒ 本番反映完了 | | E2E テスト | 全クラウド SNS 統合テスト | ☒ 39/39 PASS | | 認証 | AWS Cognito / Azure AD / Firebase | ☒ 全完了 | | CDN | CloudFront / Front Door / Cloud CDN | ☒ 全完了 |

GitHub Actionsページから手動でワークフローを実行：

```
Actions > Deploy to Multi-Cloud > Run workflow

- environment: staging / production
- deploy_target: all / azure / gcp
```

- **Frontend**: S3 Static Website Hosting
- **Backend**: Lambda (Python 3.12) + API Gateway v2
- **Database**: DynamoDB (PAY_PER_REQUEST)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **Frontend**: Blob Storage ($web) + Azure Front Door
- **Backend**: Azure Functions (Python 3.12)
- **Database**: Cosmos DB (Serverless)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **CDN**: Azure Front Door (Standard)
- **Frontend**: Cloud Storage Static Website
- **Backend**: Cloud Functions (Python 3.12)
- **Database**: Firestore (Native Mode)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions

プロジェクトには以下の便利なスクリプトが含まれています：

```
./scripts/test-endpoints.sh

./scripts/test-e2e.sh

./scripts/manage-github-secrets.sh

./scripts/diagnostics.sh

./scripts/deploy-aws.sh
./scripts/deploy-azure.sh
./scripts/deploy-gcp.sh
```

```
make install         # 依存関係をインストール
make build-frontend  # フロントエンドをビルド
make build-backend   # Lambda パッケージを作成
make test-all        # 全クラウドのデプロイメントをテスト
make deploy-aws      # へデプロイAWS
make pulumi-preview  # 変更プレビューPulumi
make pulumi-up       # スタック適用Pulumi
make clean           # ビルド成果物を削除
```

VS Codeの Dev Containerに対応しています：

```
- Pulumi 3.x
- Node.js 18
- Python 3.12
- AWS CLI, Azure CLI, gcloud CLI
- Docker in Docker

pulumi          # Pulumi CLI
deploy-aws      # 環境にデプロイAWS
deploy-azure    # 環境にデプロイAzure
deploy-gcp      # 環境にデプロイGCP
test-all        # 全エンドポイントテスト
```

システムの健全性をチェック：

```
./scripts/diagnostics.sh
```

- ☒ インストール済みツールの確認
- ☒ クラウドプロバイダー認証状態
- ☒ デプロイメントエンドポイントのテスト
- ☒ Pulumiスタック状態の確認

```
./scripts/test-endpoints.sh

curl https://52z731x570.execute-api.ap-northeast-1.amazonaws.com/
curl https://mcad-staging-api--0000004.livelycoast-fa9d3350.japaneast.azurecontainerapps.io/
curl https://mcad-staging-api-son5b3ml7a-an.a.run.app/
```

```
cd services/frontend
npm install
npm run dev

cd services/backend
pip install -r requirements.txt
uvicorn src.main:app --reload
```

Lambda + API Gatewayの完全自動デプロイメント:

```
cd scripts
```

```
./deploy-lambda-aws.sh

PROJECT_NAME=myproject ENVIRONMENT=production ./deploy-lambda-aws.sh
```

スクリプトは以下を自動実行します：

- 依存関係のインストール（manylinux2014_x86_64）
- ZIPパッケージ作成とS3アップロード
- Lambda関数の作成/更新
- API Gateway統合設定
- Lambda権限設定（HTTP API用の正しいSourceArn）
- CloudWatch Logsアクセスログ有効化

エンドポイントの完全なCRUDテスト:

```
./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com

./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com --verbose
```

テスト項目：

- ヘルスチェック
- メッセージCRUD操作（作成、取得、更新、削除）
- ページネーション
- エラーハンドリング
- バリデーション

全環境（AWS/GCP/Azure）のエンドツーエンドCRUD動作を検証:

```
./scripts/test-sns-all.sh

./scripts/test-sns-aws.sh
./scripts/test-sns-azure.sh
./scripts/test-sns-gcp.sh
```

テストカバレッジ（2026-02-27 実績）:

- **AWS**: 9 PASS / 0 FAIL
- **Azure**: 17 PASS / 0 FAIL
- **GCP**: 13 PASS / 0 FAIL
- **Total**: 39 PASS / 0 FAIL ☒

テスト項目:

- ☒ Health Checks: 各環境のヘルスエンドポイント検証
- ☒ CRUD Operations: Create/List/Get/Update/Delete
- ☒ Authentication: Cloud-native認証（Cognito/Azure AD/Firebase）
- ☒ Image Upload: PUT /posts with images（AWS S3/Azure Blob/GCP GCS）

- ☒ Error Handling: 404/403/500エラー処理検証

クラウド固有のパス処理:

- AWS/GCP: `/api/messages/`
- Azure: `/api/HttpTrigger/api/messages/`（Flex Consumption対応）

期待される出力例:

```
═══════════════════════════════════════════════════════════════

        Multi-Cloud E2E Test Suite
═══════════════════════════════════════════════════════════════



  Testing: AWS
═══════════════════════════════════════════════════════════════✓

 Health check returned 'ok'✓
 Create message (ID: abc123...)✓
 List messages (found 5)✓
 Get message by ID✓
 Update message✓
 Delete message

[GCP/Azure: 同様]
═══════════════════════════════════════════════════════════════


        Test Summary
═══════════════════════════════════════════════════════════════

Total Tests:  18
Passed:       18
All tests passed! ✓
```

データ永続性検証:

- AWS: DynamoDB (PAY_PER_REQUEST)
- GCP: Firestore (Native Mode)
- Azure: Cosmos DB (Serverless)

包括的な監視とアラートを自動設定:

```
./scripts/setup-monitoring.sh

ALERT_EMAIL=your@email.com ./scripts/setup-monitoring.sh
```

設定内容：

- SNSトピックとメール通知
- Lambda エラー/スロットリング/実行時間/同時実行数アラーム
- API Gateway 5XXエラー/レイテンシアラーム
- DynamoDB スロットリングアラーム
- CloudWatch Logs メトリクスフィルター
- CloudWatch ダッシュボード自動作成

本番運用のために以下のサービス追加を推奨します:

Lambda関数のトレーシング有効化:

```
aws lambda update-function-configuration \
  --function-name YOUR_FUNCTION_NAME \
  --tracing-config Mode=Active
```

FastAPIにX-Ray統合:

```
aws-xray-sdk==2.12.0

from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.ext.fastapi.middleware import XRayMiddleware

app.add_middleware(XRayMiddleware, recorder=xray_recorder)
```

API Gatewayへの攻撃防御:

```
aws wafv2 create-web-acl \
  --name multicloud-auto-deploy-waf \
  --scope REGIONAL \
  --default-action Allow={} \
  --rules file://waf-rules.json

aws wafv2 associate-web-acl \
  --web-acl-arn YOUR_WEB_ACL_ARN \
  --resource-arn YOUR_API_GATEWAY_ARN
```

プロダクション用ドメイン設定:

```
aws acm request-certificate \
  --domain-name api.yourdomain.com \
  --validation-method DNS

aws apigatewayv2 create-domain-name \
  --domain-name api.yourdomain.com \
  --domain-name-configurations CertificateArn=YOUR_CERT_ARN

aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch file://route53-changes.json
```

環境変数の安全な管理:

```
aws secretsmanager create-secret \
  --name multicloud-auto-deploy/staging/db-config \
  --secret-string '{"host":"dynamodb","region":"ap-northeast-1"}'
```

```
import boto3
import json

def get_secret():
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(SecretId='multicloud-auto-deploy/staging/db-config')
    return json.loads(response['SecretString'])
```

共通ライブラリの分離でコールドスタート改善:

```
mkdir python
pip install -r requirements.txt -t python/
zip -r layer.zip python/

aws lambda publish-layer-version \
  --layer-name multicloud-auto-deploy-dependencies \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.12

aws lambda update-function-configuration \
  --function-name YOUR_FUNCTION_NAME \
  --layers YOUR_LAYER_ARN
```

リクエスト/レスポンスのエッジ処理:

```
// CloudFront Function: セキュリティヘッダ追加
function handler(event) {
  var response = event.response;
  response.headers["strict-transport-security"] = {
    value: "max-age=31536000; includeSubdomains",
  };
  response.headers["x-content-type-options"] = { value: "nosniff" };
  response.headers["x-frame-options"] = { value: "DENY" };
  return response;
}
```

DynamoDBの自動バックアップ:

```
aws backup create-backup-plan \
  --backup-plan file://backup-plan.json

aws backup create-backup-selection \
  --backup-plan-id YOUR_PLAN_ID \
  --backup-selection file://backup-selection.json
```

問題が発生した場合は トラブルシューティングガイド を参照してください：

- Azure認証問題（Service Principal、Pulumi Provider等）
- GCPリソース競合（State管理、リソースインポート）
- フロントエンドAPI接続問題（ビルド順序、API URL設定）
- 権限エラー（IAM、RBAC設定）

コントリビューションを歓迎します！詳細は CONTRIBUTING.md をご覧ください。

MIT License - 詳細は LICENSE をご覧ください。

- GitHub Actions Documentation
- Terraform Documentation
- Pulumi Documentation

Chapter 2

重要ルール（日本語訳）

> **このドキュメントはリポジトリ内の他のドキュメントを読む前に必ず読むこと。**
> データロス・本番環境障害・数時間のデバッグを招く最小限のルールセットが含まれている。
> ここに書かれたすべての項目は過去のインシデントから学んだ教訓である。

以下の14個のルールをすべて読んだら、この順序で続行する：

```
1. AI_AGENT_01_CONTEXT.md    ← プロジェクトの概要、ライブエンドポイント、使用技術（分）5
2. AI_AGENT_02_ARCHITECTURE.md ← ディレクトリツリー、各ファイルの配置（分）5
3. AI_AGENT_03_API.md ← システムトポロジー、ストレージパスレイアウト（分）5
4. AI_AGENT_06_STATUS.md      ← つのクラウド環境すべての現在の健全性3
5. AI_AGENT_09_TASKS.md       ← 優先度付きバックログ — 次に何をするか
```

タスクで必要な場合だけこれらを読む：

```
AI_AGENT_04_INFRA.md    ← エンドポイント仕様、リクエストレスポンススキーマ、データモデル/
AI_AGENT_05_CICD.md     ← スタック、リソース名、設定キー、スタック出力Pulumi
AI_AGENT_07_RUNBOOKS.md ← GitHub ワークフロー、必要なシークレット、トリガー条件Actions
AI_AGENT_08_SECURITY.md ← ステップバイステップ：Lambda デプロイ、React デプロイ、GCP Cloud Run
    など
```

すべてのソースコードコメント、コミットメッセージ、プルリクエスト説明、GitHub Actionsワークフローコメント、Pulumiスタックコメント、ドキュメントファイルは
英語で記述しなければならない（本版）。

このマルチクラウドインフラプロジェクトは異なるチームのエンジニアがレビューする可能性がある。英語で

ただし、日本語ユーザー向けの解説書や日本語訳ドキュメント（このファイルなど）は日本語で記述しても

単一のツール呼び出しバッチで大量のファイルを作成・編集すると、ネットワークエラーやタイムアウトが発

常に段階的に作業する：

1. **一度に数個のファイル**だけを作成または編集する。
2. 各ステップを検証してから続行する（コンパイルチェック、構文チェック、テスト）。
3. 途中でネットワークエラーが発生した場合、既に作成されたものを確認してから再試行する。

VS Code dev containerは `linux/aarch64` 用に構築されている。すべてのクラウドランタイム（Lambda、Azu Functions、Cloud Run、Cloud Functions）は `linux/amd64` で実行される。**aarch64上でネイティブにコンパ `.so` インポートエラーでクラッシュする。**

デプロイ用にPythonパッケージをインストールする場合、常にDocker を使用し、ターゲットプラットフォームを明示的に設定する：

```
docker run --rm --platform linux/amd64 \
  -v /tmp/deploy:/out python:3.12-slim \
  bash -c "pip install --no-cache-dir --target /out -r requirements.txt -q"
```

これは以下に適用される：

- Lambda ZIP パッケージング（`requirements-aws.txt`, `requirements-layer.txt`）
- GCP Cloud Functions ZIP パッケージング（`requirements-gcp.txt`）
- Azure Functions デプロイ（`requirements-azure.txt`）

dev container 内で `pip install` をホストマシンで実行してデプロイアーティファクトを生成しない。

```
develop  →  （ステージング）staging
main     →  （本番）←production 後すぐに本番環境で実行されるpush
```

本番環境にすぐにデプロイするつもりがない限り、`main` に直接プッシュしない。常に `develop`（またはフィーチャーブランチ）で作業し、ステージングが成功したことを確認してから `main` にマージする。

環境変数 `AUTH_DISABLED` は常に `false` でなければならない。過去のインシデントでは、誤って `true` に設定されていたため、すべてのAPIエンドポイントに対して認証なしでアクセスできていた。

CI ワークフロー、Pulumi 設定、`.env` ファイルのどこでも `AUTH_DISABLED=true` を見つけた場合、重大なバグとして扱い、直ちに修正する。

過去のバグにより、対応する GitHub シークレットが本番環境に存在しなかったため、`API_GATEWAY_ENDPOINT` 値が無声のうちに空になった。

従うべきパターン（`deploy-*.yml` 内）：

```
- name: Get Pulumi outputs
  run: |
    cd infrastructure/pulumi/aws
    echo "API_URL=$(pulumi stack output api_url --stack $STACK)" >> $GITHUB_ENV
```

pulumi stack output から導出できる値には ${{ secrets.API_GATEWAY_ENDPOINT }} のような GitHub シークレットを決して使用しない。シークレットが存在しない場合、変数は無声のうちに空の文字列になり はハードコードされたフォールバック（しばしば localhost:8000）を使用して起動する。

SNS React アプリは s3://bucket/sns/ にデプロイされる。VITE_BASE_PATH=/sns/ なしでビルドすると、すべての JS/CSS アセットが /assets/... を参照し、404 が発生する。CloudFront はすべてのアセットリクエストに対して Content-Type: text/html で index.html を配信し、MIME タイプエラーでアプリ全体が破損する。

```
cd services/frontend_react
set -a && source .env.aws.staging && set +a
VITE_BASE_PATH=/sns/ npm run build
```

CloudFront カスタムエラーページも /sns/index.html を指す必要があり、/index.html ではない。

---

multicloud-auto-deploy-staging-images はすべてのパブリックアクセスがブロックされている。バックエンドはフ _resolve_image_urls() を呼び出す必要がある。これは保存された S3 キー（imageKeys）を有効期限1時間の署名付き GET URL に変換する。

生の S3 キーを返すか、署名付き URL をデータベースにキャッシュすると、1時間後に破損した画像リンクか

以下の設定がないと、pulumi up --stack production は CloudFront をデフォルト証明書に戻し、すべての HTTPS ビジターに対して NET::ERR_CERT_COMMON_NAME_INVALID が発生する。

```
cd infrastructure/pulumi/aws
pulumi config set customDomain www.aws.ashnova.jp --stack production
pulumi config set acmCertificateArn \
  arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19c4460dcde5 \
  --stack production
```

本番 AWS スタックで pulumi up を実行する前に、これら2つの設定値が存在することを常に確認する。

フロントエンドが id_token のみを送信する場合（エスコート access_token なし）、JWT ライブラリは at_hash を計算または検証できない。jwt_verifier.py への呼び出しは verify_at_hash=False を渡さなければならない。この設定を戻すと、すべての認証付き API 呼び出しが破損する。

参照：services/api/app/jwt_verifier.py

--entry-point が別の関数に名前を付けている場合でも、Cloud Build は missing main.py で失敗する。修正方法はデプロイソースに main.py を含めることである：

```
cp services/api/function.py /tmp/deploy_gcp/.deployment/main.py
```

常に `function.py` と `main.py` の両方を ZIP に含める。これを怠ると、Cloud Build はソースを拒否し、関数は更新されない。

Cloud Run と Cloud Functions は Compute Engine 認証情報を使用する（秘密鍵なしのアクセストークンの `blob.generate_signed_url()` を呼び出すと、以下が発生する：

```
AttributeError: you need a private key to sign credentials
```

正しい呼び出し：

```
blob.generate_signed_url(
    expiration=3600,
    method="GET",
    service_account_email=settings.gcp_service_account,
    access_token=credentials.token,
)
```

サービスアカウントメールは Cloud Run サービスで環境変数 `GCP_SERVICE_ACCOUNT` を介して提供される。SA は `roles/iam.serviceAccountTokenCreator` も持つ必要がある。

新しい Cloud Run URL またはカスタムドメインが追加される場合、Firebase Auth に明示的に登録する必要がある。Identity Toolkit Admin v2 `PATCH` エンドポイントをヘッダー `x-goog-user-project: PROJECT_ID` で使用する — このヘッダーを省略すると、有効な管理者トークンがあっても `403 PERMISSION_DENIED` が返される。

このステップは `deploy-gcp.yml`（`Update Firebase Authorized Domains`）で自動化されている。これを削除またはス

このレスポンスヘッダーがないと、Firebase `signInWithPopup` は `popup.closed` をチェックできず、COOP 警告を繰り返し、厳格なブラウザー環境でのログインフローを潜在的に破損する。

これを CDN バックエンドバケットカスタムレスポンスヘッダーとして設定する（Pulumi または `gcloud` 経由）。変更後、CDN キャッシュを無効化する：

```
gcloud compute url-maps invalidate-cdn-cache <URL_MAP_NAME> \
  --path "/*" --project ashnova
```

Azure Functions（Flex Consumption）は **Python ランタイムの前に Kestrel（.NET HTTP サーバー）を実行する**。Kestrel がすべての `OPTIONS` プリフライトリクエストを Python コードに到達する前に傍受する。FastAPI CORS ミドルウェアで `Access-Control-Allow-Origin` を設定しても、**効果がない** — Kestrel がミドルウェアに到達する前にレスポンスを返す。

常に Azure CLI または Portal 経由で CORS を設定する：

```
az functionapp cors add \
  --resource-group "$RESOURCE_GROUP" \
  --name "$FUNCTION_APP_NAME" \
  --allowed-origins "https://your.domain.com"

az functionapp cors remove \
  --resource-group "$RESOURCE_GROUP" \
  --name "$FUNCTION_APP_NAME" \
  --allowed-origins '*'
```

Azure Blob Storage CORS は Function App CORS から完全に独立している。 画像アップロードは SAS URL 経由で Blob Storage に直接移動する（Function App をバイパス）。Blob Storage CORS は別途設定する必要がある：

```
az storage cors add \
  --account-name "$STORAGE_ACCOUNT" \
  --services b \
  --methods GET POST PUT DELETE OPTIONS \
  --origins "https://your.domain.com" \
  --allowed-headers "*" \
  --exposed-headers "*" \
  --max-age 3600
```

また：`SCM_DO_BUILD_DURING_DEPLOYMENT` は Flex Consumption では**サポートされていない** — `InvalidAppSettingsException` をトリガーする。常に Docker を使用してパッケージをローカルでビルドしてからデプロイする。

Azure Functions は デフォルトで `routePrefix: "api"`に設定されており、すべての HTTP トリガーに `/api/` のプリフィックスを付ける。フロントエンドは `/api/` プリフィックスなしで `/posts`、`/health` のようなパスを呼び出す。この修正がないと、すべての API エンドポイントが 404 を返す。

必須の `host.json` コンテンツ：

```
{
  "version": "2.0",
  "extensions": {
    "http": {
      "routePrefix": ""
    }
  },
  "extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[4.*, 5.0.0)"
  }
}
```

また：Azure AD `post_logout_redirect_uri` はアプリの `redirect_uris` に明示的に登録する必要がある。`/sns/auth/callback` コールバック URL のみを登録することは不十分 — ログアウトリターン URL（例：`/sns/`）も追加する必要がある：

```
az ad app update \
  --id "$AD_APP_ID" \
  --web-redirect-uris \
    "https://your.domain.com/sns/auth/callback" \
    "https://your.domain.com/sns/"
```

トピック	ファイル
ライブエンドポイント URL	AI_AGENT_01_CONTEXT.md
リポジトリディレクトリツリー	AI_AGENT_01_CONTEXT.md
システムアーキテクチャ	AI_AGENT_02_ARCHITECTURE.md
API ルート & データモデル	AI_AGENT_03_API.md
Pulumi / IaC	AI_AGENT_04_INFRA.md
CI/CD パイプライン	AI_AGENT_05_CICD.md
現在の環境健全性	AI_AGENT_06_STATUS.md
ステップバイステップランブック	AI_AGENT_07_RUNBOOKS.md
セキュリティ設定	AI_AGENT_08_SECURITY.md
残りのタスク / バックログ	AI_AGENT_09_TASKS.md
すべて — エントリーポイント	AI_AGENT_GUIDE.md

Chapter 3

コンテキスト・概要（日本語訳）

第I部 — オリエンテーション | 親: AI_AGENT_GUIDE.md

---

multicloud-auto-deploy は、**同一のフルスタックアプリケーション**（SNSスタイルのメッセージングアプ
AWS、Azure、GCP に完全に自動化された CI/CD パイプライン経由で同時にデプロイするプラットフォーム

- フロントエンド: React 18 + Vite + TypeScript + Tailwind CSS
- バックエンド: FastAPI（Python 3.12）— Lambda / Azure Functions / Cloud Run
- データベース: DynamoDB / Cosmos DB / Firestore（共有論理スキーマ）
- IaC: Pulumi Python SDK
- CI/CD: GitHub Actions

目的	URL
CDN (CloudFront)	`https://d1tf3uumcm4bo1.cloudfront.net`
API (API Gateway)	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`
カスタムドメイン	`https://www.aws.ashnova.jp`

目的	URL
CDN (Front Door)	`https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`
API (Functions)	`https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.ne`
カスタムドメイン	`https://www.azure.ashnova.jp`

> **注意**：Azure API URL は Function App ベース URL **であり、パスサフィックスは含まない**。 関数はワイルドカードルーム `{*route}` を使用するため、すべての API パス（例：`/health`、`/posts`） がベース URL で直接配信される。`/api/HttpTrigger` パスは **使用されない**。

目的	URL
CDN (Cloud CDN)	`http://34.117.111.182`
API (Cloud Run)	`https://multicloud-auto-deploy-staging-api-899621454670.asia-northeast1.run.app`
Web フロントエンド (Cloud Run)	`https://multicloud-auto-deploy-staging-frontend-web-899621454670.asia-northeast1.run.a` (legacy SSR)
カスタムドメイン	`https://www.gcp.ashnova.jp`

注意：Cloud Run `frontend-web` サービスは存在するが、CDN は `/sns/*` を GCS バケット（React SPA）にルーティングする — Cloud Run ではない。Cloud Run URL はレガシーである。

```
フロントエンド（ページ）
SNS
  AWS:   React 18.2 / Vite 7.3 / TypeScript / Tailwind CSS  ← S3 内の静的 （ステージングSPA + 本番）
  Azure: React 18.2 / Vite / TypeScript  ← Blob Storage $web/sns/ 内の静的 （本番）SPA
         (services/frontend_web Python SSR はスーパーセッドされた; CI は現在 deploy-frontend-web-azure.yml で React SPA をデプロイ)
  GCP:   React 18.2 / Vite / TypeScript  ← GCS sns/ プリフィックス内の静的 （ステージングSPA + 本番）
         (Cloud Run frontend-web は存在するが、CDN は Cloud Run ではなく GCS バケットにルーティング)バックエンド

 API
  FastAPI 1.0 / Python 3.12 / Pydantic v2
  AWS:    () Lambdax86_64+ API Gateway  () v2HTTP+ Lambda Layer + Mangum アダプター
  Azure: Azure （FunctionsPython 、3.12FC1 ） FlexConsumption
  GCP:   Cloud （RunPython 、3.12）gen2
  ローカル: uvicorn + DynamoDB Local + MinIOデータベース（共有論理スキーマ）


  AWS:   DynamoDB — テーブル: {project}-{stack}-posts
  Azure: Cosmos DB Serverless — db: messages / container: messages
  GCP:    () —FirestoreNative コレクション: messages / postsインフラストラクチャ


  Pulumi Python SDK 3.x
  状態: Pulumi （リモート）Cloud認証


  AWS:   Amazon （CognitoPulumi により自動プロビジョニング）
  Azure: Azure （ADPulumi により自動プロビジョニング）
  GCP:   Firebase （AuthGoogle Sign-、Inhttponly Cookie セッション）
  ステージング: AUTH_DISABLED=false  ← 決して true になってはいけない
```

```
multicloud-auto-deploy/              ← ワークスペースルート = git リポジトリルート │ ├──

 .github/ │
   └── workflows/                    ← ★ 実際のワークフロー — CI はこれらのみを読む │
       ├── deploy-aws.yml │
       ├── deploy-azure.yml │
       ├── deploy-gcp.yml │
       ├── deploy-landing-aws.yml │
       ├── deploy-landing-azure.yml │
       ├── deploy-landing-gcp.yml │
       ├── deploy-frontend-web-aws.yml │
       ├── deploy-frontend-web-azure.yml │
       └── deploy-frontend-web-gcp.yml │ ├──

 infrastructure/ │
   └── pulumi/ │
       ├── aws/ │
       │   ├── __main__.py           ← すべての AWS
   リソース（S3/CF/Lambda/APIGW/DDB/）Cognito │
       │   ├── Pulumi.yaml │
       │   ├── Pulumi.staging.yaml │
       │   └── requirements.txt │
       ├── azure/ │
       │   ├── __main__.py           ← すべての Azure
   リソース（Storage/FuncApp/CosmosDB/FrontDoor/）AD │
       │   ├── Pulumi.yaml │
       │   └── requirements.txt │
       └── gcp/ │
           ├── __main__.py           ← すべての GCP リソース（GCS/CloudRun/Firestore/）CDN │
           ├── Pulumi.yaml │
           └── requirements.txt │ ├──

 services/ │
   ├── api/                          ← FastAPI バックエンド │
   │   ├── app/ │
   │   │   ├── main.py               ← FastAPI アプリ、、CORSMangum ハンドラー │
   │   │   ├── config.py             ← Pydantic （環境変数をロード）Settings │
   │   │   ├── models.py             ← Pydantic モデル（、PostProfile など） │
   │   │   ├── auth.py               ← JWT 認証ミドルウェア │
   │   │   ├── jwt_verifier.py       ← Cognito / Azure AD / Firebase JWT 検証 │
   │   │   ├── backends/ │
   │   │   │   ├── base.py           ← BackendBase 抽象クラス │
   │   │   │   ├── aws_backend.py    ← DynamoDB + S3 実装 │
   │   │   │   ├── azure_backend.py  ← Cosmos DB + Blob Storage 実装 │
   │   │   │   ├── gcp_backend.py    ← Firestore + Cloud Storage 実装 │
   │   │   │   └── local_backend.py  ← DynamoDB Local + MinIO 実装 │
   │   │   └── routes/ │
   │   │       ├── posts.py          ← POST/GET/PUT/DELETE /posts │
   │   │       ├── profile.py        ← GET/PUT /profile/{userId} │
   │   │       └── uploads.py        ← POST /uploads/presigned-url │
```

```
49    │    ├── index.py                ← Lambda ハンドラー（Mangum ラッパー）│
50    │    ├── function_app.py         ← Azure Functions ハンドラー│
51    │    ├── function.py             ← GCP Cloud Functions ハンドラー│
52    │    ├── requirements.txt        ← 共有依存関係（、、fastapimangumpydantic）...│
53    │    ├── requirements-aws.txt    ← AWS 固有（boto3 など）│
54    │    ├── requirements-azure.txt  ← Azure 固有（azure-cosmos など）│
55    │    ├── requirements-gcp.txt    ← GCP 固有（google-cloud-firestore など）│
56    │    └── requirements-layer.txt  ← Lambda Layer 依存関係（boto3 を除く）│
57    │ │
58    ├── frontend_react/            ← React フロントエンド（SNS アプリ — ）AWS│
59    │    ├── src/│
60    │    │    ├── main.tsx│
61    │    │    ├── App.tsx│
62    │    │    ├── api/              ← Axios クライアント│
63    │    │    ├── components/       ← UI コンポーネント│
64    │    │    └── hooks/            ← カスタムフック│
65    │    ├── vite.config.ts        ← base: "/sns/" を設定│
66    │    └── package.json│
67    │ │
68    └── frontend_web/              ← Python FastAPI フロントエンド（SNS アプリ — Azure +
      ）GCP│├──
69
70  static-site/                   ← ランディングページ（平文 、HTMLSPA ではない）│
71    ├── index.html                 ← 共有ランディング（クラウドローカル環境を自動検出）/│
72    ├── aws/  azure/  gcp/         ← クラウドテーマ付きランディングバリアント│
73    └── nginx.conf│├──
74
75  scripts/                       ← デプロイ / テストシェルスクリプト├──
76  docs/│
77    ├── AI_AGENT_GUIDE.md          ← ★ AI エージェント エントリーポイント│
78    ├── AI_AGENT_00_CRITICAL_RULES.md ← 最初に読む│
79    ├── AI_AGENT_01_CONTEXT.md     ← このファイル│
80    └── archive/                   ← アーカイブ / スーパーセッドされたドキュメント│├──
81
82  docker-compose.yml             ← api + dynamodb-local + minio├──
83  Makefile└──
84  README.md
```

やりたいこと	編集するファイル（群）
API エンドポイントを追加	`services/api/app/routes/*.py`
DB ロジックを変更（AWS）	`services/api/app/backends/aws_backend.py`
DB ロジックを変更（Azure）	`services/api/app/backends/azure_backend.py`
DB ロジックを変更（GCP）	`services/api/app/backends/gcp_backend.py`
環境変数を追加	`services/api/app/config.py` + `Pulumi.*.yaml`
AWS インフラを変更	`infrastructure/pulumi/aws/__main__.py`

やりたいこと	編集するファイル（群）
Azure インフラを変更	`infrastructure/pulumi/azure/__main__.py`
GCP インフラを変更	`infrastructure/pulumi/gcp/__main__.py`
CI/CD ワークフロー編集	`.github/workflows/*.yml`（ワークスペースルート）
React フロントエンド UI 編集	`services/frontend_react/src/`
Python フロントエンド編集	`services/frontend_web/`
ランディングページ編集	`static-site/`

- **アーキテクチャ: ARM（Apple Silicon M シリーズ Mac）**
- 開発環境: VS Code Dev Container（`.devcontainer/`）

コンポーネント	詳細
ベースイメージ	`mcr.microsoft.com/devcontainers/base:ubuntu`
Python	3.12
Node.js	22
Docker	Docker-in-Docker v2
クラウド CLI	AWS CLI、Azure CLI、Google Cloud SDK、GitHub CLI
IaC	Pulumi CLI
公開ポート	3000（フロントエンド開発）、8000（API）

⚠ dev container は `linux/aarch64`。すべてのクラウドランタイムは `linux/amd64` で実行される。 デプロイ用に Python パッケージは常にクロスコンパイルする：

```
docker run --rm --platform linux/amd64 \
  -v /tmp/deploy:/out python:3.12-slim \
  bash -c "pip install --no-cache-dir --target /out -r requirements.txt -q"
```

詳細は AI_AGENT_00_CRITICAL_RULES.md のルール 2 を参照。

```
cd /workspaces/multicloud-auto-deploy
docker compose up -d          # API (uvicorn) + DynamoDB Local + MinIO + frontend_web
curl http://localhost:8000/health
```

- `~/.aws` → AWS CLI 認証情報
- `~/.azure` → Azure CLI 認証情報
- `~/.config/gcloud` → Google Cloud SDK 認証情報

```
feature/xxx  →  develop  →  push  →  staging 自動デプロイ
```

```
                    ↓
                main  →  push  →  本番環境 自動デプロイ  ⚠ 即座に
```

→ 02 — アーキテクチャ

Chapter 4

アーキテクチャ（日本語訳）

第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

```
ユーザー

  │
  ├── [AWS]   CloudFront ──☒ （S3React ：ランディングSPA + ページ）SNS  ← 静的
  │         API Gateway v2 ──☒ （LambdaPython ）──☒3.12 DynamoDB
  │
  ├── [Azure] Front Door ──┬── /sns/* ──☒ Blob Storage $web/sns（/React SPA ← 静的）
  │                        └── /*     ──☒ Blob Storage （ランディング$web ← 静的）
  │         Azure Functions func ──☒ Cosmos （）DBServerless
  │
  └── [GCP]   Cloud CDN ──┬── /sns/* ──☒ GCS バケット /sns（/React SPA ← 静的）
                          └── /*     ──☒ （ランディングGCS ← 静的）
            Cloud Run api ──☒ Firestore
```

フロントエンドアーキテクチャ：3つのクラウドすべてが SNS アプリを **静的 React SPA** として提供 オブジェクトストレージ（S3 / Blob Storage / GCS）にデプロイされ、CDN 経由で配信される。 Python `services/frontend_web` SSR サービスは **スーパーセッドされた** もので、CDN パスに含まれていない。 React SPA ワークフロー：`deploy-frontend-web-{aws,azure,gcp}.yml` SPA ルーミング（`/sns/*` → `/sns/index.html` にリライト）：

- AWS：CloudFront Function `spa-sns-rewrite-{stack}`
- Azure：AFD RuleSet `SpaRuleSet` + URL Rewrite アクション
- GCP：CDN バックエンドバケットが GCS をサーブ；`/sns/*` パスルール URL マップから削除（GCS にフォールスルー）

```
bucket-root/ ├──
 index.html          ← ランディングページ ├──
 error.html ├──
```

```
aws/ ├──
azure/ ├──
gcp/ └──
sns/              ← React SNS アプリ（AWS のみ — Vite ビルド、base="/sns" /"
    ├── index.html
    └── assets/
        ├── index-*.js
        └── index-*.css
```

CI デプロイ先：

コンテンツ	AWS	Azure	GCP
ランディングページ	`s3://bucket/`	`$web/`	`gs://bucket/`
SNSページ	`s3://bucket/sns/`（React SPA）	`$web/sns/`（React SPA）	`gs://bucket/sns/`（React SPA）

3つのクラウドすべてが SNS アプリを静的 React SPA として提供。CDN が SPA ルーティングを処理：

- **AWS**：CloudFront Function が `/sns` および `/sns/` → `/sns/index.html` にリライト
- **Azure**：AFD `SpaRuleSet` URL Rewrite が `/sns/*`（非アセット）→ `/sns/index.html` にリライト
- **GCP**：GCS が `sns/index.html` をデフォルトとして配信；ディープリンクには CDN レベルの処理が必要（SPA ルーティングはブラウザー History API で部分的に処理）

```
（
CloudFrontE1TBH4R432SZBZ / ステージング、E214XONKTXJEJD / 本番）
  ├── /sns/* → S3: multicloud-auto-deploy-{env}-frontend/sns（/React ）SPA
  │            CloudFront Function `spa-sns-rewrite-{stack}` が /sns を /sns/index.html
    にリライト
  └── /*     → S3: multicloud-auto-deploy-{env}-frontend（ランディングページ）/

S3: multicloud-auto-deploy-{env}-frontend
  ├── index.html        ← React SPA エントリーポイント（Vite ビルド）
  ├── assets/           ← JS / CSS バンドル
  └── （ランディング、aws、/azure、/gcp/ ページ）

API Gateway v2 （HTTPz42qmqdqac / ステージング）
  └── $default → Lambda: multicloud-auto-deploy-{env}-（バックエンド）api
                 └── DynamoDB: multicloud-auto-deploy-{env}-（）postsPAY_PER_REQUEST
                     ← Single Table （DesignPK/）SK
                     ← POSTS パーティション：投稿データ（：GSIpostId / userId / ）createdAt
                     ← PROFILES パーティション：ユーザープロフィール
                 └── S3: multicloud-auto-deploy-{env}-（画像アップロード）images
                     ← IMAGES_BUCKET_NAME 環境変数で参照

Lambda: multicloud-auto-deploy-{env}-frontend-web 削除済み[ — Python SSR は React SPA に統合]
  デッドコード削除 。2026-02-22CloudFront `/sns/*` は現在 S3 に直接ルーティング。
```

```
  参照: REFACTORING_REPORT_20260222.md § 3
```

注意：`frontend-web` Lambda は当初 Python で SNS 画面を SSR するために作られたが、React + S3 へ移行済み。Lambda 自体は削除されていない場合があるが、CloudFront の `/sns/*` 動作は現在 API Gateway（バックエンド API）を向いている。

Lambda Layer：`multicloud-auto-deploy-staging-dependencies` — FastAPI / Mangum / JWT 依存関係のみを含む；boto3 は Lambda ランタイムに含まれる。 — アプリコード（~78 KB）と Layer（~8-10 MB）は別々にデプロイ。

主要環境変数：`POSTS_TABLE_NAME`、`IMAGES_BUCKET_NAME`、`COGNITO_USER_POOL_ID` **可観測性**：AWS Lambda Powertools（Logger / Tracer / Metrics）— `SimpleSNS` ネームスペース

☒ **React SPA に移行**：SNS ページは現在 Blob Storage（静的ファイル）から配信される。 Python `frontend_web` Azure Function は `deploy-frontend-web-azure.yml` に統合された。

```
Front （Doormulticloud-auto-deploy-staging-）fd
  エンドポイント: mcad-staging-d45ihd
  ├─── /sns/*  → オリジン：Blob Storage $web/sns（/React SPA — 静的 HTML/JS/）CSS
  │              SpaRuleSet が /sns/* → /sns/index.html にリライト（SPA ルーティング）
  └─── /*      → オリジン：Blob Storage （ランディングページのみ）$web
                 mcadwebd45ihd.z11.web.core.windows.net

Azure Functions frontend-（webFC1 ）FlexConsumptionレガシー[ — CDN パスに含まれない]
  └─── デプロイ済みだが、React SPA in Blob Storage に統合された
     本番環境：multicloud-auto-deploy-production-frontend-web-（v2alwaysReady http）=1

Azure ：Functionsmulticloud-auto-deploy-staging-（funcFlex ）←Consumption バックエンド API
  └─── HTTP トリガー：/{*route（関数名：}）HttpTrigger
       │  ← （FastAPIMangum なし、カスタム ASGI ブリッジ）にフォワード
       └─── Cosmos （）DBServerless
            ← ：DBsimple-sns / コンテナー：items
            ← 環境変数：COSMOS_DB_ENDPOINT / COSMOS_DB_KEY
            ← （デフォルト：COSMOS_DB_DATABASEsimple-）sns
            ← （デフォルト：）COSMOS_DB_CONTAINERitems
       └─── Azure Blob ：Storageimages コンテナー（画像アップロード）
            ← AZURE_STORAGE_ACCOUNT_NAME / AZURE_STORAGE_ACCOUNT_KEY / AZURE_STORAGE_CONTAINER
```

リソースグループ：`multicloud-auto-deploy-staging-rg`（japaneast） **WAF**：未設定（Standard SKU；Premium SKU で追加可能）

☒ **React SPA に移行**：SNS ページは現在 GCS（静的ファイル）から Cloud CDN 経由で配信される。 Cloud Run `frontend-web` へのルーティングの `/sns/*` パスルールは URL マップから削除された。

```
グローバル
 ：IP34.117.111.182
```

```
  └─── HTTP フォワーディングルール
       └─── URL マップ
            └─── （デフォルト）→/* バックエンドバケット →
    ：GCSashnova-multicloud-auto-deploy-staging-frontend
                          （/sns/ の React SPA + / のランディング）
            注記：/sns/* パスルール削除（）—2026-02-22 GCS デフォルトにフォールスルー

Cloud ：Runmulticloud-auto-deploy-staging-frontend-web レガシー[ — CDN パスに含まれない]
  ：URLhttps://multicloud-auto-deploy-staging-frontend-web-899621454670.asia-northeast1.run.app
  └─── デプロイ済みだが CDN はここにリクエストをルーティングしない
 CDN カスタムレスポンスヘッダー：Cross-Origin-Opener-Policy: same-origin-allow-popups

Cloud ：Runmulticloud-auto-deploy-staging-（バックエンドapi ）API
  └─── （デフォルト）Firestore
      ← posts コレクション：投稿データ（、デフォルト：）GCP_POSTS_COLLECTIONposts
      ← profiles
    コレクション：ユーザープロフィール（、デフォルト：）GCP_PROFILES_COLLECTIONprofiles
  └─── ：GCSashnova-multicloud-auto-deploy-staging-（署名付きuploads URL
    アップロード画像表示）/
      ← GCP_STORAGE_BUCKET 環境変数で参照
```

注意：GCP はクラシック外部 LB（EXTERNAL スキーム）を使用。URL マップパスベースルーティング（/sns/* → Cloud Run）には EXTERNAL_MANAGED が必要；現在はすべてのパスで GCS にフォールバックする可能性 — 検証が必要。

ルータープレフィックス	主なエンドポイント	認証
/posts	GET/POST/PUT/DELETE（投稿 CRUD）	必要
/uploads	POST（署名付き URL 発行）	必要
/profile	GET/PUT（プロフィール取得・更新）	必要
/api/messages/	旧フロントエンド互換エイリアス	オプション

旧フロントエンドとの後方互換のため /api/messages/ エイリアスは維持されているが、新規実装は /posts を使用する。

```
AWS_EXECUTION_ENV   存在 → "aws"
WEBSITE_INSTANCE_ID 存在 → "azure"
K_SERVICE           存在 → "gcp"それ以外
                  → "local"
```

```
API バックエンド（services/）：api
  クライアント
    → Authorization: Bearer <JWT>
```

```
    → FastAPI auth.（pyAUTH_DISABLED=false の場合）
        → jwt_verifier.py
            AWS:   Cognito JWKS エンドポイント検証
            Azure: Azure AD JWKS 検証
            GCP:   Firebase Auth JWKS 検証
        → 検証 OK → routes/*

GCP frontend-（webservices/）—frontend_web 別の httponly Cookie フロー：
  ブラウザー → /sns/login
    → Firebase Google Sign-In ポップアップ（Firebase SDK v10）.8.0
    → POST /sns/session { token: <Firebase ID トークン> }
    → FastAPI がトークンを検証 → httponly セッションクッキーを設定
    → onIdTokenChanged → トークン有効期限時にセッションクッキーを自動更新
```

ステージング：`AUTH_DISABLED=false`（過去には誤って `true` に設定されていた — 現在は修正）

機能	AWS	Azure	GCP
HTTPS 強化	☒ CloudFront	☒ Front Door	☒ HTTP のみ（要修正）
WAF	☒ 未設定	☒ 未設定（TODO）	☒ Cloud Armor
データ暗号化	☒ SSE-S3	☒ Azure SSE	☒ Google 管理
バージョンストレージ	☒	☒	☒
アクセスログ	☒	☒	☒
セキュリティヘッダー	☒ CloudFront RP	☒	☒

→ 03 — API とデータモデル

Chapter 5

API とデータモデル（日本語訳）

第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

メソッド	パス	認証	説明
GET	`/`	☒	ルート — クラウド情報
GET	`/health`	☒	ヘルスチェック
GET	`/limits`	☒	アップロード/投稿制限（`maxImagesPerPost`）
GET	`/docs`	☒	Swagger UI
GET	`/redoc`	☒	ReDoc
GET	`/posts`	☒	投稿一覧（ページネーション+タグフィルター対応）
GET	`/posts/{post_id}`	☒	単一投稿を取得
POST	`/posts`	☒	投稿を作成
PUT	`/posts/{post_id}`	☒	投稿を更新（所有者または管理者）
DELETE	`/posts/{post_id}`	☒	投稿を削除（所有者または管理者）
GET	`/profile/{user_id}`	☒	プロフィールを取得
PUT	`/profile/{user_id}`	☒	プロフィールを更新
POST	`/uploads/presigned-urls`	☒	画像アップロード用署名付き URL を発行（単数）
POST	`/uploads/presigned-urls`	☒	一括アップロード用署名付き URL を発行

```
{
  "content": " (string1) -10000",
  "isMarkdown": false,
  "imageKeys": ["key1", "key2"], // 最大 （サーバー側で16 MAX_IMAGES_PER_POST で強制）
  "tags": ["tag1", "tag2"]
}
```

```
{ "maxImagesPerPost": 10 }
```

フロントエンドはマウント時にこれを取得。`MAX_IMAGES_PER_POST` はバックエンドの環境変数（デフォルト：10）
これが正規ソース — **フロントエンドにアップロード制限をハードコードしない。**

```
{
  "items": [ <Post> ],
  "limit": 20,
  "nextToken": "string | null"
}
```

```
{
  "postId": "uuid",
  "userId": "string",
  "nickname": "string | null",
  "content": "string",
  "isMarkdown": false,
  "imageUrls": ["https://..."],
  "tags": ["tag"],
  "createdAt": "2026-02-20T00:00:00Z",
  "updatedAt": "2026-02-20T00:00:00Z",

  // 後方互換フィールド（）snake_case
  "id": "uuid",
  "author": " （string= ）userId",
  "created_at": "...",
  "updated_at": "...",
  "image_url": "string | null"
}
```

パラメータ	型	デフォルト	説明
`limit`	int（1-50）	20	返すアイテム数
`nextToken`	string	null	ページネーショントークン
`tag`	string	null	タグフィルター

> **GCP 注記：**`generate_signed_url()` は `service_account_email` + `access_token` が必須（IAM `signBlob` API パス）Compute Engine 認証情報は秘密鍵を含まないため。`AI_AGENT_00_CRITICAL_RULES.md` のルール 11 を参照。

```
{
  "count": 3, // ファイル数（ge、=1le；実際の制限は=100 /limits 経由）
  "contentTypes": ["image/jpeg", "image/png", "image/webp"] // ファイルあたり 1 つ
}
```

```
{
```

```
  "urls": [
    { "uploadUrl": "https署名付き://...-url...", "key": "uploads/uuid.jpg" },
    ...
  ]
}
```

GCP 注記：generate_signed_url() は service_account_email + access_token が必須（IAM signBlob API パス）Compute Engine 認証情報は秘密鍵を含まないため。AI_AGENT_OO_CRITICAL_RULES.md のルール 11 を参照。

ローカルと AWS 向け共有テーブル設計。Azure/GCP は同じ論理モデルを使用。

アイテムタイプ	PK	SK	キー属性
投稿	POSTS	<ISO-timestamp>#<uuid>	postId、userId、content、tags、createdAt
プロフィール	USER#<userId>	PROFILE	postId=PROFILE#<userId>、userId、nickname、bio

キー	値
ハッシュキー	postId
プロジェクション	ALL

直接投稿ルックアップ（/posts/{id}）と プロフィールルックアップ（postId = PROFILE#<userId>）に使用。

```
 (
BackendBasebase.) py ← 抽象クラス
   ├─── list_posts(、、limitnext_tokentag) →  (list[Post、]) next_token
   ├─── get_post(post_id) → Post
   ├─── create_post(、bodyuser) → Post
   ├─── update_post(、、post_idbodyuser) → Post
   ├─── delete_post(、post_iduser) → dict
   ├─── get_profile(user_id) → Profile
   ├─── update_profile(、、user_idbodyuser) → Profile
   └─── generate_upload_urls(、keysuser) → list[UploadUrl]

AwsBackend    →  () DynamoDBboto3+ S3
AzureBackend  → Cosmos  (DBazure-) cosmos+ Blob Storage
GcpBackend    →  (Firestoregoogle-cloud-) firestore+ Cloud Storage
LocalBackend  → DynamoDB Local +  (MinIOS3 互換)
```

```python
def get_backend() -> BackendBase:
    provider = config.CLOUD_PROVIDER  # "aws" | "azure" | "gcp" | "local"
    ...
```

```
cd multicloud-auto-deploy
docker compose up   # api(:8000) + dynamodb-local(:8001) + minio(:9000/:9001)
```

サービス	URL	認証情報
FastAPI	http://localhost:8000	—
Swagger UI	http://localhost:8000/docs	—
DynamoDB Local シェル	http://localhost:8001/shell/	—
MinIO コンソール	http://localhost:9001	minioadmin / minioadmin

```
CLOUD_PROVIDER=local
DYNAMODB_ENDPOINT=http://dynamodb-local:8001
DYNAMODB_TABLE_NAME=simple-sns-local
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET_NAME=simple-sns
AUTH_DISABLED=true   ← ローカルのみ true は許容
```

```
cd services/api
pytest tests/test_backends_integration.py -v

pytest -m aws

export API_BASE_URL=https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com
pytest tests/test_api_endpoints.py -v

./scripts/test-e2e.sh
```

→ 04 — インフラストラクチャ（Pulumi）

Chapter 6

インフラ・Pulumi 解説（日本語訳）

第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

フィールド	値
IaC ツール	Pulumi Python SDK 3.x
状態	Pulumi Cloud（リモート状態）
スタック	`staging` / `production`
言語	Python
コードパス	`infrastructure/pulumi/{aws,azure,gcp}/__main__.py`

```
pulumi stack ls

pulumi up --stack staging

pulumi stack output

pulumi preview --stack staging

pulumi destroy --stack staging
```

ディレクトリ：`infrastructure/pulumi/aws/`

Pulumi 論理名	AWS リソース	名前パターン
`lambda-role`	IAM Role	`{project}-{stack}-lambda-role`
`app-secret`	Secrets Manager Secret	—
`dynamodb-table`	DynamoDB テーブル	`{project}-{stack}-posts`

Pulumi 論理名	AWS リソース	名前パターン
`lambda-function`	Lambda 関数	`{project}-{stack}-api`
`api-gateway`	API Gateway v2	—
`frontend-bucket`	S3 バケット	`{project}-{stack}-frontend`
`landing-bucket`	S3 バケット	`{project}-{stack}-landing`
`cloudfront-distribution`	CloudFront（PriceClass_200）	—
`security-headers-policy`	CloudFront ResponseHeadersPolicy	`{project}-{stack}-security-headers`
`cognito-user-pool`	Cognito ユーザープール	—
`sns-topic`	SNS トピック（アラート）	—
CloudWatch Alarms（複数）	CloudWatch	—

```
pulumi config set aws:region ap-northeast-1
pulumi config set allowedOrigins "https://example.com"
pulumi config set alarmEmail your@email.com
pulumi config set staticSiteDomain "aws.example.com"        # カスタムドメイン（オプション）
pulumi config set staticSiteAcmCertificateArn "arn:..."     # ACM 証明書（オプション）
```

⚠ **本番環境スタック向け重要事項：** `pulumi up --stack production` を実行する前に、必ず `customDomain` と `acmCertificateArn` を設定する。設定がないと、CloudFront はデフォルト証明書に戻り、 すべての HTTPS ビジターに対して破断する。

```
pulumi config set customDomain www.aws.ashnova.jp --stack production
pulumi config set acmCertificateArn \
    arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19c4460dcde5 \
  --stack production
```

ACM 証明書は `us-east-1` に存在する必要があります（CloudFront に必須）。現在の証明書は 2027-03-12 に失効します。

```
pulumi stack output api_url                    # API Gateway URL
pulumi stack output cloudfront_url             # CloudFront URL
pulumi stack output cloudfront_distribution_id # CloudFront Distribution ID
pulumi stack output frontend_bucket_name       # S3 バケット名
pulumi stack output lambda_function_name       # Lambda 関数名
pulumi stack output cognito_user_pool_id
pulumi stack output cognito_client_id
```

ディレクトリ：`infrastructure/pulumi/azure/`

Pulumi 論理名	Azure リソース	名前パターン
`resource-group`	リソースグループ	`{project}-{stack}-rg`
`functions-storage`	ストレージアカウント	`mcadfunc{suffix}`
`frontend-storage`	ストレージアカウント	`mcadweb{suffix}`
`landing-storage`	ストレージアカウント	`mcadlanding{suffix}`
`function-app`	Azure Functions	`{project}-{stack}-func`
`cosmos-account`	Cosmos DB アカウント	—
`frontdoor-profile`	Front Door プロファイル	`{project}-{stack}-fd`
`azure-ad-app`	Azure AD アプリケーション	—
`spa-rule-set`	AFD RuleSet	`SpaRuleSet`
`spa-rewrite-rule`	AFD ルール	`SpaIndexHtmlRewrite`
Action Groups + Alerts	Azure Monitor	—

SPA ルーティング：`SpaRuleSet` はすべての非静的な `/sns/*` リクエストを `/sns/index.html` にリライトして、React クライアント側ルーティングが直接 URL アクセスとページリロード時に機能するようにします。RuleSet 名は**英数字のみ**（ハイフンなし）である必要があります。条件あたり最大 10 個の match_values。

```
pulumi config set azure-native:location japaneast
pulumi config set environment staging
pulumi config set alarmEmail your@email.com
pulumi config set staticSiteDomain "azure.example.com"  # オプション
```

```
pulumi stack output api_url
pulumi stack output frontdoor_url
pulumi stack output frontend_storage_name
pulumi stack output azure_ad_tenant_id
pulumi stack output azure_ad_client_id
```

ディレクトリ：`infrastructure/pulumi/gcp/`

Pulumi 論理名	GCP リソース	名前
`frontend-bucket`	GCS バケット	`ashnova-{project}-{stack}-frontend`
`uploads-bucket`	GCS バケット	`ashnova-{project}-{stack}-uploads`
`backend-bucket`	Compute バックエンドバケット	`{project}-{stack}-cdn-backend`
`cdn-ip-address`	グローバル外部 IP	`{project}-{stack}-cdn-ip`

Pulumi 論理名	GCP リソース	名前
`url-map`	URL マップ	—
`cloud-run-service`	Cloud Run	`{project}-{stack}-api`
`firestore-db`	Firestore	（デフォルト）
`managed-ssl-cert`	SSL 証明書	オプション
Alert Policies（複数）	Cloud Monitoring	—

```
pulumi config set gcp:project ashnova
pulumi config set environment staging
pulumi config set alarmEmail your@email.com
pulumi config set staticSiteDomain "gcp.example.com"  # オプション
pulumi config set monthlyBudgetUsd 50                 # 本番環境のみ
```

```
pulumi stack output api_url
pulumi stack output cdn_url
pulumi stack output cdn_ip_address
pulumi stack output frontend_bucket_name
```

GCP 固有の注記：

- `uploads-bucket`（`ashnova-{project}-{stack}-uploads`）は `allUsers:objectViewer` を持つ — パブリック。`frontend-bucket` にパブリック読み込みを付与しないこと。
- `ManagedSslCertificate` は `ignore_changes=["name", "managed"]` を使用して、名前ハッシュが変わったときに Pulumi が証明書の置換を試みるのを防ぎます（証明書が HTTPS プロキシに接続されている場合、GCP は 400 を返します）。
- `pulumi up` が URLMap の `Error 412: Invalid fingerprint` で失敗する場合、`pulumi up` 前に `pulumi refresh --yes --skip-preview` ステップを追加します。
- Firebase 認可ドメインは Identity Toolkit Admin v2 API 経由で更新する必要があります（`x-goog-user-project` ヘッダーが必須）。これは `deploy-gcp.yml` で自動化されています。

```
pulumi import gcp:storage/bucket:Bucket uploads-bucket \
  ashnova-multicloud-auto-deploy-staging-uploads --stack staging

pulumi refresh --yes --skip-preview --stack staging
pulumi up --yes --stack staging
```

```
export AZURE_CLIENT_ID=$(echo $AZURE_CREDENTIALS | jq -r '.clientId')
export AZURE_CLIENT_SECRET=$(echo $AZURE_CREDENTIALS | jq -r '.clientSecret')
export AZURE_SUBSCRIPTION_ID=$(echo $AZURE_CREDENTIALS | jq -r '.subscriptionId')
export AZURE_TENANT_ID=$(echo $AZURE_CREDENTIALS | jq -r '.tenantId')
```

```
pulumi stack export | \
  python3 -c "import sys,json; d=json.load(sys.stdin);
    d['deployment']['pending_operations']=[]; print(json.dumps(d))" | \
  pulumi stack import --force
```

2つのオプション（詳細は `LAMBDA_LAYER_OPTIMIZATION.md` を参照）：

オプション A — Klayers（デフォルト、推奨）：ビルド不要。パブリックなコミュニティ管理 Lambda Layers を使用。Pulumi 設定で有効化：

```
pulumi config set use_klayers true
```

オプション B — カスタムレイヤー（完全な制御、特定バージョン）：`./scripts/build-lambda-layer.sh` でビルド（boto3 / Azure / GCP SDK を ZIP から除外）。

```
./scripts/build-lambda-layer.sh

aws lambda publish-layer-version \
  --layer-name multicloud-auto-deploy-staging-dependencies \
  --zip-file fileb://services/api/lambda-layer.zip \
  --compatible-runtimes python3.12 \
  --region ap-northeast-1

cd services/api
cp -r app .build/package/
cp index.py .build/package/
cd .build/package && zip -r ../../lambda.zip .

aws lambda update-function-code \
  --function-name multicloud-auto-deploy-staging-api \
  --zip-file fileb://lambda.zip
```

→ 05 — CI/CD パイプライン

Chapter 7

CI/CDパイプライン（日本語訳）

Part II — Architecture & Design | Parent: AI_AGENT_GUIDE.md

⚠ リポジトリルートの `.github/workflows/` を常に編集してください。`multicloud-auto-deploy/.github/workflows/` は CI から無視されます。

ファイル	トリガー条件	ターゲット	環境
`deploy-aws.yml`	push: develop/main, 手動	Lambda + API	staging / production
`deploy-azure.yml`	push: develop/main, 手動	Functions + API	staging / production
`deploy-gcp.yml`	push: develop/main, 手動	Cloud Run (API)	staging / production
`deploy-frontend-aws.yml`	push: develop/main, 手動	S3 `sns/`	staging / production
`deploy-frontend-azure.yml`	push: develop/main, 手動	Blob `$web/sns/`	staging / production
`deploy-frontend-gcp.yml`	push: develop/main, 手動	GCS `sns/`	staging / production
`deploy-frontend-web-aws.yml`	push: develop/main, 手動	Lambda (frontend-web)	staging / production

ファイル	トリガー条件	ターゲット	環境
`deploy-frontend-web-azure.yml`	push: develop/main, 手動	Functions (frontend-web)	staging / production
`deploy-frontend-web-gcp.yml`	push: develop/main, 手動	Cloud Run (frontend-web, Docker)	staging / production
`deploy-landing-aws.yml`	push: develop/main, 手動	S3 /	staging / production
`deploy-landing-azure.yml`	push: develop/main, 手動	Blob `$web/`	staging / production
`deploy-landing-gcp.yml`	push: develop/main, 手動	GCS /	staging / production

```
develop  →  staging
main     →  production  ⚠ すぐに本番環境に反映されます
```

変更されたパス	トリガーされるワークフロー
`services/api/**`	deploy-{aws,azure,gcp}.yml
`infrastructure/pulumi/**`	deploy-{aws,azure,gcp}.yml
`services/frontend_react/**`	deploy-frontend-{aws,azure,gcp}.yml
`services/frontend_web/**`	deploy-frontend-web-{aws,azure,gcp}.yml
`static-site/**`	deploy-landing-{aws,azure,gcp}.yml

単一の信頼できるソース として機能する、環境ごとの安定した値。以前は GitHub Secrets、インラインの `case/esac` ブロック、gitignore された `Pulumi.*.yaml` ファイルに分散していました。

```
.github/config/
  aws.production.env
  aws.staging.env
  azure.production.env
  azure.staging.env
  gcp.production.env
  gcp.staging.env
```

クラウド	キー	例
すべて	CLOUD_CUSTOM_DOMAIN	www.gcp.ashnova.jp
Azure	CLOUD_AZURE_CLIENT_ID	0b926ff6-... (AD app registration)
Azure	CLOUD_AZURE_TENANT_ID	a3182bec-...
AWS	CLOUD_ACM_CERT_ARN	arn:aws:acm:us-east-1:...
AWS	CLOUD_CLOUDFRONT_DOMAIN	d1qob7569mn5nw.cloudfront.net
AWS	CLOUD_CLOUDFRONT_DISTRIBUTION_ID	E214XONKTXJEJD

すべてのメインデプロイワークフロー (deploy-aws.yml, deploy-azure.yml, deploy-gcp.yml) は、**Pulumi スタック名の決定**の直後に**クラウド設定を読み込む**ステップを含みます。

```
CONFIG_FILE=".github/config/aws.${STACK_NAME}.env"
source "$CONFIG_FILE"
echo "custom_domain=$CLOUD_CUSTOM_DOMAIN" >> $GITHUB_OUTPUT
```

その後のすべてのステップは、フォールバックロジックの代わりに ${{ steps.cloud_config.outputs.custom_domain }} などを参照します。

イベント	アクション
カスタムドメイン変更	関連する .env ファイルの CLOUD_CUSTOM_DOMAIN を更新
ACM 証明書更新 (AWS)	CLOUD_ACM_CERT_ARN を更新
Azure AD アプリ再登録	CLOUD_AZURE_CLIENT_ID を更新
新しいスタック追加	新しい <cloud>.<stack>.env ファイルを追加

リポジトリレベルの GitHub Secrets は**環境に依存しません**。リポジトリレベルで AZURE_CUSTOM_DOMAIN=www.azure.ashnova.jp を設定すると、ステージングデプロイも本番環境のドメインを受け取り Secrets はこの問題を解決しますが、GitHub UI で設定する必要があり、コードレビューには表示されません。.github/config/ のコンフィグファイルは git でトラッキングされ、diff で表示され、GitHub UI のメンテナンスは不要です。

Secret	用途
AWS_ACCESS_KEY_ID	IAM アクセスキー
AWS_SECRET_ACCESS_KEY	IAM シークレットキー
PULUMI_ACCESS_TOKEN	Pulumi Cloud 認証トークン

> ⚠ AWS_CUSTOM_DOMAIN, AWS_ACM_CERTIFICATE_ARN, AWS_CLOUDFRONT_DOMAIN は**使用されなくなりました**。これらの値は .github/config/aws.<stack>.env から読み込まれます。

Secret	用途
`AZURE_CREDENTIALS`	Service Principal JSON (`az ad sp create-for-rbac --sdk-auth`)
`AZURE_SUBSCRIPTION_ID`	サブスクリプション ID
`AZURE_RESOURCE_GROUP`	リソースグループ名
`PULUMI_ACCESS_TOKEN`	Pulumi Cloud 認証トークン

> ⚠ `AZURE_CUSTOM_DOMAIN` は **`deploy-azure.yml` では使用されなくなりました。** `CLOUD_CUSTOM_DOMAIN`, `CLOUD_AZURE_CLIENT_ID`, `CLOUD_AZURE_TENANT_ID` は `.github/config/azure.<stack>.env` から読み込まれます。

Secret	用途
`GCP_CREDENTIALS`	Service Account JSON
`GCP_PROJECT_ID`	プロジェクト ID (`ashnova`)
`GCP_API_ENDPOINT`	Cloud Run API URL (frontend-web `API_BASE_URL` 用)
`FIREBASE_API_KEY`	Firebase Web API キー (frontend-web 認証用)
`FIREBASE_AUTH_DOMAIN`	Firebase Auth ドメイン (frontend-web 認証用)
`FIREBASE_APP_ID`	Firebase App ID (frontend-web 認証用)
`PULUMI_ACCESS_TOKEN`	Pulumi Cloud 認証トークン

```
1. Checkout
2. AWS 認証 (aws-actions/configure-aws-credentials)
3. Python 3.12 セットアップ
4. Pulumi ログイン (PULUMI_ACCESS_TOKEN)
5. Pulumi スタック名決定 → STACK_NAME = staging | production
6. クラウド設定を読み込む → source .github/config/aws.${STACK_NAME}.env
   # 出力: custom_domain, acm_cert_arn, cloudfront_domain
7. pulumi up --stack staging # インフラストラクチャ作成更新/
8. Pulumi 出力取得 → バケット名、CloudFront 、IDCognito ID など
9. Lambda Layer ビルド (./scripts/build-lambda-layer.sh)
10. Lambda Layer を公開 (name = multicloud-auto-deploy-${STACK_NAME}-dependencies)
11. アプリコードを ZIP でパッケージ (app/ + index.py)
12. Lambda 関数コードを更新
13. Lambda 環境変数を更新
    - CLOUD_PROVIDER=aws
    - AUTH_DISABLED=false
    - AUTH_PROVIDER=cognito
    - COGNITO_USER_POOL_ID (Pulumi 出力から)
    - COGNITO_CLIENT_ID (Pulumi 出力から)
```

```
1. Checkout
```

```
2. AWS 認証
3. Node.js セットアップ
4. npm ci
5. Pulumi 出力から S3 バケット名と CloudFront ID を取得
6. VITE_API_URL を設定して npm run build を実行
   # vite.config.ts: base="/sns/" は既に設定されています
7. aws s3 sync dist/ s3://{bucket}/sns/ --delete
8. CloudFront キャッシュ無効化 (/*)
```

```
gh workflow run deploy-aws.yml \
  --ref develop \
  -f environment=staging

gh workflow run deploy-aws.yml \
  --ref main \
  -f environment=production

gh run list --workflow=deploy-aws.yml --limit 5
gh run watch <run-id>
```

```
gh workflow run deploy-aws.yml \
  --ref develop \
  -f environment=staging

gh workflow run deploy-frontend-gcp.yml \
  --ref main \
  -f environment=production

gh run list --workflow=deploy-aws.yml --limit 5
gh run watch <run-id>
```

ワークフロー	ブランチ	ステータス	コミット
Deploy Frontend Web to GCP	develop	☒	`0ed0805`
Deploy to GCP (Pulumi + API)	develop	☒	`0ed0805`
Deploy Frontend to GCP	develop	☒	`591ce0b`
Deploy Frontend to AWS	develop	☒	`591ce0b`
Deploy Frontend to Azure	develop	☒	`591ce0b`
Deploy Landing to GCP	develop	☒	`591ce0b`
Deploy Landing to AWS	develop	☒	`591ce0b`
Deploy Landing to Azure	develop	☒	`591ce0b`

バグ	症状	修正
ワークフロー重複	ブディレクトリ編集のみで CI に反映されない	ルート `.github/workflows/` を編集
`deploy-landing-gcp` 認証方式	`workload_identity_provider` (secret 未設定)	`credentials_json: ${{ secrets.GCP_CREDENTIALS }}` に変更
`deploy-landing-aws` 認証方式	`role-to-assume` (secret 未設定)	`aws-access-key-id` + `aws-secret-access-key` に変更
フロントエンド	`dist/` がランディングページに同期された	`dist/` を `sns/` プレフィックスの下に同期に変更
AWS/Azure staging で `AUTH_DISABLED=true`	デプロイ後も認証が無効のまま	条件付きを削除；常に `AUTH_DISABLED=false` に設定
GCP URLMap `Error 412: Invalid fingerprint`	Pulumi 状態と GCP リソースが非同期	`deploy-gcp.yml` で `pulumi up` の前に `pulumi refresh --yes` `--skip-preview` を追加
GCP Cloud Build `missing main.py`	Cloud Build が `--entry-point` を指定してもソースを拒否	デプロイ ZIP 内に `services/api/function.py` を `main.py` としてコピー
Azure `ModuleNotFoundError: pulumi_azuread`	`pulumi-azuread` が誤って `requirements.txt` から削除された	`infrastructure/pulumi/azure/requirements.txt` に `pulumi-azuread>=6.0.0,<7.0.0` を復元
Azure `ModuleNotFoundError: monitoring`	`monitoring.py` が `main` には存在するが `develop` にない	`infrastructure/pulumi/{aws,azure,gcp}/monitoring.py` を `develop` に追加
Azure FC1: 誤った `az functionapp create`	`--consumption-plan-location` で Y1 Dynamic 作成 → AFD で stale TCP 502	FC1 FlexConsumption 作成のため `--flexconsumption-location` を使用
GCP `Error 409: uploads-bucket exists`	Pulumi が GCP 内に既存のバケットを作成しようとした	`pulumi up` 前に既存バケットの `pulumi import` ステップを追加
GCP `ManagedSslCertificate` `Error 400: in use`	ブランチ間で SSL 証明書名ハッシュが変更；GCP が削除を拒否	証明書リソースに `ignore_changes=["name", "managed"]` を追加 + `pulumi import` ステップ
Firebase Auth: 新ドメインが未認可	`signInWithPopup` 失敗 — ドメインが Firebase 認可ドメインにない	`deploy-gcp.yml` で Pulumi 出力読み取り後に「Firebase 認可ドメイン更新」ステップを実行

バグ	症状	修正
Azure dead CI ステップ (SSR Lambda)	React SPA 移行後も古い `frontend_web` Lambda/S3 ステップが実行	`deploy-aws.yml`, `deploy-azure.yml`, `deploy-gcp.yml` から 168 行の不要コードを削除 (コミット `1ae65f5`)
Lambda Layer 名が常に `staging`	`github.event.inputs.environment || 'staging'` → push イベントには `inputs` がない → production デプロイが `...-staging-dependencies` という名前のレイヤーを作成	`steps.set_stack.outputs.stack_name` に変更
環境変数が消去される / デプロイごとに誤った環境	`Pulumi.*.yaml` が gitignore 対象 → `grep` が常に失敗 → リポジトリレベル `secrets` フォールバック → Secrets は環境非依存 → staging と production が同じドメイン/client_id	コミット版の単一真実源として `.github/config/<cloud>.<stack>.env` を導入；すべての `case/esac` フォールバックを削除
Azure AD Client ID が CI	`pulumi stack output azure_ad_client_id` が空を返す → `VITE_AZURE_CLIENT_ID=''` → 認証プロバイダーが `'none'` → デプロイ後に空ログインページで「認証設定が不完全」エラー	`AZURE_CLIENT_ID` を `cloud_config` ステップ（設定ファイル）から取得、Pulumi 出力からは取得しない
AWS/Azure/GCP の誤ったカスタムドメイン	`secrets.AZURE_CUSTOM_DOMAIN` (リポジトリレベル) に staging ドメインが含まれる → production フロントエンドが `staging.azure.ashnova.jp` redirect URI でビルドされる	すべてのカスタムドメイン参照を `steps.cloud_config.outputs.custom_domain` に置換
AWS `ResourceConflictException` 競合	Pulumi の設定更新中に `update function-code` が発行される	コード/設定更新の前後に `aws lambda wait function-updated` を追加
Azure FC1: zip deploy で `InaccessibleStorageException`	`config-zip` が `BlobUploadFailedException: Name or service not known` で即座に失敗 — `Kudu StorageAccessibleCheck` 検証失敗 → すべての試行でステータス=3	ARM GET `functionAppConfig.deployment.storage.value` → `az storage account create` で欠落アカウントを再作成 → `deploy-azure.yml` で `AzureWebJobsStorage__accountName` 設定を更新
Azure FC1: `"on-going"` メッセージが CI エラー扱い	`config-zip` が `ERROR: Deployment is still on-going...` を返す — 古い `grep ERROR:` パターンがこの FC1 非同期受付応答にマッチ → パイプラインが即座にコード 1 で終了	一般的な `ERROR:` チェックの**前**に `on-going` サブストリングを検出 → `DEPLOY_SUCCESS=true` 設定 + `sleep 120` で完了を待機
Azure FC1: デプロイ後の古い `WEBSITE_RUN_FROM_PACKAGE` で 404	デプロイがエラーなく完了するが全関数ルートが 404 を返す；設定内の古い期限切れ `SAS URL` が新デプロイパッケージをオーバーライド	`deploy-azure.yml` で `config-zip` デプロイの前に毎回 `az functionapp config appsettings delete --setting-names WEBSITE_RUN_FROM_PACKAGE` を実行

バグ	症状	修正
GitHub Actions YAML ブロックスカラー内の Python heredoc	`run: \\|` ブロック内の `python3 - <<'EOF' ... EOF` が `yaml.scanner.ScannerError: while scanning a simple key` を引き起こす — YAML パーサーが 0 列目の `EOF` でエラー	シェルステップでの JSON 操作には常に `jq` を使用；GitHub Actions `run:` ブロック内で Python heredoc (`<<'EOF'`) は使用しない
Azure FC1: `POST /uploads/presigned-urls` → 500	`AZURE_STORAGE_ACCOUNT_NAME` / `AZURE_STORAGE_ACCOUNT_KEY` / `AZURE_STORAGE_CONTAINER` が production Function App で欠落 → `generate_blob_sas(account_name=None)` が例外発生	`frontend_storage_name` Pulumi 出力からキーを取得；`deploy-azure.yml` の `az functionapp config appsettings set` に 3 設定を追加 (コミット `856d6dc`)

各アーティファクトは **`A.B.C.D`** (4桁) 形式でバージョン追跡されます。バージョン定義ファイル: `versions.json` `X.Y.Z` から `A.B.C.D` に変更 (2026-02-24, commit `c2f6870`)

コンポーネント	バージョン	ステータス
`aws-static-site`	`1.0.84.204`	stable
`azure-static-site`	`1.0.84.204`	stable
`gcp-static-site`	`1.0.84.204`	stable
`simple-sns`	`1.0.84.204`	stable

桁	記号	意味	インクリメント条件	担当
1	A	メジャー	手動指示のみ	`./scripts/bump-version.sh major all`
2	B	マイナー	手動指示のみ	`./scripts/bump-version.sh minor all`
3	C	プッシュ数	`git push` のたびに +1（リセットなし）	GitHub Actions `version-bump.yml` が自動実行
4	D	コミット数	`git commit` のたびに +1（リセットなし）	`.githooks/pre-commit` が自動実行

重要: どの桁をインクリメントしても他の桁はリセットしません。 パスフィルタ: `static-site/aws/**` → A のみ、`services/frontend_react/**` → SNS のみ、それ以外 → 全コンポーネント

ファイル	役割
`versions.json`	各コンポーネントの現在バージョンを保持
`scripts/bump-version.sh`	バージョン操作スクリプト
`.githooks/pre-commit`	コミット前に D（コミット数）を自動インクリメント
`.github/workflows/version-bump.yml`	push 時に C（プッシュ数）を自動インクリメント

```
make hooks-install
```

```
./scripts/bump-version.sh show

./scripts/bump-version.sh commit all

./scripts/bump-version.sh push all

./scripts/bump-version.sh minor all

./scripts/bump-version.sh major all

./scripts/bump-version.sh set all 1.0.84.204
```

対象	スキップ方法
pre-commit hook (D +=1)	環境変数: `SKIP_VERSION_BUMP=1 git commit -m "..."`
GitHub Actions (C +=1)	コミットメッセージに `[skip-version-bump]` を含める

```
SKIP_VERSION_BUMP=1 git commit -m "docs: update readme"

git commit -m "chore: some change [skip-version-bump]"
```

→ 06 — 環境ステータス

Chapter 8

ステータス・環境情報（日本語訳）

Part III — Operations | Parent: AI_AGENT_GUIDE.md 最終確認: 2026-02-27 Session 4 (アーキテクチャ図アイコン強化完了 ☒ / デュアルアイコン配置実装 ☒ / ドキュメント更新完了 ☒) 前回: 2026-02-27 Session 3 (S1・S2・Task 13 完了 ☒ / エンドポイント本番運用 ☒ / README セキュリティ情報反映 ☒)

タスク	結果	状況
デュアルアイコン配置実装	24px（24px）+ テキスト横（20px）の2箇所にアイコン表示	☒
generate_icon_diagram.py / JavaScript 更新	2-tier icon mapping / text要素の両方に対応するスマート検出ロジック実装	☒
3環境HTML再生成	staging/production/combined の3ファイル全て再生成	☒
CLOUD_ARCHITECTURE_MAPPER.md 更新	FEATURE_MAPPER.md / Recommendations / Known Limitations セクション更新	☒
CHANGELOG.md 更新	2026-02-27 エントリに詳細な実装内容とファイルサイズ更新	☒
README.md アーキテクチャリンク追加	インタラクティブHTML図への直接リンク追加済み	☒

アイコン配置:

1. **ノード左上コーナー** (24x24px):
   - 位置: (rect.x + 6, rect.y + 6)
   - 目的: リソースタイプの素早い視覚識別
   - ノードサイズに関わらず常に表示
2. **テキストインライン** (20x20px):
   - 位置: ノードラベルテキスト左の4px

- 目的: テキスト関連付けで読みやすさ向上
- スマート検出: `foreignObject` と SVG `text/tspan` 要素の両方に対応

JavaScript DOM 操作:

```
// 1. 左上コーナー
const topIcon = createSVGImage(iconUrl, rectX + 6, rectY + 6, 24, 24);
node.appendChild(topIcon);

// 2. テキストインライン (foreignObject vs text 要素検出)
if (textElement.tagName === "foreignObject") {
  textX = parseFloat(textElement.getAttribute("x") || 0);
  textY =
    parseFloat(textElement.getAttribute("y") || 0) + height / 2 - iconSize / 2;
} else {
  const tspan = textElement.querySelector("tspan");
  textX = parseFloat((tspan || textElement).getAttribute("x") || 0);
  textY =
    parseFloat((tspan || textElement).getAttribute("y") || 0) - iconSize / 2;
}
const textIcon = createSVGImage(iconUrl, textX - 24, textY, 20, 20);
labelGroup.insertBefore(textIcon, labelGroup.firstChild);
```

ファイル	サイズ	アイコン	説明
`architecture.staging.html`	78KB	AWS (5) + Azure (4) + GCP (5)	ステージング環境（デュアルアイコン配置）
`architecture.production.html`	78KB	AWS (5) + Azure (4) + GCP (5)	本番環境（デュアルアイコン配置）
`architecture-combined.html`	84KB	AWS (5) + Azure (4) + GCP (5)	統合ビュー（color-coded nodes）

アイコンソース:

- AWS: 14KB (cloudfront, lambda, s3, dynamodb, api-gateway)
- Azure: 16KB (cdn, function, storage, cosmos-db)
- GCP: 20KB (cdn, run, storage, firestore, load-balancer)
- **埋め込みアセット合計**: ~50KB Base64 エンコード SVG データ URI

☒ **CLOUD_ARCHITECTURE_MAPPER.md**:

- Features セクション: 「Dual icon placement」箇条書きを追加
- Technical Details: デュアル配置ロジックを含む JavaScript コードサンプルを拡張
- Known Limitations: テキストインライン配置の分散に関する注記を追加

☒ **CHANGELOG.md**:

- 2026-02-27 エントリに詳細な実装ノートを更新

- ファイルサイズ変更を追加（staging/production で 85KB → 78KB）
- デュアルアイコン配置戦略を文書化

☒ **README.md**:

- アーキテクチャセクションに 3 つのインタラクティブ HTML 図へのリンクを追加
- 図リンクに視覚的インジケーター（☒）を追加

タスク	結果	状況
S1: GCP ステージング pulumi up	HTTPS リダイレクト / 監査ログ反映済み（unchanged 33）	☒
S1: AWS 本番 pulumi up	CloudTrail / CORS 反映済み（unchanged 40）	☒
S1: GCP 本番 refresh+up	State drift 解決後、監査ログ反映済み（unchanged 34）	☒
S1: Azure ステージング pulumi up	Key Vault パージ保護反映（updated 1, unchanged 32）	☒
S1: Azure 本番 pulumi up	Key Vault パージ保護本番反映済み（unchanged 33）	☒
S2: Function App マネージド ID	ステージング/本番 両方に SystemAssigned MSI 割り当て成功	☒
Task 13: README 更新	エンドポイント・セキュリティ実装・テスト結果・デプロイ状況を最新情報に更新	☒
Task 20/21 補完: Key Vault 診断設定（CLI）	`az monitor diagnostic-settings create` で Log Analytics との統合が完了（AuditEvent ストリーミング）	☒

クラウド	CDN / フロントエンド	API	ステータス
AWS	CloudFront ☒	API Gateway	☒ 本番運用中
Azure	Front Door ☒	Functions	☒ 本番運用中
GCP	CDN（www.gcp.ashnova.jp） ☒	Cloud Functions	☒ 本番運用中

対策	AWS	Azure	GCP	ステータス
CORS 絞り込み	☒	☒	☒	☒ 本番反映
CloudTrail / 監査ログ	☒	☒	☒	☒ 本番反映
Key Vault パージ保護	—	☒	—	☒ 本番反映

対策	AWS	Azure	GCP	ステータス
Key Vault 診断ログ	—	☒（Log Analytics）	—	☒ 本番反映
マネージド ID	—	☒	—	☒ ステージング/本番
HTTPS リダイレクト	—	—	☒	☒ 本番反映
Cloud Armor	—	—	☒	☒ 本番反映

タスク	結果	状況
GCP 監査ログ再有効化 (IAMAuditConfig)	staging/production で Cloud Audit Logs (`allServices`) を有効化。Pulumi リソース作成完了	☒
ADC (Application Default Credentials) 更新	`gcloud auth application-default login` で sat0sh1kawada00@gmail.com 再認証。quota project=ashnova に設定	☒
GCP 課金アカウント設定	Pulumi設定に `billingAccountId: 01F139-282A95-9BBA25` を追加	☒
課金予算エラー軽減	ADC quota project エラー回避。monitoring.py で `billing_account_id` をoptional パラメータ化。コードで `enable_billing_budget=False` にデフォルト無効化	☒
GCP side budget cleanup	`gcloud billing budgets delete` で古いbudgetリソース削除	☒
monitoring.py リファクタリング	`create_billing_budget()` に `billing_account_id` 追加。`setup_monitoring()` で `billing_budget=None` when not enabled	☒

- **infrastructure/pulumi/gcp/main.py**: `enable_billing_budget = False` (ハードコード無効化) + `billing_account_id=None` を常時 monitoring へ参照
- **infrastructure/pulumi/gcp/monitoring.py**: `billing_account_id` パラメータ追加、optional化。budget作成条件を `if stack=="production" and billing_account_id:` に変更
- GCP production `pulumi up` が preview conflict 状態。コード修正後は再実行予定（次セッション）
- staging/production 共に監査ログ有効化完了、billing warning 回避完了
- billingbudgets API 認証エラーは ADC quotaProjectで解消するが、service account接続時の権限不足で deprecated。本番運用では GCP service account 設定または ignore_changes で対応推奨

クラウド	ランディング (/)	SNS アプリ (/sns/)	API
GCP	☒	☒	☒ Cloud Run (2026-02-24)
AWS	☒	☒	☒ Lambda (完全運用中, 2026-02-24)
Azure	☒	☒	☒ Azure Functions FC1 (2026-02-24)

```
CDN URL  : https://d1tf3uumcm4bo1.cloudfront.net
API URL  : https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com
```

リソース	名前 / ID	ステータス
CloudFront	`E1TBH4R432SZBZ` (PriceClass_200: NA/EU/JP/KR/IN)	☒
CloudFront RHP	`multicloud-auto-deploy-staging-security-headers` (HSTS + CSP + 4 headers)	☒
S3 (フロントエンド)	`multicloud-auto-deploy-staging-frontend`	☒
S3 (画像)	`multicloud-auto-deploy-staging-images` (CORS: *)	☒
Lambda (API)	`multicloud-auto-deploy-staging-api` (Python 3.12, **1769MB** = 1 vCPU)	☒
Lambda (frontend-web)	`multicloud-auto-deploy-staging-frontend-web` (512MB, 30s)	☒
API Gateway	`z42qmqdqac` (HTTP API v2)	☒
DynamoDB	`multicloud-auto-deploy-staging-posts` (PAY_PER_REQUEST)	☒
Cognito	Pool `ap-northeast-1_AoDxOvCib` / Client `1k41lqkds4oah55ns8iod30dv2`	☒
WAF	WebACL が CloudFront に紐付け済み	☒

確認済み機能 (2026-02-22):

- Cognito ログイン → `/sns/auth/callback` → セッションクッキー設定 ☒
- ポストフィード、最大10枚のマルチイメージ投稿作成 ☒
- 画像が正しく表示（S3 プリサインド GET URL、1時間有効） ☒
- `GET /posts/{post_id}` 個別ポスト表示 ☒
- プロフィールページ（ニックネーム、アバター、自己紹介） ☒
- ニックネームがポストリストに保存・表示 ☒
- 画像アップロード: S3 プリサインド URL、`MAX_IMAGES_PER_POST` でサーバーサイド制限 ☒
- `GET /limits` エンドポイント（認証不要）が `{"maxImagesPerPost": 10}` を返す ☒
- ログアウト → Cognito ホステッドログアウト → `/sns/` へリダイレクト ☒

- CI/CD パイプライン: push ごとに環境変数が正しく設定される ☒
- フロントエンドバンドルが `VITE_BASE_PATH=/sns/` でビルドされ、アセットパスが正常 ☒
- CloudFront カスタムエラーページ: `/sns/index.html` (403+404) ☒
- CloudFront Response Headers Policy: HSTS/CSP(`upgrade-insecure-requests`)/X-Content-Type-Options/X-Frame-Options/Referrer-Policy/XSS-Protection ☒ (2026-02-23)
- CloudFront PriceClass_200: 日本・韓国・インドのエッジを使用 ☒ (旧: PriceClass_100 = 米国/欧州のみ)
- OAuth フロー PKCE (S256) 実装: `response_type=code` + code_verifier/challenge ☒ (2026-02-23)
- Cognito `implicit` フロー削除: `allowed_oauth_flows=["code"]` のみ ☒ (2026-02-23)
- S3 パブリックアクセス完全遮断: `BlockPublicAcls/IgnorePublicAcls/BlockPublicPolicy/RestrictPublicBuckets=True` ☒ (2026-02-23)
- S3 バケットポリシー OAI 専用: `Principal:*` 削除 ☒ (2026-02-23)
- Lambda `_resolve_image_urls`: `http://` URL をスキップして Mixed Content を防止 ☒ (2026-02-23)

現在のフロントエンドバンドル: `index-B0gzRu__.js` (2026-02-23 アップロード, PKCE対応)

AWS staging ビルドコマンド:

```
cd services/frontend_react
set -a && source .env.aws.staging && set +a
VITE_BASE_PATH=/sns/ npm run build
```

既知の制限:

- Production スタックは staging リソースと共有（独立した prod スタック未デプロイ）
- WAF ルールセット未調整
- `DELETE /posts` で SNS Unsubscribe 呼び出しが失敗する可能性（未テスト）

```
CDN URL  : https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net
API URL  :
    https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net
```

リソース	名前	ステータス
Front Door	`multicloud-auto-deploy-staging-fd` / endpoint: `mcad-staging-d45ihd`	☒
Storage Account	`mcadwebd45ihd`	☒
Function App	`multicloud-auto-deploy-staging-func` (Python 3.12, always-ready=1)	☒
Cosmos DB	`messages` (Serverless, db: messages / container: messages)	☒

リソース	名前	ステータス
Resource Group	`multicloud-auto-deploy-staging-rg`	☒

構成済み (2026-02-23):

- FlexConsumption always-ready インスタンス: `http=1` → コールドスタート解消 ☒

未解決の課題:

- `PUT /posts/{id}` のエンドツーエンド検証が不完全
- WAF 未構成 (Front Door Standard SKU)

```
CDN URL : http://34.117.111.182
API URL : https://multicloud-auto-deploy-staging-api-899621454670.asia-northeast1.run.app
```

リソース	名前 / ID	ステータス
グローバル IP	`34.117.111.182`	☒
GCS バケット (前面)	`ashnova-multicloud-auto-deploy-staging-frontend`	☒
GCS バケット (画像)	`ashnova-multicloud-auto-deploy-staging-uploads` (public read)	☒
Cloud Run (API)	`multicloud-auto-deploy-staging-api` (Python 3.12, **min=1**)	☒
Firestore	`(default)` — collections: messages, posts	☒
Backend Bucket	`multicloud-auto-deploy-staging-cdn-backend`	☒

確認済み機能 (2026-02-21):

- Firebase Google Sign-In → `/sns/auth/callback` → httponly Cookie セッション ☒
- ポストフィード、作成/編集/削除 ☒
- 画像アップロード: GCS プリサインド URL (IAM `signBlob` API 経由署名)、最大16ファイル/投稿 ☒
- アップロードされた画像がポストフィードに表示される ☒
- Firebase ID トークン自動更新 (`onIdTokenChanged`) ☒
- ダークテーマ背景 SVG (星空、リング) が正しくレンダリングされる ☒

修正済みの課題 (2026-02-21):

- `GcpBackend` に未実装の `like_post/unlike_post` 抽象メソッド → `TypeError` → `/posts` が 500 を返す → `like_post/unlike_post` のスタブ実装を追加（コミット `a9bc85e`）

- `frontend-web` Cloud Run で `API_BASE_URL` 未設定 → localhost:8000 へフォールバック → `gcloud run services update` で環境変数を設定
- Firebase Auth 未実装 → Google Sign-In フロー全体を実装（コミット `3813577`）
- `x-ms-blob-type` ヘッダーが GCS CORS に未登録 → CORS 更新 + uploads.js 修正（コミット `1cf53b7, b5b4de5`）
- GCS プリサインド URL 生成で `content_type` が `"image/jpeg"` にハードコード → `content_types[index]` を正しく使用（コミット `148b7b5`）
- Firebase ID トークン期限切れ（401）→ `onIdTokenChanged` で自動更新（コミット `8110d20`）
- CI/CD で `GCP_SERVICE_ACCOUNT` 環境変数欠落 → `deploy-gcp.yml` に追加（コミット `27b10cc`）
- CSS 背景 SVG が絶対パス `/static/` を使用 → 相対パス `./` に変更（コミット `0ed0805`）
- GCS アップロードバケットが非公開 → `allUsers:objectViewer` を付与 + Pulumi 定義に IAMBinding 追加（コミット `0ed0805`）

構成済み (2026-02-23):

- Cloud Run `--min-instances=1` → コールドスタート（最大5秒）解消 ☒
- `gcp_backend.py`: `google.auth.default()` を `__init__()` でキャッシュ → リクエストごとのメタデータサーバー呼び出し排除 ☒

残る課題:

- CDN に HTTPS 未構成（HTTP のみ）。`TargetHttpsProxy` + マネージド SSL 証明書が必要
- CDN 経由の SPA ディープリンクが HTTP 404 を返す（Cloud Run URL はブラウザで正常動作）

```
curl -I https://d1tf3uumcm4bo1.cloudfront.net/
curl -s https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com/health

curl -I https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net/
curl -s "https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net/health"

curl -s http://34.117.111.182/ | head -3
curl -s https://multicloud-auto-deploy-staging-api-899621454670.asia-northeast1.run.app/health
```

> 本番環境は独自の Pulumi スタック（デプロイ完了）を持ち、ステージングと完全に分離されています。フロントエンドは **React SPA**（Vite ビルド）として、CDN を経由するオブジェクトストレージから配信されます。`frontend_web`（Python SSR）は本番環境では使用されなくなりました。詳細マイグレーション情報: REACT_SPA_MIGRATION_REPORT.md

本番環境のすべての API エンドポイントおよび CDN が正常に稼働しており、3つのクラウド環境すべてで同

クラウド	CDN ランディング (/)	SNS アプリ (/sns/)	API
AWS	☒ HTTP 200	☒ React SPA	☒ /health /posts ok
Azure	☒ HTTP 200	☒ React SPA	☒ /api/health ok
GCP	☒ HTTP 200	☒ React SPA	☒ /health /limits ok

3つのクラウドプロバイダーの本番環境における CDN / フロントエンド配信エンドポイント、API エンドポイント、および Distribution ID / リソース識別子を以下に示します。

クラウド	CDN / エンドポイント	API エンドポイント	Distribution ID
AWS	`d1qob7569mn5nw.cloudfront.net` / `www.aws.ashnova.jp`	`https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com`	E2HAXONKTXJEJD
Azure	`mcad-production-diev0w-f9ekdhbb0bgaaf5h.z01.azurefd.net`	`https://mcad-production-func-cdne7ecbngnh0d0.japaneast-01.az`	mcad-production-diev0w
GCP	`www.gcp.ashnova.jp`	`https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app`	

AWS Production SNS App (`https://www.aws.ashnova.jp/sns/`):

AWS 本番環境の SNS アプリケーション構成の詳細。すべてのコアサービスが Pulumi により管理・デプロイされています。

項目	値
フロントエンド	React SPA — S3 `multicloud-auto-deploy-production-frontend/sns/`
CF Function	`spa-sns-rewrite-production` (LIVE) — `/sns/` → `/sns/index.html` 変換
Lambda (API)	`multicloud-auto-deploy-production-api`
API_BASE_URL	`https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com`
Cognito Pool	`ap-northeast-1_50La963P2`
Cognito Client	`4h3b285v1a9746sqhukk5k3a7i`
Cognito Redirect	`https://www.aws.ashnova.jp/sns/auth/callback`
DynamoDB	`multicloud-auto-deploy-production-posts`

`ashnova.jp` ドメインの3つのクラウド環境での SSL/TLS デプロイ状態。すべてのカスタムドメインが完全に運
による安全な通信が確立されています。

クラウド	URL	ステータス
AWS	https://www.aws.ashnova.jp	☒ **完全運用中** (HTTP/2 200, ACM 証明書設定済み)
Azure	https://www.azure.ashnova.jp	☒ **完全運用中** (HTTPS 200, DigiCert/GeoTrust 管理証明書)
GCP	https://www.gcp.ashnova.jp	☒ **完全運用中** (HTTPS 200, TLS 証明書 Pulumi 管理)

Landing page test (2026-02-23): `test-landing-pages.sh --env production` → **37/37 PASS (100%)** ☒

クラウド	作業内容	結果
AWS	ACM 証明書検証	☒ 証明書 `914b86b1` の `www.aws.ashnova.jp` (2027-03-12 有効) ISSUED 確認
AWS	`aws cloudfront update-distribution` でエイリアス+ACM設定	☒ Distribution `E214XONKTXJEJD` にエイリアス `www.aws.ashnova.jp` + 証明書 `914b86b1` を設定 → `NET::ERR_CERT_COMMON_NAME_INVALID` 解決 → HTTP/2 200 運用中
AWS	Production `frontend-web` Lambda 環境変数修正 (2026-02-21)	☒ `API_BASE URL` が空→`localhost:8000` フォールバック（原因: `deploy-frontend-web-aws.yml` がシークレット依存、本番シークレット未設定）→ CI/CD を Pulumi outputs 使用に更新（コミット `fd1f422`）
Azure	`az afd custom-domain create` + ルート紐付け	☒ DNS 承認 → 管理証明書成功 (GeoTrust, 2026-02-21 – 2026-08-21)
Azure	AFD ルート無効化→有効化 (200 運用中)	☒ エッジノードへのデプロイをトリガー → HTTPS 200 運用中
Azure	`az afd custom-domain update` (証明書エッジデプロイ)	☒ `CN=www.azure.ashnova.jp` 証明書を AFD POP に配布
Azure	`frontend-web` Function App 環境変数設定	☒ API_BASE_URL, AUTH_PROVIDER, AZURE_TENANT_ID, AZURE_CLIENT_ID 等を設定
Azure	Azure AD app redirect URI 追加	☒ `https://www.azure.ashnova.jp/sns/auth/callback` を追加
GCP	`pulumi up --stack production` (SSL証明書作成)	☒ 証明書 `multicloud-auto-deploy-production-ssl-cert-3ee2c3ce` PROVISIONING
GCP	ACTIVE 証明書 `ashnova-production-cert-4131` 追加	☒ HTTPS プロキシに追加 → `https://www.gcp.ashnova.jp` HTTPS 即座に運用開始

クラウド	作業内容	結果
GCP	Firebase 認可ドメイン更新	☒ Firebase Auth 認可ドメインに `www.gcp.ashnova.jp` を追加

- **GCP**: ☒ `ashnova-production-cert-c41311` を HTTPS プロキシから切り離し・削除済み (2026-02-24)。`multicloud-auto-deploy-production-ssl-cert-3ee2c3ce` のみ使用中。推奨される設定状態は以降の pulumi up では自動的に維持されます。

本番環境で検出された 7 つのイシューはすべて解決され、確認テストも全て PASS しています。

症状: `/api/health` → HTTP 404。Function App は Running 状態だが、Admin インターフェースで関数が表示されない。

根本原因 (多層的で複雑):

1. `AzureWebJobsStorage` が非存在ストレージ `multicloudautodeploa148` を指していた
2. `functionAppConfig.deployment.storage.value` が非存在 blob URL を指していた
3. **決定的原因**: `functionAppConfig.runtime.version = "3.12"` だが、デプロイされた zip は Python 3.11 (`cpython-311`) でビルドされていた → クラウド実行時にモジュールロード失敗 → 関数が読み込まれない

修正: Python 3.12 (`docker run python:3.12-slim`) でパッケージを正しいアーキテクチャ向けに再ビルド・デプロ

確認: `admin/functions` → `[{"name":"HttpTrigger"}]` ☒ / `/api/health` → HTTP 200 ☒

症状: `https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app/limits` へのリクエストが HTTP 404 を返す。直接 Cloud Run サービスへのアクセスも同様に失敗。

修正: GCP production API を最新リビジョン (`00013-big`) に再デプロイ。`CORS_ORIGINS` 環境変数に本番ドメイン `https://www.gcp.ashnova.jp` を追加して、クロスオリジンリクエストを許可。

確認済み: `GET /limits` → `{"maxImagesPerPost":10}` HTTP 200 ☒ CDN 経由のリクエストも成功

症状: CloudFront Distribution が HSTS、CSP、X-Frame-Options などのセキュリティヘッダーを返していない。

修正: 既存ポリシー `multicloud-auto-deploy-production-security-headers` (ID: `aaad020f-c94c-4143-ba2c-4b7921a1a6de`) を DefaultCacheBehavior と `/sns*` CacheBehavior の両方に適用。ETag が `E3P5ROKL5A1OLE` から `E3JWKAKR8XB7XF` に更新されました。

確認され: CloudFront Distribution `E214XONKTXJEJD` がすべてのセキュリティヘッダー（HSTS/CSP/X-Content-Type-Options/X-Frame-Options/Referrer-Policy/XSS-Protection）をレスポンスに含める ☒

症状: `https://www.aws.ashnova.jp/sns/` の SNS アプリで API 呼び出し時に “Network Error” が発生。Axios が HTTP status 0 を返す（CORS エラーの典型的な兆候）。

根本原因チェーン:

1. `deploy-aws.yml` の "Sync Pulumi Config" ステップが GitHub リポジトリシークレット `${{ secrets.AWS_CUSTOM_DOMAIN }}` = `staging.aws.ashnova.jp` を使用
2. `pulumi config set multicloud-auto-deploy-aws:customDomain "staging.aws.ashnova.jp"` が本番 `Pulumi.production.yaml` を誤って上書き
3. `pulumi stack output custom_domain` が `staging.aws.ashnova.jp` を返す（期待値: `www.aws.ashnova.jp`）
4. Lambda 環境変数 `CORS_ORIGINS = ...,https://staging.aws.ashnova.jp,...`（誤）→ FastAPI が `Origin: https://www.aws.ashnova.jp` を拒否
5. ブラウザが "Network Error" を報告

修正 (コミット `3ea6a08` v1.17.10):

- `deploy-aws.yml` を GitHub Secrets ではなく `Pulumi.${STACK_NAME}.yaml` から値を読むように修正
- React SPA 再ビルド・デプロイ（新バンドル `index-Ch-ro-3Y.js`）
- Lambda `CORS_ORIGINS` から `staging.aws.ashnova.jp` を即時削除

確認: Lambda `CORS_ORIGINS` = `https://d1qob7569mn5nw.cloudfront.net,https://www.aws.ashnova.jp,http://localhost:5173` ☒

症状: `https://www.azure.ashnova.jp/sns/profile` でプロフィール取得時に CORS エラー

根本原因:

1. Azure Function App は Kestrel がプラットフォームレベル CORS 判定を FastAPI `CORSMiddleware` の手前に実行
2. `deploy-azure.yml` が `secrets.AZURE_CUSTOM_DOMAIN` (= `staging.azure.ashnova.jp`) を使用
3. `CORS_ORIGINS` とプラットフォーム CORS の両方に `https://www.azure.ashnova.jp` が欠落

即時修正:

- `az functionapp config appsettings set ... CORS_ORIGINS=...,https://www.azure.ashnova.jp,...` ☒
- `az functionapp cors add --allowed-origins "https://www.azure.ashnova.jp"` ☒

根本修正 (v1.17.15): `deploy-azure.yml` を stack 名から `customDomain` を読むように変更

症状: `www.azure.ashnova.jp/sns/` でログイン後に `staging.azure.ashnova.jp/sns/` にリダイレクト

根本原因:

1. フロントエンドが `VITE_AZURE_REDIRECT_URI=https://staging.azure.ashnova.jp/sns/auth/callback` でビルドされていた
2. Azure AD アプリの redirect URIs に `www.azure.ashnova.jp` がなかった

即時修正:

- `az ad app update` で `www.azure.ashnova.jp` を redirect URIs に追加 ☒

- フロントエンドを `VITE_AZURE_REDIRECT_URI=https://www.azure.ashnova.jp/sns/auth/callback` で再ビルド → `index-CPcQQsCR.js` ☒

根本修正 (v1.17.16): `deploy-azure.yml` の全4箇所の `secrets.AZURE_CUSTOM_DOMAIN` を stack マッピングに変更

テスト結果 (2026-02-24):

すべての本番環境エンドポイントでテストを実施し、完全な機能動作を確認しました。

```
test-cloud-env.sh production → PASS: 14, FAIL: 0, WARN: 3
test-azure-sns.sh            → PASS: 10, FAIL: 0
test-gcp-sns.sh              → PASS: 10, FAIL: 0
```

---

commit `73af560` — `scripts/` 配下の4ファイルを改良

3クラウド統合ラッパー。すべてのクラウドを一括でテストし、最後にサマリーテーブルを表示する。

```
bash scripts/test-sns-all.sh --env production

bash scripts/test-sns-all.sh --env production --only azure

bash scripts/test-sns-all.sh --env production --write \
  --aws-username user@example.com --aws-password *** --aws-client-id 4h3b285v1a9746sqhukk5k3a7i

bash scripts/test-sns-all.sh --env production --write --gcp-auto-token
```

サマリー出力例 (production read-only, 2026-02-24):

```
  Cloud       PASS    FAIL    SKIP  Status
  ──────────  ──────  ──────  ────  ──────────────
  aws            9       0       4  ☒ PASS
  azure         17       0       2  ☒ PASS
  gcp           13       0       4  ☒ PASS
  ──────────  ──────  ──────  ────  ──────────────
  TOTAL         39       0      10
```

スクリプト	追加機能
`scripts/test-sns-aws.sh`	`-username`/`--password`/`--client-id` で Cognito 自動認証、X-Amz-Signature 検証、binary PUT to S3、imageUrl HTTP 200 確認
`scripts/test-sns-gcp.sh`	`-auto-token` で `gcloud auth print-identity-token` 自動認証、X-Goog-Signature 検証、binary PUT to GCS、imageUrl HTTP 200 確認
`scripts/test-sns-azure.sh`	`-sns-blob-type: BlockBlob` で binary PUT to Azure Blob (HTTP 201)、SAS read URL HTTP 200 確認

#	テスト	概要
5-2	署名URL検証	presigned URL に `X-Amz-Signature=` / `X-Goog-Signature=` / SAS token が含まれることを確認
5-3	binary PUT	1×1 PNG を実際に presigned URL へ PUT し HTTP 200/201 を確認
5-4	imageUrl アクセス確認	PUT したキーで POST /posts → GET /posts/:id → imageUrls[0] に GET → HTTP 200 を確認

マルチクラウド + GitHub の費用を一元管理するツールが `scripts/` 配下に実装済みです。

```
python3 scripts/cost_report.py                # デフォルト: 過去ヶ月3
python3 scripts/cost_report.py --months 6     # 過去ヶ月6
python3 scripts/cost_report.py --json         # JSON 出力
python3 scripts/cost_report.py --aws-only     # AWS のみ
python3 scripts/cost_report.py --azure-only   # Azure のみ
```

xbar を使った 1 時間ごと自動更新ウィジェット。

```
brew install --cask xbar
bash scripts/mac-widget/install.sh
open -e scripts/mac-widget/.env    # 認証情報を設定
```

Provider	方式
AWS	Cost Explorer は USD 固定 → open.er-api.com で リアルタイム USD/JPY 変換 (失敗時 ¥150 固定)
Azure	Cost Management API の `rows[n][2]` から通貨コードを直接取得 (JPY)
GCP	Billing API — サービスアカウント or `gcloud auth`
GitHub	Billing API 廃止 (HTTP 410) → `actions/cache/usage` + runs 件数で代替

`scripts/mac-widget/.env` (git 管理外) に認証情報を記載します。 テンプレート: `scripts/mac-widget/cost-monitor.env.sample`

変数	用途
`AZURE_SUBSCRIPTION_ID`	Azure Cost Management
`GCP_BILLING_ACCOUNT`	GCP Billing (`01XXXX-XXXXXX-XXXXXX` 形式)
`GCP_PROJECT_ID`	GCP プロジェクト ID
`GITHUB_TOKEN`	GitHub Actions 使用量取得
`GH_REPO`	`owner/repo` 形式 (個人リポジトリ用)

AWS は ~/.aws/credentials の default プロファイルを使用（追加設定不要）。

マルチクラウド + GitHub の費用を一元管理するツールが scripts/ 配下に実装済みです。

```
python3 scripts/cost_report.py                # デフォルト: 過去ヶ月3
python3 scripts/cost_report.py --months 6     # 過去ヶ月6
python3 scripts/cost_report.py --json         # JSON 出力
python3 scripts/cost_report.py --aws-only     # AWS のみ
python3 scripts/cost_report.py --azure-only   # Azure のみ
```

xbar を使った 1 時間ごと自動更新ウィジェット。

```
brew install --cask xbar
bash scripts/mac-widget/install.sh
open -e scripts/mac-widget/.env    # 認証情報を設定
```

プロバイダー	方式
AWS	Cost Explorer は USD 固定 → open.er-api.com で リアルタイム USD/JPY 変換 (失敗時 ¥150 固定)
Azure	Cost Management API の rows[n][2] から通貨コードを直接取得 (JPY)
GCP	Billing API — サービスアカウント or gcloud auth
GitHub	Billing API 廃止 (HTTP 410) → actions/cache/usage + runs 件数で代替

scripts/mac-widget/.env (git 管理外) に認証情報を記載します。 テンプレート: scripts/mac-widget/cost-monitor.env.sample

変数	用途
AZURE_SUBSCRIPTION_ID	Azure Cost Management
GCP_BILLING_ACCOUNT	GCP Billing (01XXXX-XXXXXX-XXXXXX 形式)
GCP_PROJECT_ID	GCP プロジェクト ID
GITHUB_TOKEN	GitHub Actions 使用量取得
GH_REPO	owner/repo 形式 (個人リポジトリ用)

AWS は ~/.aws/credentials の default プロファイルを使用（追加設定不要）。

- API Gateway
- Lambda
- CloudFront
- リソースグループ

- Function Apps
- Front Door
- Cloud Run
- Cloud Storage
- Firestore

GCP FinOps から の指摘に対応して実施した監査。プロジェクト `ashnova` のすべての静的 IP アドレスをレビューしました。

```
gcloud compute addresses list --project=ashnova \
  --format="table(name,address,status,addressType,users.list())"
```

名前	IP アドレス	ステータス	作成日	使用中
`multicloud-auto-deploy-production-cdn-ip`	34.8.88.22	☒ IN_USE	—	Production CDN (Forwarding Rule ×2)
`multicloud-auto-deploy-staging-cdn-ip`	34.111.17.182	☒ IN_USE	—	Staging CDN (Forwarding Rule ×2)
`ashnova-production-ip-c41311`	34.54.250.208	⚠ **RESERVED**	2026-02-11	なし
`multicloud-frontend-ip`	34.120.43.83	⚠ **RESERVED**	2026-02-14	なし
`simple-sns-frontend-ip`	34.149.225.123	⚠ **RESERVED**	2026-01-30	なし

名前	推定履歴
`simple-sns-frontend-ip`	プロジェクト初期（`simple-sns` という名前だった時期、2026-01-30）に作成。Pulumi コードや Forwarding Rule に参照なし。
`ashnova-production-ip-c41311`	Production CDN 用に作成（Pulumi サフィックス `c41311` から判断、2026-02-11）されたが、後に `multicloud-auto-deploy-production-cdn-ip` に置き換えられた。
`multicloud-frontend-ip`	2026-02-14 に作成。コードベースやドキュメントに参照なし。実験的に予約されて放置されたと推定。

注: これら3つはすべて Pulumi コードや Forwarding Rule とリンクされておらず、すぐに解放できます。

```
gcloud compute addresses delete ashnova-production-ip-c41311 --global --project=ashnova --quiet
gcloud compute addresses delete multicloud-frontend-ip          --global --project=ashnova
    --quiet
gcloud compute addresses delete simple-sns-frontend-ip          --global --project=ashnova
    --quiet
```

⚠ 削除は不可逆です。実行前に `gcloud compute addresses describe <name> --global` で各 IP に関連リソースがないことを確認してください。

静的 IP 監査のフォローアップとして実施。Terraform 時代のレガシーバケットと壊れた Cloud Function を特定しました。

バケット名	サイズ	判定	備考
`ashnova-multicloud-auto-deploy-production-frontend`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-production-function-source`	5 MB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-production-uploads`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-frontend`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-function-source`	5 MB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-landing`	8 KB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-uploads`	—	☒ Active	Pulumi 管理
`ashnova.firebasestorage.app`	—	☒ Keep	Firebase システム管理
`ashnova_cloudbuild`	—	☒ Keep	Cloud Build システム管理
`gcf-v2-sources-899621454670-asia-northeast1`	433 MB	☒ Keep	アクティブな Cloud Function v2 のソース
`gcf-v2-uploads-899621454670.asia-northeast1.cloudfunctions.appspot.com`	—	☒ Keep	Cloud Functions アップロードステージング
`ashnova-staging-frontend`	**empty**	☒ **削除**	Terraform レガシー。`ashnova-multicloud-auto-deploy-staging-frontend` に移行済み
`ashnova-staging-function-source`	**65 MB**	☒ **削除**	Terraform レガシー。2026-02-14 の古い zip を含む
`multicloud-auto-deploy-tfstate`	**empty**	☒ **削除**	古い Terraform state バケット。空。
`multicloud-auto-deploy-tfstate-gcp`	**6 KB**	☒ **削除**	上記2つのバケットの Terraform state のみ保持

バケット名	推定履歴
`ashnova-staging-frontend`	旧Terraform 設定のフロントエンドバケット（`ashnova-staging-*` 命名）。`ashnova-multicloud-auto-deploy-staging-frontend`（Pulumi 管理）に完全移行済み。空。
`ashnova-staging-function-source`	同上。Terraform 設定の Cloud Function ソースバケット。2026-02-14 の古い 65 MB の zip を含む。`ashnova-multicloud-auto-deploy-staging-function-source`（5 MB）に置き換え済み。
`multicloud-auto-deploy-tfstate`	AWS Terraform state バケット候補として作成されたが未使用。空。
`multicloud-auto-deploy-tfstate-gcp`	`ashnova-staging-*` 2つのバケットの Terraform state を保持。コードベースに `.tf` ファイルなし。4つをセットで削除。

リソース	状態	詳細
`mcad-staging-api` (Cloud Function v2)	**FAILED**	`Cloud Run service not found` エラー。Cloud Run サービスは削除されたが Function 定義が残っている。Pulumi/現行コードに参照なし。削除可能。

```
gcloud storage rm --recursive gs://ashnova-staging-frontend          --project=ashnova
gcloud storage rm --recursive gs://ashnova-staging-function-source    --project=ashnova
gcloud storage rm --recursive gs://multicloud-auto-deploy-tfstate     --project=ashnova
gcloud storage rm --recursive gs://multicloud-auto-deploy-tfstate-gcp --project=ashnova

gcloud functions delete mcad-staging-api \
  --region=asia-northeast1 --project=ashnova --v2 --quiet
```

⚠ `multicloud-auto-deploy-tfstate-gcp` は `ashnova-staging-frontend` と `ashnova-staging-function-source` の Terraform state を含みます。4つのバケットをセットで削除してください。

→ 07 — ランブック

Chapter 9

ソース実装解説書

本ドキュメントは `docs/AI_AGENT_0*.md` の設計書を前提に、実際のソースコードがどのように実装されてい

---

1. プロジェクト全体像
2. バックエンド API（FastAPI）
3. 認証・JWT 検証
4. クラウド別バックエンド実装
5. クラウド別エントリーポイント
6. フロントエンド（React SPA）
7. ローカル開発環境

---

multicloud-auto-deploy は、SNS スタイルの投稿アプリ（文章・画像・タグ）を AWS / Azure / GCP の **3つのクラウドに同時デプロイ** するプラットフォームである。

```
ユーザー

  │
  ├── [AWS]   CloudFront → S3 (React SPA)
  │          API Gateway v2 → Lambda (FastAPI) → DynamoDB
  │
  ├── [Azure] Front Door → Blob Storage (React SPA)
  │          Azure Functions → Cosmos DB
  │
  └── [GCP]   Cloud CDN → GCS (React SPA)
             Cloud Run → Firestore
```

```
services/
  api/                      ← FastAPI バックエンド（クラウド共通）3
    app/
      main.py               ← FastAPI アプリ本体・・ルーター登録CORS
      config.py             ← 環境変数を Pydantic Settings で管理
      models.py             ← Pydantic データモデル
      auth.py               ← JWT 認証ミドルウェア
```

```
      jwt_verifier.py          ← Cognito / Azure AD / Firebase JWT 検証
      backends/
        base.py                ← 抽象基底クラス（インターフェース定義）
        aws_backend.py         ← DynamoDB + S3 実装
        azure_backend.py       ← Cosmos DB + Blob Storage 実装
        gcp_backend.py         ← Firestore + Cloud Storage 実装
        local_backend.py       ← DynamoDB Local + MinIO 実装
      routes/
        posts.py               ← /posts エンドポイント
        profile.py             ← /profile エンドポイント
        uploads.py             ← /uploads エンドポイント
    index.py                   ← AWS Lambda ハンドラー () Mangum
    function_app.py            ← Azure Functions ハンドラー
    function.py                ← GCP Cloud Run ハンドラー
  frontend_react/              ← React 18 + Vite + TypeScript SPA
    src/
      App.tsx                  ← ルーティング定義
      api/                     ← Axios API クライアント
      contexts/AuthContext.←tsx 認証状態管理
      components/              ← UI コンポーネント
      pages/                   ← ページコンポーネント
      config/auth.ts           ← 認証プロバイダー設定
```

次セクション → 2. バックエンド API（FastAPI）

`services/api/app/main.py` が FastAPI アプリの中心。

主な処理の流れ:

```python
try:
    from aws_lambda_powertools import Logger, Metrics, Tracer
    logger = Logger(service="simple-sns-api")
    ...
except ImportError:
    logger = logging.getLogger(__name__)

app = FastAPI(title="Simple SNS API", version="3.0.0", ...)

app.add_middleware(CORSMiddleware, allow_origins=origins, ...)

app.add_middleware(GZipMiddleware, minimum_size=1000)

app.include_router(posts.router)    # /posts
app.include_router(uploads.router)  # /uploads
app.include_router(profile.router)  # /profile
app.include_router(limits.router)   # /limits
```

ライフサイクル管理: `@asynccontextmanager` の `lifespan` 関数でアプリ起動・終了時のログを出力する。

後方互換エイリアス: 旧フロントエンド向けに /api/messages/ エンドポイントを維持している。このエンドポイ anonymous ユーザーとして扱われる（テスト互換のための意図的な設計）。

---

pydantic_settings.BaseSettings を継承した Settings クラスで全ての環境変数を管理する。

```
class Settings(BaseSettings):
    cloud_provider: CloudProvider = CloudProvider.LOCAL  # "aws"|"azure"|"gcp"|"local"
    auth_disabled: bool = False
    posts_table_name: Optional[str] = None   # 用AWS
    cosmos_db_endpoint: Optional[str] = ...  # 用Azure
    gcp_project_id: Optional[str] = None     # 用GCP
    ...
```

ポイント: - 環境変数名の揺れ（例: COGNITO_USER_POOL_ID vs AWS_COGNITO_USER_POOL_ID）を AliasChoices で吸収している。 - CLOUD_PROVIDER が未設定の場合は、実行環境の特徴から自動検出される（config.py 末尾の auto-detect ロジック）。

クラウド自動検出ロジック:

環境変数	判定クラウド
AWS_EXECUTION_ENV	aws
WEBSITE_INSTANCE_ID	azure
K_SERVICE	gcp
（なし）	local

Pydantic v2 で定義。**camelCase と snake_case 両方を返す**ことが最大の特徴。

```
class Post(BaseModel):
    id: str = Field(..., alias="postId")
    user_id: str = Field(..., alias="userId")
    content: str
    ...

    @model_serializer
    def serialize_model(self) -> dict:
        return {
            # 新フロントエンド用 (camelCase)
            "postId": self.id,
            "userId": self.user_id,
            ...
            # 旧フロントエンド用 (snake_case)
            "id": self.id,
            "author": self.user_id,
            "created_at": self.created_at,
        }
```

`model_config = {"populate_by_name": True}` により、リクエスト受信時は `postId`（alias）でも `id`（フィールド名）のどちらでも受け付ける。

`ListPostsResponse` も同様に `items` / `results` / `messages` の3つのキーで同じデータを返し、複数バージョンのフロン

ファイル	prefix	主な処理
`posts.py`	`/posts`	投稿の CRUD（GET/POST/PUT/DELETE）
`profile.py`	`/profile`	プロフィール取得・更新
`uploads.py`	`/uploads`	画像アップロード用署名付き URL 発行

認証の使い分け:

```
@router.get("")
def list_posts(...) -> ListPostsResponse:
    ...

@router.post("", status_code=201)
def create_post(body: CreatePostBody, user: UserInfo = Depends(require_user)):
    ...
```

`Depends(require_user)` は `auth.py` の関数で、トークンがない場合 HTTP 401 を返す。`Depends(get_current_user)` はトークンなしの場合 `None` を返す（認証任意）。

```
クライアント

  Authorization: Bearer <JWT>
          ↓
  get_current_user() / require_user()
          ↓
  get_jwt_verifier()  ← settings.auth_provider で切替
          ↓
  JWTVerifier.verify_token()
          ↓
  UserInfo { user_id, email, groups }
```

`auth_disabled=true` の場合（ローカル開発時のみ許容）は検証をスキップし、テスト用ユーザーを返す。

UserInfo クラス:

```
@dataclass
class UserInfo:
    user_id: str
    email: Optional[str] = None
    groups: Optional[list[str]] = None

    @property
```

```
    def is_admin(self) -> bool:
        return self.groups is not None and "Admins" in self.groups
```

管理者判定は groups リストに "Admins" が含まれるかで行う。

JWTVerifier クラスが3クラウドの JWT 検証を統一的に処理する。

JWKS エンドポイント:

プロバイダー	JWKS URI
Cognito (AWS)	https://cognito-idp.{region}.amazonaws.com/{pool_id}/.well-known/jwks.json
Firebase (GCP)	https://www.googleapis.com/service_accounts/v1/jwk/securetoken@...
Azure AD	https://login.microsoftonline.com/{tenant_id}/discovery/v2.0/keys

JWKS キャッシュ: 毎リクエストごとに HTTP 取得すると遅延が大きいため、1時間のインメモリキャッシュ

```
def get_jwks(self) -> dict:
    if キャッシュ有効:
        return self._jwks_cache
    # 取得失敗時はキャッシュを使い続ける
    if self._jwks_cache:
        logger.warning("Using expired JWKS cache")
        return self._jwks_cache
    raise
```

トークン検証処理: python-jose ライブラリを使い、JWKS から公開鍵を取得して署名検証 + issuer / audience 検証を行う。検証後 extract_user_info() でクラウドごとの claims 差異を吸収して UserInfo に変換する。

```
@lru_cache(maxsize=1)
def get_backend() -> BackendBase:
    provider = settings.cloud_provider  # 環境変数 CLOUD_PROVIDER
    if provider == CloudProvider.AWS:
        return AwsBackend()
    elif provider == CloudProvider.AZURE:
        return AzureBackend()
    elif provider == CloudProvider.GCP:
        return GcpBackend()
    else:
        return LocalBackend()
```

@lru_cache(maxsize=1) により、同一プロセス内ではバックエンドインスタンスを1回だけ生成する（接続の使い

全バックエンドが実装すべきインターフェースを定義する。

```python
class BackendBase(ABC):
    @abstractmethod
    def list_posts(limit, next_token, tag) -> (list[Post], next_token)
    @abstractmethod
    def get_post(post_id) -> Optional[Post]
    @abstractmethod
    def create_post(body, user) -> dict
    @abstractmethod
    def update_post(post_id, body, user) -> dict   # ※ AWS / Local のみ完全実装
    @abstractmethod
    def delete_post(post_id, user) -> dict
    @abstractmethod
    def get_profile(user_id) -> ProfileResponse
    @abstractmethod
    def update_profile(user, body) -> ProfileResponse
    @abstractmethod
    def generate_upload_urls(count, user, content_types) -> list[dict]
```

データストア: DynamoDB Single Table Design + S3

テーブル設計:

```
PK        SK                          内容
POSTS     <ISO timestamp>#<uuid>      投稿データ
PROFILES  <userId>                    ユーザープロフィール
```

SK に `<timestamp>#<uuid>` を使うことで、`ScanIndexForward=False` クエリが標準の降順（新着順）になる。

投稿一覧取得:

```python
def list_posts(self, limit, next_token, tag):
    query_kwargs = {
        "KeyConditionExpression": "PK = :pk",
        "ExpressionAttributeValues": {":pk": "POSTS"},
        "ScanIndexForward": False,  # 新しい順
        "Limit": limit,
    }
    if next_token:
        query_kwargs["ExclusiveStartKey"] = {"PK": "POSTS", "SK": next_token}
    if tag:
        query_kwargs["FilterExpression"] = "contains(tags, :tag)"
    response = self.table.query(**query_kwargs)
    # next_token = LastEvaluatedKey["SK"]
```

ページネーションには `LastEvaluatedKey["SK"]`（= 最後のアイテムの SK）をトークンとして使う。

画像 URL の解決:

S3 に保存された画像キーは、取得時に `generate_presigned_url`（1時間有効）に変換する。- `https://` で始まる場合はそのまま返す - `http://` はセキュリティ上 Mixed Content となるためスキップする - それ以外（S3 キー）は署名付き URL に変換する

データストア: Cosmos DB（posts / profiles コンテナ） + Blob Storage

DynamoDB との違いは、Cosmos DB では**コンテナ(テーブル)を用途ごとに分ける**点。posts コンテナと profiles コンテナを別々に持つ。

```
self.posts_container = self.database.create_container_if_not_exists(
    id="posts",
    partition_key=PartitionKey(path="/postId"),
)
self.profiles_container = self.database.create_container_if_not_exists(
    id="profiles",
    partition_key=PartitionKey(path="/userId"),
)
```

画像 URL: Blob SAS（24時間有効）に変換。フル Blob URL が既に保存されている場合はキー部分を抽出して SAS を改めて付与する。

インポート安全設計: `azure-cosmos` や `azure-storage-blob` がインストールされていない環境でも `ImportError` をキャッチして警告ログだけ出す。実際に `AzureBackend()` を生成しようとした時に例外を投げる。

データストア: Firestore（posts / profiles コレクション） + Cloud Storage

Firestore ではドキュメント ID でページネーションを行う。

```
def list_posts(self, limit, next_token, tag):
    query = col.order_by("createdAt", direction=DESCENDING).limit(limit + 1)
    if next_token:
        cursor_doc = col.document(next_token).get()
        query = query.start_after(cursor_doc)  # カーソルベースのページネーション
```

`limit + 1` 件取得して `len(docs) > limit` なら次ページがあると判断し、最後のドキュメント ID を `next_token` として返す。

署名付き URL の特殊対応: GCP では Compute Engine のデフォルト認証情報には秘密鍵がないため、`generate_signed_url()` は使えない。代わりに IAM `signBlob` API を使うカスタム実装を行う。認証情報は `google.auth.default()` で取得し、`credentials.valid` を確認してから期限切れ時のみ refresh する（毎リクエストのメタデータサーバー呼び出しを回避）。

データストア: DynamoDB Local + MinIO（S3 互換）

AWS バックエンドと同じ Single Table Design を使うことで、ローカルで発見したバグが AWS でも同様に再現することを保証している。テーブルが存在しない場合は**初回起動時に自動作成**する。

```python
def _init_dynamodb(self):
    # テーブルが存在しなければ作成
    try:
        self.table = self.dynamodb.create_table(
            TableName=table_name,
            KeySchema=[{"AttributeName": "PK", ...}, {"AttributeName": "SK", ...}],
            ...
        )
    except self.dynamodb.meta.client.exceptions.ResourceInUseException:
        self.table = self.dynamodb.Table(table_name)  # 既存テーブルを使う
```

FastAPI アプリ本体（`app/main.py`）は3クラウドで**共通**。クラウドごとにその前段となるアダプター層が異な

```python
from mangum import Mangum
from app.main import app

handler = Mangum(app, lifespan="off")
```

`Mangum` が Lambda イベント（API Gateway Proxy v2 形式）を ASGI に変換し、FastAPI へ渡す。`lifespan="off"` は Lambda の短命実行モデルに対応するための設定。

Lambda Layer に FastAPI 等の依存ライブラリを格納し、アプリコード（〜78KB）と別管理することでデプ

Azure Functions には Mangum 相当の標準アダプターがないため、**カスタム ASGI ブリッジ**を自前実装している。

主な処理フロー:

```
Azure Functions HttpRequest
    ↓
route_params["route"] からパスを取得
    "HttpTrigger/" プレフィックスを除去
    ↓
ASGI scope dict を組み立て
    {type: "http", path: "/posts", method: "GET", ...}
    ↓
FastAPI ASGI app を呼び出し
    ↓レスポンスを
 func.HttpResponse に変換して返す
```

CORS Preflight（OPTIONS） は Azure Functions レベルで直接処理し、FastAPI に渡さずに 204 を返す（速度・安定性のため）。

インポートエラーが発生しても 503 でエラー内容を返すフェイルセーフ設計：

```python
_IMPORT_ERROR = None
try:
    from app.main import app as fastapi_app
```

```
except Exception as e:
    _IMPORT_ERROR = traceback.format_exc()
...
if fastapi_app is None:
    return func.HttpResponse(
        body=json.dumps({"error": "Service unavailable", "detail": _IMPORT_ERROR}),
        status_code=503,
    )
```

Cloud Run は通常の HTTP サーバーとして動作するため、`uvicorn` でそのまま FastAPI を起動できる。Lambda や Azure Functions のような特殊なアダプターは必要ない。

`Dockerfile` で `CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]` のような起動コマンドを指定するだけで Cloud Run 上で動作する。

`services/frontend_react/` に React 18 + Vite + TypeScript + Tailwind CSS で構築された SPA。3クラウドすべてで**同一のビルド成果物**をオブジェクトストレージ（S3 / Blob Storage / GCS）に配置し、CDN で配信する。

`vite.config.ts` でビルド時のベースパスを `/sns/` に設定している（CDN の `/sns/*` パスにマッピングされるため）。

```
<AuthProvider>
  <Layout>
    <Routes>
      <Route path="/"              element={<HomePage />} />
      <Route path="/post/:postId"  element={<PostPage />} />
      <Route path="/profile"       element={<ProfilePage />} />
      <Route path="/login"         element={<LoginPage />} />
      <Route path="/auth/callback" element={<CallbackPage />} />
      <Route path="*"              element={<HomePage />} />
    </Routes>
  </Layout>
</AuthProvider>
```

`AuthProvider` が認証状態をアプリ全体に提供する（React Context パターン）。

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? '',  // 空文字 → Vite プロキシ経由でローカル API
  timeout: 15000,
});

// 全リクエストに JWT トークンを付与
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('id_token') || localStorage.getItem('access_token');
```

```
  if (token) config.headers['Authorization'] = `Bearer ${token}`;
  return config;
});
```

Cognito は id_token（audience が client_id と一致する）を優先して使う。access_token はフォールバック。

ビルド時の環境変数 VITE_AUTH_PROVIDER で認証プロバイダーを切り替える。

VITE_AUTH_PROVIDER	認証方式
aws	Cognito PKCE 認証コードフロー
azure	Azure AD 暗黙的フロー（id_token フラグメント）
firebase	Firebase SDK Google Sign-In（ポップアップ）
none	認証なし（ローカル開発）

AWS (Cognito) の PKCE フロー: 1. generateCodeVerifier() でランダム文字列生成 → sessionStorage に保存 2. SHA-256 ハッシュ → base64url エンコード → code_challenge 3. Cognito 認証エンドポイントへリダイレクト 4. コールバック（/auth/callback）で code を受取り + code_verifier で検証 → token 取得

---

localStorage にトークンを保存・読込し、window の storage イベントを監視することでタブ間でログイン状態を

```
// タブ間同期
useEffect(() => {
  const onStorage = () => setToken(readToken());
  window.addEventListener("storage", onStorage);
  return () => window.removeEventListener("storage", onStorage);
}, []);
```

コンポーネント	役割
HomePage	投稿一覧表示・タグフィルタ・キーワード検索・ページネーション（“もっと見る”）
PostForm	投稿作成フォーム（テキスト・マークダウン・画像複数枚・タグ）
PostCard	1件の投稿表示（マークダウン対応・画像ライトボックス）
ProfilePage	ニックネーム・自己紹介の表示と編集
LoginPage	各クラウドの認証フローへのリダイレクト
CallbackPage	Cognito / Azure AD の認証コールバック処理・token 保存

```
1. POST /uploads/presigned-urls  →  { urls: [{uploadUrl, key}] }
2. 各 uploadUrl に （ブラウザから直接PUT S3/Blob/GCS へアップロード）
3. POST /posts { imageKeys: [key1, key2, ...] }  →  投稿作成
```

バックエンドを経由しないため、大容量画像でもバックエンドに負荷をかけない設計。アップロード上限は `GET /limits` で取得し、ハードコードしない。

`docker-compose.yml` で以下の4サービスを起動する。

サービス	ポート	用途
`api`	8000	FastAPI（ホットリロード付き uvicorn）
`frontend_web`	8080	Python SSR フロントエンド（レガシー）
`dynamodb-local`	8001	DynamoDB Local（AWS 互換）
`minio`	9000/9001	MinIO（S3 互換オブジェクトストレージ）

API サービスの環境変数（ローカル）：

```
CLOUD_PROVIDER=local
AUTH_DISABLED=true      # ローカルのみ許容
DYNAMODB_ENDPOINT=http://dynamodb-local:8000
DYNAMODB_TABLE_NAME=simple-sns-local
MINIO_ENDPOINT=http://minio:9000
MINIO_PUBLIC_ENDPOINT=http://localhost:9000   # ブラウザからの直接アクセス用
```

`MINIO_ENDPOINT`（コンテナ内部 URL）と `MINIO_PUBLIC_ENDPOINT`（ブラウザからの URL）を分けることで、コンテナ内→MinIO の通信とブラウザ→MinIO の通信が両立する。

```
cd /workspaces/multicloud-auto-deploy
docker compose up -d

curl http://localhost:8000/health

open http://localhost:8000/docs

open http://localhost:9001  # minioadmin / minioadmin
```

`services/api/app` ディレクトリが `docker-compose.yml` でコンテナにマウントされるため、ソース変更は即座にホッ

`LocalBackend._init_dynamodb()` が初回起動時に DynamoDB Local へテーブルを自動作成する。手動でのテーブル作成は不要。

MinIO のバケットも同様に `LocalBackend._init_storage()` 内で自動作成する。

パターン	実装箇所	説明
Strategy パターン	`backends/`	`BackendBase` を実装するクラスをクラウドごとに差し替え
Factory + LRU Cache	`backends/__init__.py`	`get_backend()` で1回だけインスタンスを生成
Pydantic Settings	`config.py`	環境変数を型安全に管理、`AliasChoices` で名前揺れを吸収
Dual serialization	`models.py`	camelCase と snake_case を同時に返してフロントエンド互換
JWKS キャッシュ	`jwt_verifier.py`	1時間キャッシュ、失敗時は期限切れキャッシュをフォールバック
Presigned URL	各バックエンド	画像をバックエンド経由せずブラウザから直接アップロード
PKCE フロー	`config/auth.ts`	AWS Cognito で認証コードフローを安全に実装

Chapter 10

インフラ・CI/CD解説書

本ドキュメントは services/api/・services/frontend_react/ のアプリコードを解説した IMPLEMENTATION_GUIDE.md の**姉妹ドキュメント**である。 ここでは **インフラ (Pulumi)・CI/CD (GitHub Actions)・スクリプト** のソースを扱う。

---

1. インフラ全体の考え方
2. Pulumi AWS (infrastructure/pulumi/aws/__main__.py)
3. Pulumi Azure (infrastructure/pulumi/azure/__main__.py)
4. Pulumi GCP (infrastructure/pulumi/gcp/__main__.py)
5. CI/CD ワークフロー (GitHub Actions)
6. フロントエンド SPA デプロイワークフロー
7. スクリプト群 (scripts/)

Terraform ではなく **Pulumi** を使う理由は、Python コードで条件分岐・ループ・関数を自由に使えるため
Pulumi.<stack>.yaml に記述し、デプロイ状態は **Pulumi Cloud**（リモートステート）で管理する。

スタック名	ブランチ	用途
staging	develop	自動デプロイ・動作確認用
production	main	本番環境（即時反映）

各スタックは独立したクラウドリソースを持つ（S3　バケット名・Lambda　関数名などに {stack} サフィックスが付く）。

```
project_name = "multicloud-auto-deploy"
stack = pulumi.get_stack()   # "staging" または "production"

f"{project_name}-{stack}-api"
```

全リソースに common_tags（Project / ManagedBy / Environment）を付与してコスト管理・検索を容易にする

ファイル: infrastructure/pulumi/aws/**main**.py（約965行）

```
IAM Role (Lambda 用)
   └── AWSLambdaBasicExecutionRole ポリシーアタッチ
   └── DynamoDB + S3 アクセス用インラインポリシー

Cognito User Pool
   └── User Pool Client (PKCE コードフロー)
   └── User Pool Domain 認証ホスト( UI)

DynamoDB Table (Single Table Design)
   ├── PK="PK" / SK="SK"
   ├── GSI: PostIdIndex (hash=postId)
   └── GSI: UserPostsIndex (hash=userId, range=createdAt)

S3 Bucket: （画像アップロード先）images
   └── CORS 設定（ブラウザから直接 PUT できるよう）
   └── （PublicAccessBlockpublic 禁止）

Lambda （LayerFastAPI / Mangum / Pydantic 等）
   └── services/api/lambda-layer.zip から生成

Lambda （FunctionFastAPI + ) Mangum
   └── ENV: POSTS_TABLE_NAME, IMAGES_BUCKET_NAME, COGNITO_* 等

API Gateway v2 (HTTP API)
   └── $default ルート → Lambda 統合

S3 Bucket: （frontendReact SPA 配信）
   └── OAC (Origin Access Control) で CloudFront のみ許可

CloudFront Distribution
   ├── /sns/* → S3 frontend / CloudFront Function で /sns → /sns/index.html リライト
   └── /*     → S3 frontend (landing pages)
```

```python
user_pool_client = aws.cognito.UserPoolClient(
    allowed_oauth_flows=["code"],         # PKCE コードフロー
    allowed_oauth_scopes=["openid", "email", "profile"],
    callback_urls=(
        ([f"https://{cf_domain}/sns/auth/callback"] if cf_domain else [])
        + ([f"https://{custom_domain}/sns/auth/callback"] if custom_domain else [])
    ),
    access_token_validity=1,   # 時間1
    id_token_validity=1,       # 時間1
    refresh_token_validity=30, # 日30
)
```

`cf_domain`（CloudFront ドメイン）と `custom_domain`（カスタムドメイン）の両方を登録することで、どちらの CI とローカルでは ZIP のパスが異なるため、`GITHUB_WORKSPACE` 環境変数で分岐している。

```
workspace_root = os.environ.get("GITHUB_WORKSPACE")
if workspace_root:
    layer_zip_path = pathlib.Path(workspace_root) / "services/api/lambda-layer.zip"
else:
    # __file__ から階層上がリポジトリルート3
    layer_zip_path = pathlib.Path(__file__).parent.parent.parent /
    "services/api/lambda-layer.zip"
```

ZIP が存在しない場合はウォーニングを出してスキップする（初回デプロイ時など）。

React SPA は HTML5 History API でルーティングするため、CDN 側で /sns → /sns/index.html のリライトが必要。CloudFront Function を Pulumi のコード内にインラインで定義している。

```
spa_rewrite_function = aws.cloudfront.Function(
    "spa-sns-rewrite",
    name=f"spa-sns-rewrite-{stack}",
    runtime="cloudfront-js-2.0",
    code="""
function handler(event) {
    var uri = event.request.uri;
    if (uri === '/sns' || uri === '/sns/') {
        event.request.uri = '/sns/index.html';
    } else if (uri.startsWith('/sns/') && !uri.includes('.')) {
        event.request.uri = '/sns/index.html';
    }
    return event.request;
}
""",
)
```

ファイル: infrastructure/pulumi/azure/**main**.py（約612行）

```
Resource Group (japaneast)
   ├── Storage Account: mcadfunc{suffix}  ← Azure Functions 用
   ├── Storage Account: mcadweb{suffix}   ← フロントエンド静的サイト用
   |     └── $web コンテナ（静的 Web サイトホスティング）
   ├── Application （監視）Insights
   ├── Cosmos DB アカウント
   |     └── コンテナ: posts / profiles
   ├── Azure AD アプリ登録（認証）
   |     └── リダイレクト URI: https://{frontend_domain}/sns/auth/callback
   ├── Azure （FunctionsFlex ) Consumption
   |     └── HTTP Trigger: {*route} → FastAPI
   └── Azure Front （) DoorCDN
         ├── /sns/* → Blob Storage $web/sns/ (React SPA)
         └── /*     → Blob Storage $web (landing pages)
```

Azure ストレージアカウント名はグローバル一意が必要なため pulumi_random で6文字サフィックスを生成する。同じインスタンスを2つのアカウントで共有することで、識別が容易にな

```
storage_suffix = random.RandomString("storage-suffix", length=6, special=False, upper=False)

functions_storage_name = storage_suffix.result.apply(lambda s: f"mcadfunc{s}")
frontend_storage_name  = storage_suffix.result.apply(lambda s: f"mcadweb{s}")
```

AWS Cognito が PKCE コードフローを使うのに対し、Azure AD は**暗黙的フロー**（id_token フラグメント）を使う。

```
app_registration = azuread.Application(
    "app-registration",
    web=azuread.ApplicationWebArgs(
        redirect_uris=[f"https://{frontend_domain}/sns/auth/callback"],
        implicit_grant=azuread.ApplicationWebImplicitGrantArgs(
            id_token_issuance_enabled=True,   # 暗黙的フロー
        ),
    ),
)
```

フロントエンドのコールバックハンドラーは URL フラグメント（#id_token=...）を解析してトークンを取得す

AFD の SpaRuleSet URL リライトルールで /sns/* の非アセットパスを /sns/index.html にリライトし、React SPA のクライアントルーティングを実現する。

ファイル: infrastructure/pulumi/gcp/**main**.py（約585行）

```
GCP API 有効化（Cloud Run / Secret Manager / Firebase 等）
Firebase プロジェクト・Web アプリ登録
Cloud Audit （全サービス対象）Logs

GCS バケット: frontend   ← React SPA + landing pages
GCS バケット: uploads    ← 画像アップロード

Secret （環境変数を秘密として管理）Manager

Cloud Run: api           ← FastAPI (uvicorn)
  └── サービスアカウント（Firestore / GCS / Secret Manager 権限）

Cloud （CDNClassic External ）LB
  └── /* → GCS バケット（Backend ）Bucket
```

```
services = ["run.googleapis.com", "secretmanager.googleapis.com", "firebase.googleapis.com",
    ...]
enabled_services = [gcp.projects.Service(f"enable-{s}", ...) for s in services]

firebase_project = gcp.firebase.Project(
    "firebase-project",
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
```

```
)
```

PulumiはCloud Runの**サービス設定**（メモリ・環境変数・サービスアカウント）を管理するが、**コンテナ**
CI/CDワークフロー（`gcloud run deploy`）が担う。Pulumiデプロイ時にコンテナイメージを要求するという制

Firebaseの認証プロバイダー設定（Googleサインイン）はOAuth同意画面の設定が必要で
Pulumiから自動化できない。コードにコメントで明示している。

ファイル	役割
`deploy-aws.yml`	AWS全リソース（Pulumi + Lambda + フロントエンド含む）
`deploy-azure.yml`	Azure全リソース（Pulumi + Functions + フロントエンド含む）
`deploy-gcp.yml`	GCP全リソース（Pulumi + Cloud Run + フロントエンド含む）
`deploy-frontend-web-aws.yml`	React SPAをS3/CloudFront にのみ再デプロイ（差分更新）
`deploy-frontend-web-azure.yml`	React SPAをBlob Storage/AFDにのみ再デプロイ
`deploy-frontend-web-gcp.yml`	React SPAをGCS/CDNにのみ再デプロイ
`deploy-landing-*.yml`	ランディングページ（静的HTML）の各クラウドへの配信
`version-bump.yml`	`versions.json`のバージョン自動更新

3つのワークフローすべてで同じルールを採用している。

```
environment: >-
  ${{ github.event_name == 'workflow_dispatch'
        && github.event.inputs.environment
        || (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
```

Pulumi は同一スタックへの並行デプロイで `ResourceConflictException` / `409 Conflict` が発生するため、`concurrency`で直列化している。

```
concurrency:
  group: deploy-aws-${{ ... }}
  cancel-in-progress:
    false # AWS/GCP: 実行中のジョブはキャンセルしない
    # Azure staging: （コスト削減のため古いtrue Run をキャンセル）
```

```
1. AWS 認証（aws-actions/configure-aws-）credentials
2. Pulumi CLI インストール
3. スタック名の決定（→developstaging / →）mainproduction
```

```
4. .github/config/aws.{stack}.env から設定を読込（single source of ）truth
5. Pulumi スタック選択 / 初期化
6. Pulumi （インフラ変更適用）up
7. Pulumi outputs 取得（Lambda 名・バケット名・Cognito 設定等）
8. Lambda Layer ZIP ビルド（scripts/build-lambda-layer.）sh
9. Lambda 関数コードの ZIP 化・アップロード（aws lambda update-function-）code
10. CloudFront エイリアス + ACM 証明書の確認・修正（safety ）net
11. Cognito コールバック URL の確認・修正（safety ）net
12. React SPA ビルド（npm run ）build
13. S3 同期（aws s3 ）sync
14. CloudFront キャッシュ無効化（aws cloudfront create-）invalidation
```

「Safety net」ステップの目的: Pulumiがカスタムドメイン設定を誤って消してしまうことがあるため、デ AWS CLI で直接確認・修正するステップを追加している。Pulumiの宣言的な制御と、より強制力のある命令的な確認の二重構造になっている。

設定ファイルの一元管理: `Pulumi.*.yaml` はgitignoreされておりリモートで使えないため、代わりに `.github/config/aws.staging.env` 等のコミット済みファイルに設定値を記述する。CIはこのファイルを `source` して環境変数を読み込む。

```
1. （PulumiAzure インフラ）
2. Azure Functions 用パッケージビルド
   - linux/amd64 Docker コンテナで pip （installARM ホストとのアーキテクチャ差異を回避）
   - ZIP 化して Azure CLI でアップロード（az functionapp deployment source config-）zip
3. az storage blob service-properties update で静的 Web サイトホスティング有効化
4. React SPA ビルド・Blob Storage アップロード（az storage blob upload-）batch
5. AFD キャッシュパージ（az afd endpoint ）purge
```

ARM → AMD64 クロスビルド: GitHub Actions のランナーは ARM (aarch64) のホストで動作する場合があるため、Lambda / Functions 用の Python パッケージは `--platform linux/amd64` のDockerコンテナ内でインストールする。

```
docker run --rm --platform linux/amd64 \
  -v "$(pwd)/package:/package" \
  python:3.12-slim \
  bash -c "pip install -r /package/requirements.txt -t /package"
```

```
1. google-github-actions/auth で認証（GCP_CREDENTIALS シークレット）
2. （PulumiGCP インフラ）
3. Docker イメージビルド・Artifact Registry プッシュ
4. Cloud Run サービス更新（gcloud run ）deploy
5. React SPA ビルド・GCS アップロード（gsutil ）rsync
6. CDN キャッシュ無効化（gcloud compute url-maps invalidate-cdn-）cache
```

`deploy-frontend-web-*.yml` はAPIやインフラを触らずに**フロントエンドだけを更新**したい場合に使う差分更新用 SPAのファイルを変更した際にPulumiを再実行せずに済むため、デプロイ時間を短縮できる。

```
1. Pulumi outputs 取得（バケット名・Cognito 設定・CloudFront ）ID
   ← インフラには触れず読み取りのみ
2. npm ci / npm run build
   - VITE_API_URL / VITE_AUTH_PROVIDER=aws / VITE_COGNITO_* を環境変数として注入
3. S3 同期（差分更新）
   - アセット類（JS/）CSS: cache-control max-age=31536000, immutable
   - index.html のみ: max-age（ブラウザキャッシュを短くして更新を即反映）=300
4. Cognito コールバック URL を aws cognito-idp update-user-pool-client で同期
   - CloudFront ドメイン + カスタムドメインの両方を登録
5. CloudFront キャッシュ無効化（/sns）/*
```

キャッシュ戦略の意図: ハッシュ付きアセット（`index-abc123.js` 等）は Vite がファイル名を変えるため長期キャッシュ可。一方 `index.html` は常に最新のアセット名を参照するため短期キャッシュにする。

```
1. Pulumi outputs 取得（ストレージアカウント名・AFD エンドポイント）
2. npm run （buildVITE_AUTH_PROVIDER=azure 等を注入）
3. az storage blob upload-batch で $web/sns/ へアップロード
4. az afd endpoint purge でキャッシュパージ
```

```
1. Pulumi outputs 取得（GCS バケット名）
2. npm run （buildVITE_AUTH_PROVIDER=firebase 等を注入）
3. gsutil -m rsync -r dist/ gs://{bucket}/sns/
4. gcloud compute url-maps invalidate-cdn-cache
```

Lambda Layer を `linux/amd64` Docker コンテナでビルドして ZIP 化するスクリプト。デプロイホストが ARM (Apple M1/M2 / aarch64) でも正しいアーキテクチャのバイナリを生成できる。

```
1. requirements-layer.txt を参照
2. docker run --platform linux/amd64 python:3.12-slim で pip install
3. python/ 以下に配置して ZIP 化 → services/api/lambda-layer.zip
```

Layer には `fastapi`, `mangum`, `pydantic`, `python-jose` 等が含まれる。 `boto3` / `botocore` は Lambda ランタイムに同梱されているため**除外**し、Layer サイズを削減している。

各クラウドに手動デプロイするためのラッパースクリプト。主に開発時の動作確認に使う。CI/CD ワークフロー（`.github/workflows/*.yml`）と同等の処理を行うが、インタラクティブな確認ステップを含む。

スクリプト	内容
`test-api.sh`	各クラウドの API エンドポイントに curl でリクエストし、ステータスコードを確認
`test-e2e.sh`	全クラウドを対象とした E2E テスト（投稿作成→取得→削除）
`test-sns-aws.sh` 等	クラウド個別の SNS 機能テスト
`test-auth-crud.sh`	認証付き CRUD 操作のテスト
`diagnose-environments.sh`	各クラウドの設定値・エンドポイントの診断

`versions.json` のバージョン番号をインクリメントするスクリプト。`version-bump.yml` ワークフローから呼び出される。セマンティックバージョニング（major/minor/patch）に対応。

原則	実装箇所	説明
IaC 一元管理	`infrastructure/pulumi/`	クラウドリソースはすべて Pulumi Python コードで定義
設定の単一ソース	`.github/config/*.env`	`Pulumi.*.yaml` は gitignore されるため、CI 用設定ファイルをリポジトリにコミット
Safety net	`deploy-aws.yml`	Pulumi だけに頼らず AWS CLI で CloudFront / Cognito を事後確認・修正
ARM/AMD64 分離	CI ワークフロー	Python パッケージは常に `linux/amd64` Docker でビルド
キャッシュ戦略	`deploy-frontend-web-*.yml`	ハッシュ付きアセット: 永続キャッシュ / `index.html`: 短期キャッシュ
差分デプロイ	`deploy-frontend-web-*.yml`	フロントエンド変更時は Pulumi を再実行しない
同時実行防止	`concurrency:` ブロック	Pulumi の競合エラーを防ぐためスタック単位で直列化

Chapter 11

カスタムドメイン設定

Steps for configuring custom domains when using different domains per cloud.

All 3 cloud custom domains are operational.

Cloud	Custom Domain	Target Endpoint	Status
AWS	`www.aws.ashnova.jp`	`d1qob7569mn5nw.cloudfront.net`	☒ HTTPS active (directly fixed 2026-02-21)
Azure	`www.azure.ashnova.jp`	`nwaj-production-diev0w-f9ekdmehb0bga3aw.z01.azurefd.net`	☒ HTTPS active ⚠ /sns/* needs investigation
GCP	`www.gcp.ashnova.jp`	`34.8.38.222` (A record)	☒ HTTPS active

```
curl -I https://www.aws.ashnova.jp        # 200 OK
curl -I https://www.azure.ashnova.jp      # 200 OK
curl -I https://www.gcp.ashnova.jp        # 200 OK

curl https://www.aws.ashnova.jp/health    # 200 {"status":"healthy"}
curl https://www.gcp.ashnova.jp/health    # 200 {"status":"healthy"}
```

AWS

- ☒ ACM certificate `ISSUED` confirmed → `arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19` (expires 2027-03-12)
- ☒ DNS: CNAME `www.aws.ashnova.jp` → `d1qob7569mn5nw.cloudfront.net` configured
- ☒ CloudFront `E214XONKTXJEJD` alias + ACM certificate set directly (fixed via AWS CLI on 2026-02-21)
  - Issue: `pulumi up` did not apply alias/certificate (remained on CloudFrontDefaultCertificate)
  - Fix: Set alias `www.aws.ashnova.jp` + cert `914b86b1` via `aws cloudfront update-distribution`

- [!] **Note before re-running Pulumi**: Must set `pulumi config set customDomain www.aws.ashnova.jp --stack production` and `pulumi config set acmCertificateArn arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19c4460dcde5 --stack production`

☒ CORS updated

Azure

☒ `az afd custom-domain create` executed (`azure-ashnova-jp`)
☒ Custom domain attached to both routes
☒ DNS: TXT record `_dnsauth.www.azure.ashnova.jp` configured (verified)
☒ DNS: CNAME `www.azure.ashnova.jp` → `mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net` configured
☒ Managed Certificate issued, HTTPS active

- [⚠] `/sns/*` intermittent 502 issue → under investigation (see AZURE_SNS_FIX_REPORT.md)

GCP

☒ Pulumi config set (`customDomain: www.gcp.ashnova.jp`)
☒ `pulumi up --stack production` completed
☒ DNS: A record `www.gcp.ashnova.jp` → `34.8.38.222` configured
☒ Managed SSL certificate `ACTIVE` confirmed

Cloud	Custom Domain	Target Endpoint
AWS	`www.aws.ashnova.jp`	`d1qob7569mn5nw.cloudfront.net`
Azure	`www.azure.ashnova.jp`	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`
GCP	`www.gcp.ashnova.jp`	`34.8.38.222` (A record)

Cloud	Type	Current Endpoint	Distribution ID
AWS	CloudFront	`d1tf3uumcm4bo1.cloudfront.net`	E1TBH4R432SZBZ
Azure	Front Door	`mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`	mcad-staging-d45ihd
GCP	Cloud CDN	`34.117.111.182` (IP address)	-

Cloud	Type	Current Endpoint	Distribution ID
AWS	CloudFront	`d1qob7569mn5nw.cloudfront.net`	E214XONKTXJEJD

Cloud	Type	Current Endpoint	Distribution ID
Azure	Front Door	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`	mcad-production-diev0w
GCP	Cloud CDN	`34.8.38.222` (IP address)	-

Custom domains configured for this project:

- **AWS**: `www.aws.ashnova.jp`
- **Azure**: `www.azure.ashnova.jp`
- **GCP**: `www.gcp.ashnova.jp`

> Note: Generic procedures use placeholders such as `aws.yourdomain.com`. In practice, use the ashnova.jp domains above.

- Domain ownership verified
- AWS Route 53 (recommended) or external DNS provider

```
aws acm request-certificate \
  --domain-name www.aws.ashnova.jp \
  --validation-method DNS \
  --region us-east-1

CERT_ARN=$(aws acm list-certificates \
  --region us-east-1 \
  --query "CertificateSummaryList[?DomainName=='www.aws.ashnova.jp'].CertificateArn" \
  --output text)

echo "Certificate ARN: $CERT_ARN"
```

```
aws acm describe-certificate \
  --certificate-arn $CERT_ARN \
  --region us-east-1 \
  --query 'Certificate.DomainValidationOptions[0].ResourceRecord'
```

Configure in your DNS provider:

- Record type: `CNAME`
- Name: `_abc123.www.aws.ashnova.jp`
- Value: `_xyz456.acm-validations.aws.`

Modify the CloudFront Distribution section in `infrastructure/pulumi/aws/__main__.py`:

```
cert_arn = config.get("acmCertificateArn")  # Retrieved from Pulumi config
custom_domain = config.get("customDomain")  # e.g. aws.yourdomain.com
```

```

cloudfront_kwargs = {
    # ... existing config ...
    "aliases": [custom_domain] if custom_domain else [],
    "viewer_certificate": aws.cloudfront.DistributionViewerCertificateArgs(
        acm_certificate_arn=cert_arn,
        ssl_support_method="sni-only",
        minimum_protocol_version="TLSv1.2_2021",
    ) if cert_arn else aws.cloudfront.DistributionViewerCertificateArgs(
        cloudfront_default_certificate=True,
    ),
    # ... remaining config ...
}
```

```
cd infrastructure/pulumi/aws
pulumi config set customDomain www.aws.ashnova.jp --stack production
pulumi config set acmCertificateArn arn:aws:acm:us-east-1:ACCOUNT_ID:certificate/CERT_ID
    --stack production
pulumi up --stack production
```

Verify the CloudFront domain for your Pulumi environment (production/staging):

```
cd infrastructure/pulumi/aws
pulumi stack select production  # or staging
CLOUDFRONT_DOMAIN=$(pulumi stack output cloudfront_domain)
echo "CloudFront Domain: $CLOUDFRONT_DOMAIN"
```

For Route 53:

```
aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "www.aws.ashnova.jp",
        "Type": "CNAME",
        "TTL": 300,
        "ResourceRecords": [{"Value": "d1qob7569mn5nw.cloudfront.net"}]
      }
    }]
  }'
```

For external DNS providers:

- Record type: `CNAME`
- Name: `www.aws.ashnova.jp`
- Value:
  - Production: `d1qob7569mn5nw.cloudfront.net`
  - Staging: `d1tf3uumcm4bo1.cloudfront.net`

- Domain ownership verified
- Azure DNS (recommended) or external DNS provider

Retrieve environment resource info:

```
cd infrastructure/pulumi/azure
pulumi stack select production  # or staging
FRONTDOOR_HOSTNAME=$(pulumi stack output frontdoor_hostname)
FRONTDOOR_PROFILE=$(pulumi stack output frontdoor_profile_name)
FRONTDOOR_ENDPOINT=$(pulumi stack output frontdoor_endpoint_name)
RESOURCE_GROUP=$(pulumi stack output resource_group_name)

echo "Front Door Hostname: $FRONTDOOR_HOSTNAME"
echo "Profile Name: $FRONTDOOR_PROFILE"
```

Create custom domain:

```
ENVIRONMENT="production"
RESOURCE_GROUP="multicloud-auto-deploy-${ENVIRONMENT}-rg"
PROFILE_NAME="multicloud-auto-deploy-${ENVIRONMENT}-fd"
CUSTOM_DOMAIN_NAME="azure-ashnova-jp"
HOSTNAME="www.azure.ashnova.jp"

az afd custom-domain create \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --host-name $HOSTNAME \
  --certificate-type ManagedCertificate
```

```
az afd custom-domain show \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --query "validationProperties"
```

Configure in your DNS provider:

- Record type: `TXT`
- Name: `_dnsauth.www.azure.ashnova.jp`
- Value: `abc123def456` (validationToken)

```
ENDPOINT_NAME=$(pulumi stack output frontdoor_endpoint_name)
echo "Endpoint Name: $ENDPOINT_NAME"

az afd route create \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
```

```
  --endpoint-name $ENDPOINT_NAME \
  --route-name custom-domain-route \
  --origin-group-name default-origin-group \
  --supported-protocols Https \
  --custom-domains $CUSTOM_DOMAIN_NAME \
  --forwarding-protocol HttpsOnly \
  --https-redirect Enabled
```

Verify the Front Door Hostname:

```
cd infrastructure/pulumi/azure
pulumi stack select production  # or staging
FRONTDOOR_HOSTNAME=$(pulumi stack output frontdoor_hostname)
echo "Front Door Hostname: $FRONTDOOR_HOSTNAME"
```

For Azure DNS:

```
az network dns record-set cname set-record \
  --resource-group YOUR_DNS_RG \
  --zone-name ashnova.jp \
  --record-set-name www.azure \
  --cname $FRONTDOOR_HOSTNAME
```

For external DNS providers:

- Record type: `CNAME`
- Name: `www.azure.ashnova.jp`
- Value:
  - Production: `mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`
  - Staging: `mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`

```
az afd custom-domain show \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --query "{provisioningState, domainValidationState, deploymentStatus}"
```

- Domain ownership verified
- Google Cloud DNS (recommended) or external DNS provider

Modify the SSL certificate section in `infrastructure/pulumi/gcp/__main__.py`:

```
custom_domain = config.get("customDomain")  # e.g. gcp.yourdomain.com

managed_ssl_cert = gcp.compute.ManagedSslCertificate(
    f"{project_name}-{stack}-ssl-cert",
    managed=gcp.compute.ManagedSslCertificateManagedArgs(
        domains=[custom_domain] if custom_domain else ["example.com"],
```

```
    ),
    opts=pulumi.ResourceOptions(
        delete_before_replace=True,
    ),
)
```

```
cd infrastructure/pulumi/gcp
pulumi config set customDomain www.gcp.ashnova.jp --stack production
pulumi up --stack production
```

Note: Managed SSL certificate provisioning can take up to 60 minutes.

Verify the CDN IP address:

```
cd infrastructure/pulumi/gcp
pulumi stack select production  # or staging
CDN_IP=$(pulumi stack output cdn_ip_address)
echo "CDN IP Address: $CDN_IP"
```

For Google Cloud DNS:

```
gcloud dns record-sets create www.gcp.ashnova.jp. \
  --zone=YOUR_ZONE_NAME \
  --type=A \
  --ttl=300 \
  --rrdatas=$CDN_IP
```

For external DNS providers:

- Record type: `A`
- Name: `www.gcp.ashnova.jp`
- Value:
  - Production: `34.8.38.222`
  - Staging: `34.117.111.182`

```
gcloud compute ssl-certificates describe multicloud-auto-deploy-production-ssl-cert-3ee2c3ce \
  --global \
  --format="value(managed.status)"
```

If provisioning takes a long time:

- Verify that the DNS record is correctly configured
- Wait for DNS propagation (up to 48 hours, usually a few minutes to hours)
- Verify DNS resolution with `dig www.gcp.ashnova.jp`

After configuring custom domains, CORS settings for each cloud must be updated.

```
cd infrastructure/pulumi/aws
pulumi config set allowedOrigins "https://www.aws.ashnova.jp,http://localhost:5173" --stack
    production
pulumi up --stack production
```

```
cd infrastructure/pulumi/azure
pulumi config set allowedOrigins "https://www.azure.ashnova.jp,http://localhost:5173" --stack
    production
```

```
cd infrastructure/pulumi/gcp
pulumi config set allowedOrigins "https://www.gcp.ashnova.jp,http://localhost:5173" --stack
    production
pulumi up --stack production
```

```
dig www.aws.ashnova.jp
nslookup www.aws.ashnova.jp

dig www.azure.ashnova.jp
nslookup www.azure.ashnova.jp

dig www.gcp.ashnova.jp
nslookup www.gcp.ashnova.jp
```

```
curl -vI https://www.aws.ashnova.jp
curl -vI https://www.azure.ashnova.jp
curl -vI https://www.gcp.ashnova.jp

openssl s_client -connect www.aws.ashnova.jp:443 -servername www.aws.ashnova.jp < /dev/null
```

```
curl https://www.aws.ashnova.jp
curl https://www.azure.ashnova.jp
curl https://www.gcp.ashnova.jp

curl https://www.aws.ashnova.jp/health
curl https://www.azure.ashnova.jp/api/health
curl https://www.gcp.ashnova.jp/health
```

Problem: SSL certificate error occurs

Solution:

1. Check certificate status
2. Verify that domain aliases are correctly configured
3. Verify that the certificate is associated with CloudFront / Front Door / Cloud CDN

Problem: Domain does not resolve

Solution:

1. Verify that DNS records are correctly configured
2. Wait for DNS propagation (up to 48 hours)
3. Check from Google DNS: `dig @8.8.8.8 www.aws.ashnova.jp`
4. Check TTL values (after changes, wait for old TTL to expire)

Problem: Certificate remains in PROVISIONING state for a long time

Solution:

1. Verify that the DNS A record points to the correct IP address
2. Verify that the load balancer is operating normally
3. Verify that the domain is globally resolvable (run `dig` from multiple locations)

Problem: Custom domain validation fails

Solution:

1. Verify that the TXT record (`_dnsauth`) is correctly configured
2. Verify that validationToken is correct
3. Wait for DNS propagation
4. Verify with `dig TXT _dnsauth.www.azure.ashnova.jp`

Cloud	Additional Cost
AWS	ACM certificate: FreeCloudFront custom domain: Free
Azure	Front Door Managed Certificate: FreeCustom domain: Free
GCP	Managed SSL Certificate: FreeLoad balancer already exists

---

Recommended tasks after custom domain setup:

1. **Update monitoring alerts**: Configure monitoring for new domains
2. **Validate CORS settings**: Access from a browser to verify behavior
3. **Add security headers**: Configure HSTS, CSP, etc.
4. **Configure redirects**: Redirect from old endpoints to new domains
5. **Update documentation**: Add new URLs to README.md

- AWS CloudFront - Custom Domain Setup
- Azure Front Door - Custom Domain
- GCP - Managed SSL Certificates

Chapter 12

ステージング環境テスト

Last updated: 2026-02-23
Applies to: staging environments on AWS, Azure, GCP

This guide describes how to test the **landing page** and **SNS application** across all three cloud staging environments. The test suite covers:

Layer	What is tested
Landing page	CDN delivery, HTML content, HTTPS, cache headers
SNS frontend	React SPA served from CDN (`/sns/`), env-var injection
SNS API	Public endpoints (health, list posts), auth guard, full CRUD
Image upload	Presigned URL generation (S3 / Blob Storage / GCS)

Cloud	CDN URL	API URL
AWS	`https://d1tf3uumcm4bo1.cloudfront.net`	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`
Azure	`https://mcad-staging-d45ihd-dseygrcbcpa3h/gjuzc1.azure-edge.net`	`https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-`
GCP	`https://www.gcp.ashnova.jp`	`https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app`

> **Note**: The Azure API URL (`AZURE_API_URL`) is the Function App base URL **without** the `/api/HttpTrigger` path. The function uses a wildcard route `{*route}` so all API paths (e.g. `/health`, `/posts`) are served directly at the base URL.

Script	Purpose
`scripts/test-staging-all.sh`	☒ **Recommended**: orchestrates all 3 clouds
`scripts/test-landing-pages.sh`	Landing page (/) tests only

Script	Purpose
`scripts/test-e2e.sh`	Lightweight multi-cloud smoke test (v2)
`scripts/test-sns-aws.sh`	Full AWS-only test suite (authenticated)
`scripts/test-sns-azure.sh`	Full Azure-only test suite (authenticated)
`scripts/test-sns-gcp.sh`	Full GCP-only test suite (authenticated)

These checks verify connectivity and public endpoints without any login.

```
./scripts/test-staging-all.sh --quick
```

This checks:

- CDN landing page (/) returns HTTP 200
- SNS app (`/sns/`) returns HTTP 200
- API `/health` returns HTTP 200 with `{"status": "ok"}`

```
./scripts/test-landing-pages.sh
```

Tests per cloud (11 checks each):

- HTTP 200 + response time < 8s
- Content-Type: text/html
- Brand name "Ashnova" present
- Cloud badge (AWS / Azure / GCP) present
- SNS link present
- No `localhost` leak in rendered HTML
- HTTPS transport (where applicable)
- Cache-Control header
- Page size sanity (200B – 2MB)
- No server error markers
- `/sns/` path reachable

```
./scripts/test-e2e.sh
```

Public tests per cloud:

- `GET /health` → 200, `.status == "ok"`
- `GET /posts` → 200, `.items` array present
- `POST /posts` without token → 401 (auth guard)

Authenticated tests require tokens from each cloud's identity provider.

```
AWS_TOKEN=$(aws cognito-idp initiate-auth \
  --auth-flow USER_PASSWORD_AUTH \
  --client-id 1k41lqkds4oah55ns8iod30dv2 \
  --auth-parameters USERNAME=YOUR_EMAIL,PASSWORD=YOUR_PASSWORD \
  --region ap-northeast-1 \
  --query 'AuthenticationResult.AccessToken' \
  --output text)
```

```
GCP_TOKEN=$(gcloud auth print-identity-token)
```

```
AZURE_TOKEN="<paste id_token here>"
```

```
./scripts/test-staging-all.sh \
  --aws-token   "$AWS_TOKEN" \
  --azure-token "$AZURE_TOKEN" \
  --gcp-token   "$GCP_TOKEN"
```

```
./scripts/test-sns-aws.sh --token "$AWS_TOKEN"

./scripts/test-sns-azure.sh --token "$AZURE_TOKEN"

./scripts/test-sns-gcp.sh --token "$GCP_TOKEN"
```

```
./scripts/test-staging-all.sh \
  --aws-token "$AWS_TOKEN" \
  --gcp-token "$GCP_TOKEN" \
  --verbose
```

```
./scripts/test-staging-all.sh \
  --aws-token "$AWS_TOKEN" \
  --skip-cleanup
```

Section	Tests
1	CloudFront returns /, /sns/ with HTML (Content-Type check)
2	API `/health` → 200, `.status == "ok"`
3	Auth guard: POST without token → 401
4	Authenticated: GET profile, POST/GET/PUT post
5	Presigned URLs: count=2 returned, URL starts with `https://...s3`
6	Image key validation: 16 keys → 201; 17 keys → 422
7	Cleanup: DELETE all created posts
8	React SPA env-var sanity: no `localhost` in bundle

Section	Tests
1	Front Door returns `/sns/` with React SPA HTML
2	Function App `/health` → 200, `/posts` → 200 with `.items`
3	AFD SPA deep-link URL Rewrite: `/sns/login`, `/sns/profile`, `/sns/feed` → 200
4	Auth guard: POST without token → 401
5	Authenticated: GET/POST/PUT/DELETE posts, presigned URL
6	Cleanup

Section	Tests
1	Cloud CDN returns `/sns/` with React SPA HTML (no SSR artifacts)
2	Cloud Run `/health` → 200, `/posts` → 200 with `.items`
3	Auth guard: POST/presigned-urls/profile without token → 401
4	Authenticated: GET profile, POST/GET/PUT post
5	GCS presigned URLs: count=2, URL starts with `https://storage.googleapis.com`
6	Image key validation: 16 → 201; 17 → 422
7	Cleanup

See Quick Start section above.

Lightweight smoke test with a consolidated table output. Covers 8 test points per cloud.

All scripts support URL overrides via environment variables:

Variable	Default	Used by
`AWS_CF_URL`	`https://d1tf3uumcm4bo1.cloudfront.net`	test-staging-all, test-landing-pages
`AWS_API_URL`	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`	test-staging-all, test-e2e
`AZURE_FD_URL`	`https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`	test-staging-all, test-landing-pages
`AZURE_API_URL`	`https://multicloud-auto-deploy-staging-func-d8a2guhfercfcqjg.japaneast-01.azurewebsites.net`	test-staging-all, test-e2e
`GCP_CDN_URL`	`https://www.gcp.ashnova.jp`	test-staging-all, test-landing-pages

Variable	Default	Used by
GCP_API_URL	https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app	test-staging-all, test-e2e

Example — test production URLs instead of staging:

```
AWS_CF_URL=https://www.aws.ashnova.jp \
AWS_API_URL=https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com \
GCP_CDN_URL=https://www.gcp.ashnova.jp \
GCP_API_URL=https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app \
./scripts/test-landing-pages.sh
```

---

1. Check if the Cloud Run / Lambda / Function App is stopped:

```
# GCP
gcloud run services describe multicloud-auto-deploy-staging-api \
  --region asia-northeast1 --project ashnova \
  --format="value(status.conditions[0].status)"

# AWS
aws lambda get-function-configuration \
  --function-name multicloud-auto-deploy-staging-api \
  --region ap-northeast-1 --query State

# Azure
az functionapp show \
  --name multicloud-auto-deploy-staging-func \
  --resource-group multicloud-auto-deploy-staging-rg \
  --query state
```

2. See AI_AGENT_08_RUNBOOKS.md for redeploy procedures.

- **AWS**: Token may have expired (1h). Re-run the `aws cognito-idp initiate-auth` command.
- **GCP**: Firebase ID tokens expire in 1 hour. Re-run `gcloud auth print-identity-token`.
- **Azure**: Azure AD tokens expire in 1h. Re-extract from browser DevTools.
- Check that `AUTH_DISABLED=false` is set on the deployment (do **not** set to `true` in staging).
- **GCP**: The `GCP_SERVICE_ACCOUNT` environment variable must be set on Cloud Run. See GCP_SNS_FIX_REPORT_20260223.md (issue G5).
- **AWS**: Verify that the Lambda execution role has `s3:PutObject` on the images bucket.
- **Azure**: Verify that the Function App has the `AZURE_STORAGE_CONNECTION_STRING` env var set.
- The React SPA may not have been built/deployed to the storage bucket.
- Verify that the CI/CD pipeline ran after the last `develop` push:

```
gh run list --workflow deploy-aws.yml --limit 3
gh run list --workflow deploy-gcp.yml --limit 3
gh run list --workflow deploy-azure.yml --limit 3
```

- See AI_AGENT_06_CICD.md for deployment details.

症状: `https://staging.aws.ashnova.jp/` にアクセスすると Chrome のアドレスバーに「保護されていない通信」と表示される。

調査チェックリスト (サーバーサイドが正常かどうか確認する):

```
curl -sI https://staging.aws.ashnova.jp/ | grep -i content-security

curl -sI https://staging.aws.ashnova.jp/ | grep -i strict-transport

curl -sI http://staging.aws.ashnova.jp/ | grep -iE "http/|location"

curl -sI http://multicloud-auto-deploy-staging-frontend.s3-website-ap-northeast-1.amazonaws.com/
```

原因の切り分け:

DevTools > Security タブの表示	原因	対処
`chrome-exte...` が Secure origins に表示されている	Chrome 拡張機能が証明書エラーのあるリソースを注入している	拡張機能を1つずつオフにして特定
“You have recently allowed content with certificate errors”	過去にブラウザがこのサイトで証明書エラーの許可をした記録が残っている	下記の「ブラウザキャッシュのクリア手順」参照
Mixed Content の `http://` URLが表示される	サーバーサイドに http:// リソースが残っている	JS バンドル・API レスポンス・DBを調査

ブラウザキャッシュのクリア手順 (“recently allowed certificate errors” の場合):

1. `chrome://settings/content/siteDetails?site=https%3A%2F%2Fstaging.aws.ashnova.jp` を開く
2. 「権限をリセット」と「データを消去」を実行
3. **ブラウザを完全に再起動** (タブを閉じるだけでは不十分 — メモリキャッシュが残る)
4. `https://staging.aws.ashnova.jp/` に再アクセス

> **重要**: サイトデータを消去しただけでは解消しない場合がある。Chrome はこの許可状態をメモリにキャッシュするため、ブラウザ再起動が必要。

シークレットモードで警告が消える場合: サーバーサイドは正常。ブラウザの状態 (拡張機能またはキャッシュ) が原因。

GCS objects are served via Cloud CDN with cache TTL. Force invalidation:

```
gcloud compute url-maps invalidate-cdn-cache multicloud-auto-deploy-staging-lb \
  --path "/*" --project=ashnova --async
```

See GCP_SNS_FIX_REPORT_20260223.md (issue G3).

The Front Door Rule Set rewrites `/sns/*` → `/sns/index.html`. If this fails:

1. Open Azure Portal → Front Door → Rule Sets → check the rewrite rule.
2. Verify the Rule Set is associated with the route.

The test scripts can be wired into GitHub Actions as a post-deploy verification step:

```
- name: Run staging smoke tests (public)
  run: |
    ./scripts/test-staging-all.sh --quick

- name: Run full staging tests (authenticated)
  if: secrets.AWS_TEST_TOKEN != ''
  env:
    AWS_TOKEN: ${{ secrets.AWS_TEST_TOKEN }}
    AZURE_TOKEN: ${{ secrets.AZURE_TEST_TOKEN }}
    GCP_TOKEN: ${{ secrets.GCP_TEST_TOKEN }}
  run: |
    ./scripts/test-staging-all.sh \
      --aws-token   "$AWS_TOKEN" \
      --azure-token "$AZURE_TOKEN" \
      --gcp-token   "$GCP_TOKEN"
```

Document	Description
AI_AGENT_07_STATUS.md	Current health of all 3 environments
AI_AGENT_04_API.md	API endpoint spec and request/response schema
AI_AGENT_08_RUNBOOKS.md	Step-by-step deploy / redeploy procedures
GCP_SNS_FIX_REPORT_20260223.md	GCP bug fixes (G1–G6)
AWS_SNS_FIX_REPORT_20260222.md	AWS fixes (12 bugs)
AZURE_SNS_FIX_REPORT.md	Azure fixes

Chapter 13

統合テストガイド

Created: 2026-02-18
Version: 1.0.0
Target Environments: All (AWS/GCP/Azure)

---

1. Overview
2. Test Structure
3. How to Run
4. Test Coverage
5. Troubleshooting
6. CI/CD Integration

The integration tests in this project comprehensively test the backend implementations across **3 cloud providers (AWS/GCP/Azure)**.

Test Type	Description	Tool
Unit Tests	Tests for individual backend class methods	pytest (mocked)
Integration Tests	Full-flow tests for CRUD operations	pytest (mocked)
API Endpoint Tests	HTTP tests against actually deployed APIs	pytest + requests
E2E Tests	End-to-end tests across all clouds	bash + curl

```
services/api/ ├──
 tests/ │
    ├── __init__.py                    # Test package initialization │
    ├── conftest.py                    # pytest configuration and fixtures │
```

```
    ├── test_backends_integration.py   # Backend integration tests │
    └── test_api_endpoints.py          # API endpoint tests ├──
 pytest.ini                          # pytest config file └──
 requirements-dev.txt                # Development/test dependencies

scripts/ ├──
 run-integration-tests.sh            # Python test runner script (NEW) ├──
 test-api.sh                         # Single API HTTP test ├──
 test-e2e.sh                         # Multi-cloud E2E test └──
 test-endpoints.sh                   # Endpoint health check
```

Features:

- Fixture definitions for testing
- Mock user credentials
- Sample data generation
- Cleanup processing

Key Fixtures:

```
test_user()              # Regular user
admin_user()             # Admin user
another_user()           # Different user
sample_post_body()       # Data for creating posts
sample_update_body()     # Data for updating posts
sample_profile_update()  # Data for updating profiles
aws_config()             # AWS configuration
gcp_config()             # GCP configuration
azure_config()           # Azure configuration
```

Test Classes:

Test cases common to all backends:

- ☒ `test_backend_initialization()` - Backend initialization
- ☒ `test_create_post_success()` - Create post
- ☒ `test_list_posts_empty()` - List posts (empty)
- ☒ `test_list_posts_with_tag_filter()` - Tag filtering
- ☒ `test_update_post_success()` - Update post
- ☒ `test_update_post_permission_denied()` - Permission error (update)
- ☒ `test_update_post_admin_can_update()` - Admin permission (update)
- ☒ `test_delete_post_success()` - Delete post
- ☒ `test_delete_post_permission_denied()` - Permission error (delete)
- ☒ `test_delete_post_admin_can_delete()` - Admin permission (delete)
- ☒ `test_get_profile_not_found()` - Get profile (not found)

- ☒ `test_update_profile_success()` - Update profile
- ☒ `test_get_profile_after_update()` - Get profile after update
- ☒ `test_generate_upload_urls()` - Generate upload URLs
- DynamoDB + S3 mock tests
- Marker: `@pytest.mark.aws`
- Firestore + Cloud Storage mock tests
- Marker: `@pytest.mark.gcp`
- Cosmos DB + Blob Storage mock tests
- Marker: `@pytest.mark.azure`

Test Classes:

Tests against actually deployed API endpoints:

- ☒ `test_health_check()` - Health check
- ☒ `test_list_messages_initial()` - Fetch message list
- ☒ `test_crud_operations_flow()` - Full CRUD flow
- ☒ `test_pagination()` - Pagination
- ☒ `test_invalid_message_id()` - Invalid ID (404 error)
- ☒ `test_empty_content_validation()` - Validation error

Reference: `scripts/test-api.sh` test cases 1-12

- ☒ `test_all_cloud_health_checks()` - Health check across all clouds

Reference: `scripts/test-endpoints.sh`

- ☒ `test_response_format_consistency()` - Response format consistency
- ☒ `test_api_version_consistency()` - API version consistency

Reference: consistency checks in `scripts/test-e2e.sh`

```
cd services/api
pytest tests/
```

```
pytest tests/ -m aws
```

```
pytest tests/ -m gcp
```

```
pytest tests/ -m azure
```

```
pytest tests/ -vv
```

```
pytest tests/ --cov=app --cov-report=html
```

```
pytest tests/ -k "test_create_post"
```

```
./scripts/run-integration-tests.sh
```

```
./scripts/run-integration-tests.sh -v
```

```
./scripts/run-integration-tests.sh -m aws
```

```
export AWS_API_ENDPOINT="https://abc123.execute-api.ap-northeast-1.amazonaws.com"
export GCP_API_ENDPOINT="https://app-xyz.a.run.app"
export AZURE_API_ENDPOINT="https://func-xyz.azurewebsites.net/api/HttpTrigger"

./scripts/run-integration-tests.sh --endpoints
```

```
./scripts/run-integration-tests.sh --coverage
```

```
./scripts/test-api.sh -e https://your-api-endpoint.com
```

```
./scripts/test-e2e.sh
```

```
./scripts/test-endpoints.sh
```

Method	AWS	GCP	Azure	Tests
`list_posts()`	☒	☒	☒	3
`create_post()`	☒	☒	☒	3
`update_post()`	☒	☒	☒	9
`delete_post()`	☒	☒	☒	9
`get_profile()`	☒	☒	☒	3
`update_profile()`	☒	☒	☒	6
`generate_upload_urls()`	☒	☒	☒	3

Total: 108 test cases (36 cases × 3 clouds)

Endpoint	Method	Test	Reference Script
/	GET	Health check	test-api.sh #1
/api/messages/	GET	List retrieval	test-api.sh #2, #4

Endpoint	Method	Test	Reference Script
`/api/messages/`	POST	Create	test-api.sh #3
`/api/messages/{id}`	GET	Single retrieval	test-api.sh #5
`/api/messages/{id}`	PUT	Update	test-api.sh #6, #7
`/api/messages/{id}`	DELETE	Delete	test-api.sh #8, #9
`/api/messages/?page=1`	GET	Pagination	test-api.sh #10
`/api/messages/invalid-id`	GET	Error 404	test-api.sh #11
`/api/messages/`	POST	Validation error	test-api.sh #12

Total: 27 endpoint tests (9 endpoints × 3 clouds)

Markers for classifying and filtering tests:

Marker	Description	Example
`@pytest.mark.aws`	AWS-specific tests	`pytest -m aws`
`@pytest.mark.gcp`	GCP-specific tests	`pytest -m gcp`
`@pytest.mark.azure`	Azure-specific tests	`pytest -m azure`
`@pytest.mark.integration`	Integration tests	`pytest -m integration`
`@pytest.mark.unit`	Unit tests	`pytest -m unit`
`@pytest.mark.slow`	Slow-running tests	`pytest -m "not slow"`
`@pytest.mark.requires_network`	Requires network	`pytest -m requires_network`
`@pytest.mark.requires_credentials`	Requires credentials	Excluded by default

Solution:

```
pip install pytest pytest-mock pytest-asyncio requests
```

Solution:

```
cd /workspaces/ashnova/multicloud-auto-deploy/services/api
pytest tests/
```

Solution:

```
pip install pytest-mock
```

Cause: Endpoint not configured or not yet deployed

Solution:

```
export AWS_API_ENDPOINT="https://your-endpoint.com"

pytest tests/ -m "not requires_network"
```

Cause: Test script lacks execute permission

Solution:

```
chmod +x scripts/run-integration-tests.sh
```

Example `.github/workflows/test.yml`:

```
name: Integration Tests

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          cd services/api
          pip install -r requirements.txt
          pip install pytest pytest-mock pytest-asyncio requests

      - name: Run integration tests
        run: |
          ./scripts/run-integration-tests.sh -v

      - name: Run endpoint tests (if deployed)
        if: env.AWS_API_ENDPOINT != ''
        env:
          AWS_API_ENDPOINT: ${{ secrets.AWS_API_ENDPOINT }}
          GCP_API_ENDPOINT: ${{ secrets.GCP_API_ENDPOINT }}
          AZURE_API_ENDPOINT: ${{ secrets.AZURE_API_ENDPOINT }}
```

```
        run: |
          ./scripts/run-integration-tests.sh --endpoints
```

```
./scripts/run-integration-tests.sh -v

./scripts/run-integration-tests.sh --coverage
```

```
cd services/api

pytest tests/ -v
```

```
./scripts/run-integration-tests.sh -m aws -v
```

```
export AWS_API_ENDPOINT="https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com"

./scripts/run-integration-tests.sh --endpoints -v
```

```
./scripts/run-integration-tests.sh --coverage
```

---

1. **Quality Assurance**: Ensure all backends operate according to specifications
2. **Regression Prevention**: Detect unintended behavior changes during code modifications
3. **Documentation**: Test code serves as usage examples for the implementation
4. **CI/CD Integration**: Automated tests perform quality checks before deployment

- ☒ **Run tests before committing**: `./scripts/run-integration-tests.sh`
- ☒ **Add tests when adding new features**: Write corresponding tests for new methods
- ☒ **Run endpoint tests after deployment**: `./scripts/run-integration-tests.sh --endpoints`
- ☒ **Run E2E tests regularly**: `./scripts/test-e2e.sh`
- ☒ **Maintain 90%+ coverage**: Check with `--coverage`

☐ Add performance tests (`TestBackendPerformance`)

☐ Add end-to-end workflow tests (`TestEndToEnd`)

☐ Load testing (Locust, etc.)

☐ Security testing (authentication & authorization)

☐ Chaos engineering tests (failure simulation)

Author: GitHub Copilot
Last Updated: 2026-02-18
Related Documents:

- API_OPERATION_VERIFICATION_REPORT.md
- AWS_BACKEND_COMPLETE_FIX_REPORT.md
- BACKEND_IMPLEMENTATION_INVESTIGATION.md

Chapter 14

Lambda Layer最適化

To reduce Lambda function deployment size, we use Lambda Layers to separate dependencies from application code.

☒ **Recommended: Using Klayers (public Lambda Layers) enables even more efficient deployments!**

See LAMBDA_LAYER_PUBLIC_RESOURCES.md for details.

- **Package size**: Over 50MB (including all dependencies)
- **Deployment method**: Upload via S3 required
- **Deployment time**: Slow due to S3 upload + Lambda update
- **Inefficient**: Every deployment re-uploads even when dependencies haven't changed
- **Package size**: A few MB (application code only)
- **Deployment method**: Direct upload possible (under 50MB)
- **Deployment time**: Completes in seconds
- **Efficient**: Dependencies managed in Layer; only update when code changes

```
Lambda Function (lightweight) ├──
 app/              # Application code (~2-5MB) │
    ├── main.py │
    ├── auth.py │
    ├── config.py │
    └── ... └──
 index.py

Lambda Layer (dependencies) └──
 python/           # Dependencies (~20-40MB)
     ├── fastapi/
     ├── pydantic/
     ├── mangum/
     ├── jwt/
```

```
    └──── ...
```

Benefits:

- ☒ No build required (instantly deployable)
- ☒ No maintenance needed (community managed)
- ☒ Easy to update to latest version

```

cd infrastructure/pulumi/aws/simple-sns
pulumi config set use_klayers true
pulumi up
```

See LAMBDA_LAYER_PUBLIC_RESOURCES.md for details.

Benefits:

- ☒ Full control (use specific versions)
- ☒ Size optimization (only what's needed)
- ☒ Use in private environments

```
cd /workspaces/ashnova/multicloud-auto-deploy
./scripts/build-lambda-layer.sh
```

This script performs the following:

- Install only AWS-specific dependencies
- Exclude boto3/botocore (included in Lambda runtime)
- Exclude Azure/GCP SDKs (not needed for AWS)
- Remove test files and documentation to reduce size
- Generate `services/api/lambda-layer.zip`

```
cd infrastructure/pulumi/aws/simple-sns

pulumi config set use_klayers false

pulumi up
```

Pulumi will automatically:

- Create the custom Lambda Layer
- Deploy only application code to the Lambda function
- Attach the Layer to the Lambda function

```
gh workflow run deploy-aws.yml
```

The GitHub Actions workflow automatically:

1. **Fetch ARN**: Uses the latest Klayers ARN
2. **Package application code**: ZIPs code only
3. **Update Lambda function**:
   - Package under 50MB: Direct upload ☒
   - Package over 50MB: Via S3 (fallback)
4. **Attach Klayers**: Connect the public Layer to Lambda

```
name: Deploy
on:
  workflow_dispatch:
    inputs:
      use_klayers:
        description: "Use Klayers (public Lambda Layers)"
        default: true # Use Klayers by default
```

Select `use_klayers: false` in the GitHub Actions workflow:

1. **Build Layer**: Run `build-lambda-layer.sh`
2. **Package application code**: ZIPs code only
3. **Deploy custom Layer**: Publish as Lambda Layer
4. **Update Lambda function**: Direct upload or via S3
5. **Attach Layer**: Connect custom Layer to Lambda

```
fastapi==0.115.0
pydantic==2.9.0
pydantic-settings==2.5.2
python-jose[cryptography]==3.3.0
python-multipart==0.0.9
pyjwt==2.9.0
mangum==0.17.0
requests==2.32.3
```

- **boto3/botocore**: Included in Lambda runtime
- **Azure SDK**: Not required for AWS
- **GCP SDK**: Not required for AWS
- **PostgreSQL/SQLAlchemy**: DynamoDB only is used

```
Lambda package: 65MB (code + all dependencies) └──
 S3 deployment required
```

```
Lambda package: 3MB (code only) ├──
 Direct upload possible └──
 Layer: 25MB (dependencies)
     └── Re-deploy only when dependencies change
```

```
./scripts/build-lambda-layer.sh

```

```
ls -lh services/api/lambda-layer.zip
```

```
aws lambda get-function-configuration \
  --function-name multicloud-auto-deploy-staging-api \
  --query 'Layers[*].Arn'
```

Lambda Layer limits:

- **ZIP size**: 50MB
- **Unzipped size**: 250MB

Size reduction methods:

1. Remove unnecessary files in `build-lambda-layer.sh`
2. Use `--no-deps` to exclude extra dependencies
3. Remove test files and documentation

- Pin dependency versions
- Only update Lambda when application code changes
- Only update Layer when dependencies change

```
aws lambda list-layer-versions \
  --layer-name multicloud-auto-deploy-staging-dependencies

aws lambda delete-layer-version \
  --layer-name multicloud-auto-deploy-staging-dependencies \
  --version-number 1
```

```
LAYER_NAME: multicloud-auto-deploy-staging-dependencies

LAYER_NAME: multicloud-auto-deploy-production-dependencies
```

- Lambda Layer Public Resources Guide ☒ **Recommended**
- Klayers GitHub
- Klayers API
- AWS Lambda Layers Documentation
- Lambda Deployment Packages
- Lambda Quotas

The `scripts/build-lambda-layer.sh` has been fully optimized for CI/CD environments:

Enhancements (2026-02-27):

1. **pip Output Suppression**
   - Added `--disable-pip-version-check` to suppress version warnings
   - Added `--quiet` flag to reduce noise in CI logs
   - Proper error handling with detailed failure messages

2. **ZIP Compression Optimization**
   - Changed `zip -r9` to `zip -r9q` (quiet mode)
   - Reduces log output while maintaining compression level 9
   - Error handling ensures ZIP creation succeeds
3. **Cross-Platform Compatibility**
   - Fixed `stat` command for both Linux and macOS
   - Fallback to `du` if stat fails
   - No platform-specific code paths
4. **GitHub Actions Integration**
   - Exports `GITHUB_OUTPUT` variables for subsequent steps:
     - `layer_size_mb`: Layer size in megabytes
     - `layer_zip_path`: Full path to created ZIP
   - Enables conditional workflows based on layer size

`.github/workflows/deploy-aws.yml` enhancements:

1. **Build Lambda Layer Step**

```
- name: Build Lambda Layer
  id: build_layer
  run: |
    bash scripts/build-lambda-layer.sh
    # Exports layer_created=true on success
```

2. **Deploy Lambda Layer Step**
   - Enhanced error handling for `publish-layer-version`
   - Validates ZIP file existence before publishing
   - Proper ARN extraction with null-check
   - Clear error messages on failure
3. **Cleanup Old Versions Step**
   - Uses `continue-on-error: true` to allow graceful failure
   - Improved version sorting (descending, keeps latest 3)
   - Better output formatting and reporting
   - Suppress AWS CLI warnings while showing essential info

Before Optimization:

- Build time: ~3-5 minutes (pip with verbose output)
- Log size: ~20-50 MB (noisy with warnings)
- Deploy time: ~2-3 minutes

After Optimization (Task 12):

- Build time: ~2-3 minutes (quiet pip)
- Log size: ~1-2 MB (clean output)
- Deploy time: ~1-2 minutes
- **Overall reduction: ~40-50% faster CI/CD**

☒ **We strongly recommend using Klayers (public Layers)**

By using Lambda Layers:

☒ Reduces deployment size from **65MB → 3MB** ☒ Reduces deployment time from **minutes → seconds** ☒ S3 upload **no longer required** ☒ Dependency management is **separated** for efficiency ☒ Build time is **zero** (when using Klayers)

Selection Criteria:

Scenario	Recommended Approach
Standard development / production	**Klayers** ☒
Rapid prototyping	**Klayers** ☒
Specific version required	Custom Layer
Private dependencies	Custom Layer
Minimize size to the extreme	Custom Layer

See Lambda Layer Public Resources Guide for details.

Chapter 15

アーキテクチャマッパー

This tool collects current resource information from AWS, Azure, and GCP, normalizes it into JSON, and renders a Mermaid system architecture diagram.

- Build a single architecture snapshot from live cloud resources
- Cross-check Pulumi expected state vs actual cloud resources
- Generate a portable diagram artifact (`.mmd`) for documentation and reviews
- `scripts/cloud_architecture_mapper.py`
- `scripts/refresh-architecture.sh`
- Pulumi CLI authenticated (`pulumi login`)
- AWS CLI authenticated (`aws sts get-caller-identity`)
- Azure CLI authenticated (`az account show`)
- GCP CLI authenticated (`gcloud auth list`)
- Access to stack outputs for:
  - `infrastructure/pulumi/aws`
  - `infrastructure/pulumi/azure`
  - `infrastructure/pulumi/gcp`

```
python3 scripts/cloud_architecture_mapper.py collect \
  --environment staging \
  --output docs/generated/architecture/snapshot.json
```

```
python3 scripts/cloud_architecture_mapper.py render \
  --input docs/generated/architecture/snapshot.json \
  --output docs/generated/architecture/architecture.mmd

python3 scripts/cloud_architecture_mapper.py render \
  --input docs/generated/architecture/snapshot.json \
  --output docs/generated/architecture/architecture.mmd \
  --validate
```

```
python3 scripts/cloud_architecture_mapper.py all \
  --environment staging \
  --snapshot-output docs/generated/architecture/snapshot.json \
  --diagram-output docs/generated/architecture/architecture.mmd \
  --validate
```

```
./scripts/refresh-architecture.sh staging
```

```
python3 scripts/cloud_architecture_mapper.py collect \
  --environment staging \
  --output docs/generated/architecture/snapshot.staging.json

python3 scripts/cloud_architecture_mapper.py collect \
  --environment production \
  --output docs/generated/architecture/snapshot.production.json

python3 scripts/cloud_architecture_mapper.py compare \
  --staging-input docs/generated/architecture/snapshot.staging.json \
  --production-input docs/generated/architecture/snapshot.production.json \
  --output docs/generated/architecture/architecture-combined.mmd \
  --validate
```

- `architecture: refresh (staging)`
- `architecture: validate (render)`
- `docs/generated/architecture/snapshot.json`
  - `clouds.<provider>.pulumiOutputs`: Pulumi stack outputs
  - `clouds.<provider>.resources`: normalized resources
  - `clouds.<provider>.errors`: collection warnings/errors
- `docs/generated/architecture/architecture.mmd`
  - Mermaid `flowchart LR` diagram with provider subgraphs
- AWS: CloudFront, S3, API Gateway v2, Lambda, DynamoDB
- Azure: Front Door profile, Storage Account, Function App, Cosmos DB
- GCP: Backend Bucket / URL Map, Cloud Run, GCS, Firestore
- The collector is intentionally read-only and uses list/get APIs.
- If one cloud collection fails, the tool still writes snapshot/diagram with warnings.
- `--validate` checks Mermaid node ID safety and edge references, and exits with non-zero status when invalid.
- This is an MVP foundation for future diff and drift analysis.

`scripts/generate_icon_diagram.py` generates interactive HTML architecture diagrams with official cloud service icons embedded. This tool enhances the basic Mermaid diagrams with:

- **Official SVG icons** from AWS, Azure, and GCP
- **Interactive flowchart nodes** with icons embedded in-node (top-left corner)
- **Base64-encoded data URIs** for zero external dependencies
- **Responsive sidebar legend** with resource counts
- ☒ Official AWS Architecture Icons (48px SVG, Q3-2021 package)
- ☒ Official Azure Public Service Icons V14 (18px SVG with gradients)
- ☒ Official GCP Product Icons (SVG)
- ☒ JavaScript icon injection into Mermaid-rendered nodes
- ☒ **Dual icon placement**: Top-left corner (24px) + Text-inline (20px)
- ☒ No external CDN dependencies (all icons embedded)
- ☒ Separate diagrams for staging, production, and combined view

Icons are stored in `assets/icons/{provider}/`:

```
assets/icons/ ├──
 aws/ │
    ├── cloudfront.svg      (AWS CloudFront) │
    ├── lambda.svg          (AWS Lambda) │
    ├── s3.svg              (Amazon S3) │
    ├── dynamodb.svg        (Amazon DynamoDB) │
    └── api-gateway.svg     (Amazon API Gateway) ├──
 azure/ │
    ├── cdn.svg             (Azure Front Door) │
    ├── function.svg        (Azure Functions) │
    ├── storage.svg         (Azure Storage Account) │
    └── cosmos-db.svg       (Azure Cosmos DB) └──
 gcp/
     ├── cdn.svg             (Cloud CDN)
     ├── run.svg             (Cloud Run)
     ├── storage.svg         (Cloud Storage)
     ├── firestore.svg       (Cloud Firestore)
     └── load-balancer.svg   (Cloud Load Balancer)
```

Total size: ~50KB (14KB AWS + 16KB Azure + 20KB GCP) **Format**: SVG (embedded as Base64 data URIs in HTML) **Source**:

- AWS: AWS Architecture Icons Asset Package
- Azure: Azure Public Service Icons V14
- GCP: Google Cloud Icons

```
python3 scripts/generate_icon_diagram.py staging
```

```

python3 scripts/generate_icon_diagram.py production

python3 scripts/generate_icon_diagram.py combined
```

Generated HTML files in `docs/generated/architecture/`:

- `architecture.staging.html` (~85KB) - Staging environment with official icons
- `architecture.production.html` (~85KB) - Production environment with official icons
- `architecture-combined.html` (~91KB) - Staging + Production overlay with color-coded nodes

Each HTML file includes:

- Embedded Mermaid.js v10 (ESM module)
- Base64-encoded SVG icons (no external requests)
- JavaScript DOM manipulation for icon injection
- Interactive legend sidebar with resource counts
- Responsive layout (CSS Grid)

Icon Loading:

```python
def load_official_icon(provider: str, resource_type: str) -> str:
    """Load official SVG icon and return Base64 data URI"""
    # Icon mappings for AWS, Azure, GCP
    # Returns: ...
```

Icon Injection (JavaScript):

```javascript
// Pattern: flowchart-{provider}_{resource_type}-{name}-{index}
const nodeId = node.id;
const match = nodeId.match(/flowchart-([a-z]+)_([a-z_]+)-/);
if (match && resourceIcons[`${match[1]}:${match[2]}`]) {
  // 1. Top-left corner icon (24x24px at x+6, y+6)
  const topIcon = document.createElementNS(
    "http://www.w3.org/2000/svg",
    "image",
  );
  topIcon.setAttribute("x", rectX + 6);
  topIcon.setAttribute("y", rectY + 6);
  topIcon.setAttribute("width", 24);
  topIcon.setAttribute("height", 24);
  node.appendChild(topIcon);

  // 2. Text-inline icon (20x20px, 4px left of text)
  const textIcon = document.createElementNS(
    "http://www.w3.org/2000/svg",
    "image",
  );
  textIcon.setAttribute("x", textX - 24); // 4px left of text
```

```
  textIcon.setAttribute("y", textY);
  textIcon.setAttribute("width", 20);
  textIcon.setAttribute("height", 20);
  labelGroup.insertBefore(textIcon, labelGroup.firstChild);
}
```

Dual Icon Placement:

- **Top-left corner**: 24x24px icon at (rect.x + 6, rect.y + 6) for quick resource type identification
- **Text-inline**: 20x20px icon placed 4px left of node label text for enhanced readability
- Both icons use the same provider-specific official SVG

Node ID Convention:

- Pattern: `flowchart-{provider}_{resource_type}-{name}-{index}`
- Examples: `flowchart-aws_cdn-E1TBH4R432SZBZ-0`, `flowchart-azure_compute-mcadfuncdiev0w-1`
- Provider: `aws` | `azure` | `gcp`
- Resource types: `cdn`, `object_storage`, `serverless_function`, `database`, `api_gateway`, `load_balancer`, `compute`
- Icons assume 48x48px source size (scaled to 24px top-left, 20px text-inline)
- Injection timing requires 800ms setTimeout for Mermaid rendering
- No icon caching (data URIs embedded in each HTML file)
- Resource type mapping is manually maintained in `load_official_icon()`
- Text-inline icon positioning may vary depending on Mermaid node type (foreignObject vs text elements)

The tool can identify isolated/orphaned resources (nodes with no connections). As of 2026-02-27:

13 isolated resources identified across staging:

- ☒ **Delete candidates** (5): `multicloud-auto-deploy-staging-posts` (AWS DynamoDB), 4 Azure Storage Accounts not in Pulumi
- ☒ **In use but unconnected** (2): GCP function-source bucket, uploads bucket
- ☒ **Needs verification** (6): 3 AWS CloudFront distributions for ashnova.jp domains, GCP landing bucket, 2 others

> **Note**: Resource cleanup postponed pending further investigation (2026-02-27)